

Instalando o JDK

- Distribuição de código aberto: Corretto é uma versão gratuita e de código aberto do JDK, mantida pela Amazon;
- Suporte a longo prazo (LTS): Corretto 21 oferece atualizações regulares de segurança e desempenho com suporte garantido pela Amazon;
- Compatível com múltiplas plataformas: Funciona em várias plataformas, incluindo Windows, macOS e Linux.
- Através dele é possível executar o Java de forma fácil na nossa máquina;

Instalando o JDK

```
C:\> Prompt de Comando
Microsoft Windows [versão 10.0.19045.5487]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Carlos> java --version
openjdk 21.0.6 2025-01-21 LTS
OpenJDK Runtime Environment Corretto-21.0.6.7.1 (build 21.0.6+7-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.6.7.1 (build 21.0.6+7-LTS, mixed mode, sharing)
```

(comando para checar versão)

Primeiro programa

- Para criar nosso primeiro programa vamos precisar de um arquivo .java em uma pasta no nosso computador;
- O código todo fica dentro de uma classe (Class);
- A execução das instruções ficam em um método chamado main;
- Utilizaremos o método `println` para exibir uma mensagem;
- Através da opção Run conseguimos rodar/executar o programa;
- Vamos lá!



Primeiro Programa

A princípio vamos ignorar as **class** e o método **main**, uma vez que estamos começando e estes são conceitos aplicados a orientação a objetos ou **poo**

```
// Nome do pacote, referente a pasta raiz
package secao1;
// Nome da classe, referente ao nome do arquivo
public class HelloWorld {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        System.out.println("Olá, Mundo!");
        // As instruções precisam de ; no final!

    }
}
```

Como mencionado previamente as instruções serão mais detalhadas depois, estas são as formas de rodar o código (botão direito no arquivo e **run java** também funciona)



Considerações gerais

- Java precisa de ponto e vírgula no final de cada linha;
- É uma linguagem case sensitive, ou seja: println != PrintLn;
- Por ser baseada em orientação a objetos, tem diversos conceitos que utilizaremos de forma abstrata por enquanto (Class, main, System);

Considerações Gerais

```

public class HelloWorld {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        System.out.println(x:"que bagunça da p####a");

    }
}

```

Note como a classe **System** possui a primeira letra maiúscula tal como o nome de nossa classe **HelloWorld**

Executando código pelo terminal

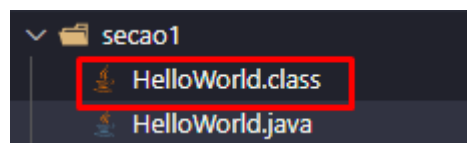
- Primeiramente precisamos compilar o arquivo, utilizando o comando: `javac <nome>;`
- Depois, para executá-lo, temos o comando: `java <nome_arquivo_compilado>;`
- Obs: dependendo da estrutura de pastas a execução pode ficar um pouco diferente (secao1.HelloWorld), e não precisamos colocar a extensão `.java` ou `.class`;
- Isso faz a mesma função de executar no VS Code, e é o processo natural de execução sem uma IDE/Editor de código;
- Note que a cada alteração precisamos compilar o arquivo novamente, vamos lá!



Executando código pelo terminal

```
javac .\HelloWorld.java
```

O comando gera um arquivo **.class** com o mesmo nome da classe compilada!



Com esse comando podemos rodar o arquivo `.class`, note que tivemos de subir um diretório acima (`..`)

```
\Java HdC> java secao1.HelloWorld
```

```

PS C:\Users\carlos\Documents>
que bagunça da p####a
PS C:\Users\Carlos\Documents>

```

Código do Curso

- Os arquivos do curso podem ser encontrados em:
https://github.com/matheusbattisti/curso_java
- Faça o download para seguir o curso e ter o código como referência;
- O código está estruturado da mesma maneira passada em aula;

[Aqui](#)

O que são variáveis?

- Variáveis são espaços na memória que armazenam valores que podem ser manipulados pelo programa;
 - Cada variável possui um nome único que identifica o dado armazenado;
 - Variáveis têm tipos específicos (como int, String, boolean) que definem o tipo de valor que podem armazenar;
 - O valor armazenado em uma variável pode ser alterado durante a execução do programa;
- Variáveis permitem a criação de lógicas dinâmicas, armazenando e manipulando informações ao longo do código;

Variáveis

```
// 1 - O que são variáveis  
// Tipo > Nome > Valor  
String nome = "Fulano";  
  
System.out.println(nome);
```

Fulano

Algumas regras para variáveis

- A sintaxe para declarar variável é: TIPO DE DADO NOME = VALOR;
- O formato de nomes é lowerCamelCase, ou seja: nomeCompleto;
- Os tipos de dados variam conforme a necessidade da variável no programa;
- Cada tipo de dado ocupa um tamanho na memória (tabela na próxima aula), devemos sempre otimizar isso;
- O nome precisa ser único, não pode começar com números;
- Regra para otimização: escolha sempre a que ocupa menos memória;
- O sinal de igual é chamado de atribuição;

Algumas regras para variáveis

Tipos de dados

- Veja abaixo os tipos de dados em Java:

DATA TYPES	SIZE	DEFAULT	EXPLANATION
boolean	1 bit	false	Stores true or false values
byte	1 byte/ 8bits	0	Stores whole numbers from -128 to 127
short	2 bytes/ 16bits	0	Stores whole numbers from -32,768 to 32,767
int	4 bytes/ 32bits	0	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes/ 64bits	0L	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes/ 32bits	0.0f	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes/ 64bits	0.0d	Stores fractional numbers. Sufficient for storing 15 decimal digits
char	2 bytes/ 16bits	'\u0000'	Stores a single character/letter or ASCII values

Tipos de Dados (Primitivos)

Esses são apenas os tipos primitivos, strings e arrays serão vistos depois!

Atribuição de valor com outra variável

- O valor de uma variável pode ser utilizado para atribuir o valor de outra variável;
- Ou seja, iniciamos uma segunda variável com o valor de uma primeira, isso é aceito em Java;
- Outra consideração importante são os limites dos tipos de dados, que foram passados na tabela da aula passada;
- Vamos explorar estes dois cenários!



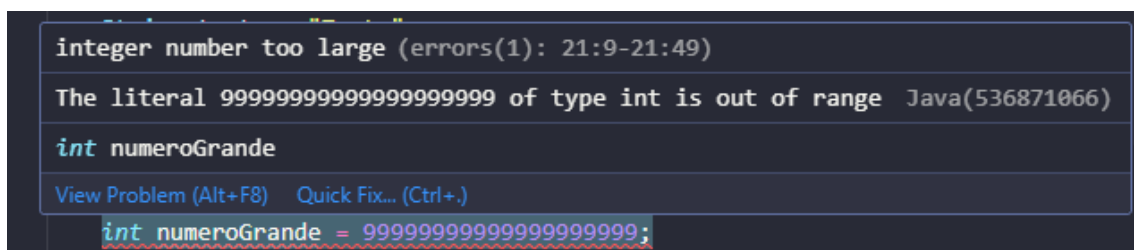
Atribuição de valor com outra variável

```
// 2 - Atribuição de variável com outra  
String teste = "Teste";  
  
String testando = teste;  
  
System.out.println(testando);
```

Teste

E

```
int numeroGrande = 999999999999999999;
```



Esse valor ultrapassa o limite de **int** visto na tabela anteriormente

```
Long numeroGrande = 999999999999999999l;
```

Long aceitaria esse tipo de número (com ajustes rs) note que precisamos por um 'L' no final do número!

```
Long numeroGrande = 9999999999999999991;  
int numeroGrande2 = numeroGrande;
```

O Java não permite a atribuição de uma variável a outra quando ela foge do escopo de suas funcionalidades, **int** não poderia ser capaz de imprimir um número tão grande!

Comentário em Java

- Comentários são usados para explicar o código, tornando-o mais fácil de entender para outros desenvolvedores;
- Comentários não afetam a execução do programa, são completamente ignorados pelo compilador;

- Tipos de comentários:
 - **Comentário de linha:** Usado para breves explicações em uma linha (`// Comentário`).
 - **Comentário de bloco:** Explicações mais longas em várias linhas (`/* Comentário */`).
 - **Comentário de documentação:** Usado para gerar documentação do código (`/** Comentário */`).
- Comentários ajudam a manter o código legível e compreensível, facilitando a manutenção;

Comentário em java

```
// 3 - Comentários  
  
/*  
Comentários longos  
de várias linhas!  
*/  
  
/**  
 * Documentação também  
 * com várias linhas  
 * note o asterisco!  
 */
```

Strings

- Strings são sequências de caracteres usadas para armazenar e manipular texto;
- Classe String: Em Java, as Strings são objetos da classe String e oferecem métodos para manipulação de texto;
- Uma String não pode ser alterada, qualquer modificação gera uma nova String;
- Podemos concatenar (unir) duas strings com o operador +;
- Vamos testar!



Strings

```
// 4 - Strings

String firstName = "Fulano";
String lastName = "Beltrano";

System.out.println(x:"Fulano Beltrano");

System.out.println(firstName + " " + lastName);

System.out.println("O nome dele é: " + firstName);

String fullName = firstName + " " + lastName;

System.out.println(fullName);
```

Perceba o uso de concatenação, usando + para unir strings vazias com outras strings ou variáveis!

```
Fulano Beltrano
Fulano Beltrano
O nome dele é: Fulano
Fulano Beltrano
```

Char

- char é um tipo de dado primitivo que armazena um único caractere;
- O char pode armazenar qualquer caractere Unicode, incluindo letras, números e símbolos;
- Ocupa 2 bytes de memória, permitindo armazenar 65.536 caracteres diferentes;
- Vamos testar!

Char

```
char letra = 'A';  
  
System.out.println(letra);
```

Note que **char** usa aspas simples " ao contrário das aspas duplas "" que são reservadas para as strings no Java!

```
// Isso tem o mesmo efeito ocupando mais espaço!  
String letra2 = "A";  
  
System.out.println(letra2);
```

Lembre-se que é importante decidir com qual tipo de dados iremos trabalhar para tornar o código mais eficiente!

```
char simbolo = '$';  
  
System.out.println(simbolo);
```

E é claro, apenas um único caractere pode ser exibido!

```
char simbolo = '$a';
```


Int

- `int` é um tipo de dado primitivo utilizado para armazenar números inteiros;
- Tamanho fixo: Ocupa 4 bytes de memória, permitindo armazenar valores de -2.147.483.648 a 2.147.483.647;
- Ideal para operações aritméticas e contadores em loops;
- Vamos testar!

Int

```
// 6 - Int

int n = 42;

System.out.println(n);

System.out.println(n + 5);

System.out.println(n * 2);

System.out.println(n / 5);

int soma = n + 12;

System.out.println(soma);
```

Note que a divisão resultou em 8 ao invés de 8,4, isso ocorre porque `int` não considera os números decimais!

Long

- long é um tipo de dado primitivo usado para armazenar números inteiros muito grandes;
- Ocupa 8 bytes de memória, permitindo armazenar valores de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807;
- Ideal para cálculos financeiros, contagem de tempo, ou quando o tipo int não é suficiente;
- Valores literais do tipo long devem ser seguidos por L (maiúsculo) para indicar o tipo;
- O sublinhado pode ser usado para melhorar a legibilidade em números longos, separando grupos de dígitos (Ex: 123_456_789_012_345L);
- Vamos testar!

Long

Já vimos o long anteriormente na “atribuição de valor a outra variável”

```
// 7 - Long

Long populacaoMundial = 7800000000L;

System.out.println(populacaoMundial);

Long grandeNumero = 1_000_000;

System.out.println(grandeNumero + 1);
```

Note que os sublinhados não são impressos, mas ajudam os desenvolvedores a lerem grandes números!

```
7800000000
1000001
```

Double

- `double` é um tipo de dado primitivo usado para armazenar números de ponto flutuante (decimais) com dupla precisão;
- Ocupa 8 bytes de memória, permitindo armazenar uma ampla gama de valores decimais;
- Ideal para operações matemáticas que requerem precisão, como cálculos científicos e financeiros;
- Valores literais podem ser seguidos por `d` ou `D`, embora não seja obrigatório;
- O sublinhado pode ser usado para separar grupos de dígitos em números longos para melhorar a legibilidade;

Double

```
// 8 - Double

double preco = 19.99;

System.out.println(preco);

System.out.println(preco - 12);

System.out.println(preco / 2);
```

```
19.99
7.9899999999999998
9.995
```

Com o **double** podemos trabalhar com números 'quebrados', note apenas que usando ponto `.` e não vírgula `,` para separar as casas decimais

```
double pi = 3.14_592_123;

double valorComD = 100.10D;
```

Perceba a regra opcional de `_` para separar grandes números e o `D` no final dos numerais!

Operadores aritméticos parte 1

- Adição (+): Soma de dois valores;
- Subtração (-): Subtração de um valor pelo outro;
- Multiplicação (*): Multiplicação de dois valores;
- Divisão (/): Divisão de um valor pelo outro;
- Módulo (%): Resto da divisão de dois valores;
- Vamos testar!

Operadores Aritméticos pt1

```
// 9 OP. Aritméticos p1

System.out.println(12 + 5);
System.out.println(12 - 5);
System.out.println(12 * 5);
System.out.println(12 / 5);
System.out.println(12 % 5);

/*
| Para se ter um resultado 'quebrado'
| pelo menos um número deve ser double!
*/
System.out.println(10 / 2.5);

System.out.println(10.0 / 2.5);

/*
| Para se ter resultados corretos em divisão
| um dos números deve ser double, já que o Java
| arredonda o resultado para cima!
*/
System.out.println(10 / 3);
// resultado: 3
System.out.println(10.0 / 3);
// resultado: 3,33333333

// Módulo
System.out.println(10 % 3);
// resultado: 1
```

Operadores aritméticos parte 2

- Incremento (++): Incrementa o valor de uma variável em 1;
- Decremento (--): Decrementa o valor de uma variável em 1;
- Atribuição aditiva (+=): Soma e atribui o resultado a uma variável;
- Atribuição subtrativa (-=): Subtrai e atribui o resultado a uma variável;
- Vamos testar!

Operadores Aritméticos pt2

```
// 10 OP. Aritméticos p2

int x = 5;
x++;
System.out.println(x);
// resultado: 6

int y = 5;
y--;
System.out.println(y);
// resultado: 4

int a = 10;
a += 5;
System.out.println(a);
// resultado: 15

int b = 10;
b -= 5;
System.out.println(b);
// resultado: 5

a -= b;
System.out.println(a);
// resultado: 10

b -= a;
System.out.println(b);
// resultado: -5
```


Type casting

- Casting implícito (widening): Converte automaticamente tipos menores para tipos maiores (por exemplo, int para long), sem perda de dados;
- Casting explícito (narrowing): Necessário quando se converte tipos maiores para tipos menores (por exemplo, double para int), podendo resultar em perda de dados;
- Para realizar um casting explícito, é necessário especificar o tipo de destino entre parênteses;
- Vamos testar!

Type Casting (mudança de tipo de dado)

```
// 11 - Type Casting

// Implícito (Widening)
int numero2 = 42;
int numeroLong = numero2;
double numeroDouble = numero2;
System.out.println(numeroLong); // resultado: 42
System.out.println(numeroDouble); // resultado: 42.0
// Convertemos os dados para um tipo maior sem perda de dados!

// Explícito (Narrowing)
double valorDouble = 9.78;
// Método incorreto! > int valorInt = valorDouble;
// (int) converte double para int
int valorInt = (int) valorDouble;
System.out.println(valorInt); // resultado: 9

// Casting de char para int
// Conversor para Ascii
char letra3 = 'A';
int codigoAscii = (int) letra3;
System.out.println(codigoAscii); // resultado: 65
```

Exercícios

```
// Exercício 01
int a1 = 10;
int a2 = a1 * 2;
System.out.println(a2); // resultado: 20

// Exercício 02
char b1 = 'B';
int codigoAscii2 = (int) b1;
System.out.println(codigoAscii2); // resultado: 66

// Exercício 03
double c1 = 15.75;
double c2 = 20.40;
double c3 = c1 + c2;
System.out.println(c3); // resultado: 36.15

// Exercício 04
long d1 = 200_000_000L;
int d2 = (int) d1;
System.out.println(d2); // resultado: 200000000

// Exercício 05
String e1 = ("Olá, mundo!");
String e2 = (e1 + " " + "Bem-vindo ao Java!");
System.out.println(e2); // resultado: Olá, mundo! Bem-vindo ao Java!
```

Constantes em Java com final

- final: Define uma variável como constante, impedindo que seu valor seja alterado após a inicialização;
- Uma vez atribuído, o valor não pode ser modificado;
- Boas práticas: Usado para valores que não devem mudar durante a execução do programa, como PI ou taxas de juros;
- Convenção de nomes: Constantes geralmente são nomeadas em letras maiúsculas, com palavras separadas por sublinhados (_);

Constantes em java com final

```
// 12 - Constantes
final int DIAS_DA_SEMANA = 7;
System.out.println("Dias da semana: " + DIAS_DA_SEMANA);

// Constantes não são alteradas >X DIAS_DA_SEMANA += 1;

final int TESTE2;
System.out.println(TESTE2);
// Elas não podem ser inicializadas sem valor!
```

Inferência de Tipo com var em Java

- var: Introduzido no Java 10, permite ao compilador inferir o tipo da variável com base no valor atribuído;
- Reduz a necessidade de escrever tipos longos e complexos, aumentando a legibilidade do código;
- Tipo estático: Embora o tipo seja inferido, ele é fixo após a atribuição e não pode ser alterado;
- Regras: Deve ser inicializado no momento da declaração, e não pode ser usado para variáveis não inicializadas;
- Boas práticas: Útil para tipos complexos ou quando o tipo é óbvio a partir do contexto;

Inferência de tipo com var em java

```
// 13 - Var

var z = 10;
System.err.println(z); // resultado: 10
z = 5;
System.err.println(z); // resultado: 5
// Resulta em erro, não pode haver essa conversão
// >X z = "Teste"

var texto2 = "Teste";
System.out.println(texto2); // resultado: Teste

// Nem sempre é utilizado, varia de ambiente para ambiente!
```

A classe Scanner

- A classe Scanner é utilizada para ler a entrada de dados do usuário via console/terminal;
- Parte do pacote java.util: Para usar o Scanner, é necessário importar a classe do pacote java.util;
- Scanner pode ler diferentes tipos de dados, como int, double, String, etc;
- Métodos comuns:
 - `nextLine()`: Lê uma linha inteira de texto;
 - `nextInt()`: Lê um valor inteiro;
 - `nextDouble()`: Lê um valor decimal (ponto flutuante);
 - `next()` : Lê uma única palavra;

A classe scanner

```
import java.util.Scanner;
```

Por vezes a importação é automática assim que você declara o uso de Scanner (Ao tentar importar Scanner sem usá-lo no código resultava no apagamento da linha de importação)

```
// 1 - Testando Scanner
Scanner scanner = new Scanner(System.in);

// Mensagem para o usuário
System.out.println(x:"Digite o seu nome: ");
// Resgata o valor do terminal
String nome = scanner.nextLine();
// Exibe o valor
System.out.println("Olá, " + nome + "!");

// nextInt
System.out.println(x:"Digite um número: ");
int x = scanner.nextInt();
System.out.println("O número digitado foi: " + x);

// Fecha o Scanner para evitar erros!
scanner.close();
```

(Veremos o `Scanner.close()` com mais atenção depois!)

Importação de pacotes

- Pacotes organizam classes e interfaces relacionadas, melhorando a estrutura do código;
- Importar classes de pacotes externos, como Scanner, é necessário para utilizá-las;
- A importação de pacotes previne conflitos de nomes entre classes em projetos grandes;
- Permite a reutilização de código existente, acelerando o desenvolvimento;
- Facilita a manutenção e a leitura do código ao manter uma organização lógica;

Importação de pacotes

Veremos isso mais pra frente (tenha como referência o projeto de RPG, que continha várias funções de várias classes diferentes!)

Fechamento do scanner

- O Scanner consome recursos de entrada, como fluxo de dados do teclado;
- Fechar o Scanner libera esses recursos, evitando problemas de performance;
- Deixar o Scanner aberto pode causar vazamentos de memória ou travamentos;
- `close()` é uma boa prática recomendada após a leitura dos dados;
- O fechamento do Scanner ajuda a manter a aplicação eficiente e estável;

Fechamento do scanner

Coloque no final do código!

```
// Fecha o Scanner para evitar erros!  
scanner.close();
```


Problema do nextLine

- `nextLine()` lê a linha inteira até encontrar um Enter;
- Problema ocorre ao usar `nextLine()` após `nextInt()`, `nextDouble()`, etc;
- `nextLine()` captura o caractere Enter remanescente, resultando em uma leitura vazia;
- Isso faz o programa parecer "pular" a entrada de texto;
- Solução: adicionar um `nextLine()` extra após a leitura de números;
- Vamos ver na prática;

Problema do nextLine

```
// 2 - Problema do nextLine
System.out.println(x:"Digite um número: ");
int y = scanner.nextInt();

// Esse nextline vazio impede o erro descrito abaixo de ocorrer!
scanner.nextLine();

System.out.println(x:"Digite o um texto: ");
String txt = scanner.nextLine();

System.out.println("Os dados são: y = " + y + " e txt = " + txt);
// resultado: Os dados são: y = 5 e txt =
```

`scanner.nextLine()` deve vir sempre após recebermos um número!

Projeto: Tabuada Básica

```
Scanner scanner = new Scanner(System.in);

System.out.println(x: "Digite um número: ");
int x = scanner.nextInt();

System.out.println(x + " x 1 = " + (x*1));
System.out.println(x + " x 2 = " + (x*2));
System.out.println(x + " x 3 = " + (x*3));
System.out.println(x + " x 4 = " + (x*4));
System.out.println(x + " x 5 = " + (x*5));
System.out.println(x + " x 6 = " + (x*6));
System.out.println(x + " x 7 = " + (x*7));
System.out.println(x + " x 8 = " + (x*8));
System.out.println(x + " x 9 = " + (x*9));
System.out.println(x + " x 10 = " + (x*10));

scanner.close();
```

```
Digite um número:
100
100 x 1 = 100
100 x 2 = 200
100 x 3 = 300
100 x 4 = 400
100 x 5 = 500
100 x 6 = 600
100 x 7 = 700
100 x 8 = 800
100 x 9 = 900
100 x 10 = 1000
```

Projeto: Média do Aluno

```
Scanner scanner = new Scanner(System.in);

System.out.println(x:"Digite seu nome: ");
String nome = scanner.nextLine();

System.out.println(x:"Digite a primeira nota: ");
double a = scanner.nextDouble();
System.out.println(x:"Digite a segunda nota: ");
double b = scanner.nextDouble();
System.out.println(x:"Digite a terceira nota: ");
double c = scanner.nextDouble();

double media = (a + b + c) / 3;

System.out.println("O nome é: " + nome);
System.out.println("A média das notas é: " + media);
if (media >= 7) {
    System.out.println(x:"Aprovado");
} else {
    System.out.println(x:"Reprovado");
}
scanner.close();
```

```
Digite seu nome: 33dda34-46dc-4eef-a667-
Fulano
Digite a primeira nota:
8,3
Digite a segunda nota:
5,7
Digite a terceira nota:
8,6
O nome é: Fulano
A média das notas é: 7.533333333333334
Aprovado
```

O condicional será visto adiante

O uso de , ao usar dados do tipo Double é definido pelo OS!

Condicionais

O que é boolean?

- boolean é um tipo de dado primitivo em Java que pode armazenar apenas dois valores: true ou false;
- Utilizado para controlar o fluxo de execução em estruturas de controle como if, while e for;
- Suporta operadores lógicos como && (AND), || (OR), e ! (NOT) para combinar e inverter valores booleanos;
- Uma expressão booleana é qualquer expressão que resulta em um valor true ou false;
- Embora seja representado como true ou false no código, internamente é tratado como 1 (true) ou 0 (false);

O que é boolean

```
// 1 - O que é boolean?  
boolean isTrue = true;  
boolean isFalse = false;  
  
System.out.println(isTrue);  
System.out.println(isFalse);
```

```
0d-8ed4a931e1a7 true  
false
```

Operadores de comparação

- == (Igual a): Verifica se dois valores são iguais;
- != (Diferente de): Verifica se dois valores são diferentes;
- > (Maior que): Verifica se o valor à esquerda é maior que o valor à direita;
- < (Menor que): Verifica se o valor à esquerda é menor que o valor à direita;
- >= (Maior ou igual a): Verifica se o valor à esquerda é maior ou igual ao valor à direita;
- <= (Menor ou igual a): Verifica se o valor à esquerda é menor ou igual ao valor à direita;

Operadores de comparação

```
// 2 - Operadores de Comparação
```

```
int x = 10;
```

```
System.out.println(x == 10);
```

```
System.out.println(x == 9);
```

```
System.out.println(x != 5);
```

```
System.out.println(x != 10);
```

```
System.out.println(x > 10);
```

```
System.out.println(x >= 10);
```

```
System.out.println(x < 10);
```

```
System.out.println(x <= 10);
```

```
true  
false  
true  
false  
false  
true  
false  
true
```

Diferença entre comparação e atribuição

- Atribuição (=):
 - Atribui um valor a uma variável;
 - Usado para definir ou alterar o valor armazenado em uma variável;
 - Exemplo: `int a = 5;`
- Comparação (==):
 - Compara dois valores para verificar se são iguais;
 - Retorna `true` se os valores forem iguais e `false` se forem diferentes;
 - Exemplo: `5 == 5` retorna `true`;



Diferença entre comparação e atribuição

```
// 3 - Atribuição e Comparação
int n = 5;
int m = 10;

// A ausência de um '=' torna N 12
System.out.println(n = 12);

// Aqui simplesmente comparamos ambos Ints
System.out.println(n == m);
```


Comparação de strings

- Problema com ==:
 - O operador == compara as referências de memória, não o conteúdo das strings;
 - Pode retornar false mesmo que o conteúdo das strings seja igual, se as referências forem diferentes;
- Uso do método equals():
 - equals() compara o conteúdo das strings, caractere por caractere;
 - É a maneira correta e segura de verificar se duas strings são iguais em valor;
- equalsIgnoreCase():
 - Variante de equals() que ignora diferenças entre maiúsculas e minúsculas;

Comparação de strings

```
// 4 - Comparação de Strings

String str1 = "Java";

// Uma forma alternativa de criar Strings...
String str2 = new String(original:"Java");

System.out.println(str1);
System.out.println(str2);

System.out.println(str1 == str2); // resultado: false

System.out.println(str1 == "Java"); // resultado: true

// Métodos recomendados
System.out.println(str1.equals(str2)); // resultado: true

System.out.println(str2.equals(str1)); // resultado: true

System.out.println(str1.equals(anObject:"Java")); // resultado: true

String str3 = "JAVA";

System.out.println(str1.equals(str3)); // resultado: false

System.out.println(str1.equalsIgnoreCase(str3)); // resultado: true
```

Estruturas de condição

- if: Executa um bloco de código se a condição for verdadeira;
- else: Executa um bloco de código alternativo se a condição do if for falsa;
- else if: Verifica outra condição se as condições anteriores forem falsas;
- switch: Seleciona e executa um bloco de código entre várias opções com base no valor de uma expressão;
- Importante: Todas essas estruturas permitem controlar o fluxo de execução com base em condições lógicas;

Estruturas de condição

Conhecendo o if

- if executa um bloco de código se a condição for verdadeira;
- Coloque a condição entre parênteses após a palavra-chave if;
- Uso comum: Comparações lógicas ou aritméticas para tomar decisões no código;
- Importante: O bloco de código dentro de if é delimitado por chaves {};

Conhecendo o if

```
// 5 - If

int numero = 10;

// Comparação
if (numero > 5) {
    System.out.println(x:"Acho que já passamos desse nível rs");
}

// If com strings
String texto = "Teste";

// Se a função retorna um booleano, eu posso usar no if!
if (texto.equals(anObject:"Teste")) {
    System.out.println(x:"Os textos são iguais!");
}

// Declaração do if (Comparação ou retorno de booleano) {código para executar!}
```

Explorando o else

- else executa um bloco de código quando a condição do if é falsa;
- Proporciona uma alternativa no fluxo de execução do programa;
- Sintaxe: O bloco else vem logo após um bloco if;
- Bloco único: Somente um bloco else pode seguir um if;
- Importante: Sempre use {} para delimitar o bloco de código do else;

Explorando o else

```
// 6 - Else

int q = 7;

if (q > 10) {
    System.out.println(x:"Q é maior que 10!");
} else {
    System.out.println( x:"Q não é maior que 10!");
}

// Todo else precisa de um if, mas nem todo if precisa ter um else!
```

Utilizando o else if

- else if permite testar condições adicionais após um if;
- Sintaxe: Coloque a condição entre parênteses após a palavra-chave else if;
- Uso comum: Quando há várias condições mutuamente exclusivas;
- Encadeamento: Vários blocos else if podem ser usados após um if;
- Bloco final opcional: Pode ser seguido de um else para lidar com qualquer caso não coberto;

Utilizando o else if

```
// 7 - Else if

double nota = 10;
if (nota == 10) {
    System.out.println(x:"PERFEITO!");
} else if (nota >= 9) {
    System.out.println(x:"Nota excelente!");
} else if (nota >= 7) {
    System.out.println(x:"Acima da média!");
} else {
    System.out.println(x:"Abaixo da média!");
}

// Se houverem múltiplas condições verdadeiras, o programa irá retornar a primeira!
int num = 5;

if (num > 3 ) {
    System.out.println(x:"Alguma coisa");
} else if (num == 5) {
    System.out.println(x:"Outra coisa aqui");
}

// Para resolver isso podemos fazer duas coisas:
// 1 - Escolher qual situação quero que mais ocorra
// 2 - Melhorar a lógica do programa!
// if (num > 3 && num < 5) <- Resolveria o problema!!
```

Operadores lógicos

- && (E lógico): Retorna true se ambas as condições forem verdadeiras;
- || (OU lógico): Retorna true se pelo menos uma das condições for verdadeira;
- ! (NÃO lógico): Inverte o valor lógico; retorna true se a condição for falsa e vice-versa;
- Combinação: Pode combinar múltiplas condições em uma única expressão lógica;

Operadores lógicos

Tabela verdade

- A tabela verdade simula todas as combinações possíveis dos operadores lógicos, e exibe os resultados:

AND			OR			NOT	
A	B	$A \cup B$	A	B	$A \cup B$	A	$\bar{A}$
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Tabela verdade

Trabalhando com o AND

- O operador && (AND lógico) retorna true se ambas as condições forem verdadeiras.
- Sintaxe: Condição1 && Condição2;
- Curto-circuito: Se a primeira condição for false, a segunda condição não é avaliada;
- Uso comum: Combinação de múltiplas condições que precisam ser verdadeiras ao mesmo tempo;

Trabalhando com o and

```
// 8 - And

int idade = 18;
boolean cNH = true;
boolean cnhVencida = true;

System.out.println(idade >= 18 && cNH); // true
// Tem o mesmo efeito de:
System.out.println(idade >= 18 && cNH == true); // true

System.err.println(idade >= 18 && cNH && cnhVencida == false); // false

int a = 10;
int b = 20;

// True && True => True
if(a > 5 && b > 10){
    System.out.println(x:"Deu certo!");
}

// False && True => Curto Circuito!
if(a > 55 && b > 10){
    System.out.println(x:"Deu certo!");
}
```


Operador OR

- O operador || (OR lógico) retorna true se pelo menos uma das condições for verdadeira;
- Sintaxe: Condição1 || Condição2;
- Curto-circuito: Se a primeira condição for true, a segunda condição não é avaliada;
- Uso comum: Verificação de múltiplas condições onde apenas uma precisa ser verdadeira;

Operador or

|| é chamado de pipe!

```
// 9 - Or

boolean estarChuvendo = false;
boolean temGuardaChuva = true;

System.out.println(estarChuvendo || temGuardaChuva); // true

System.out.println(10 > 20 || 100 == 200); // false

int idade2 = 5;
boolean membro = true;

// Precisa ter > 16 anos OU ser membro

if (idade2 >= 16 || membro){
    System.out.println(x:"Tudo certo");
} else {
    System.out.println(x:"Tudo errado");
}
```

Operador NOT

- O operador **!** (NOT lógico) inverte o valor lógico de uma expressão;
- Sintaxe: **!** seguido da condição ou expressão;
- Uso comum: Negar uma condição para tomar decisões baseadas no oposto;
- Útil em validações: Verificar se uma condição é falsa, ao invés de verdadeira;
- Combinação: Pode ser combinado com outros operadores lógicos (**&&**, **||**) para criar expressões mais complexas;

Operador not

```
// 10 - Not

System.out.println(estarChuvendo); // false
// ! inverte o booleano!
System.out.println(!estarChuvendo); // true

System.out.println(estarChuvendo || !temGuardaChuva); // false (ambos false!)

// Tal como na matemática o valor entre as () será levando em consideração
// primeiro, então esse valor será invertido pelo ! que está fora dos colchetes!
System.out.println(!(estarChuvendo || !temGuardaChuva));
// true (inversão por fora dos () )
```

Estrutura switch em Java: case e break

- switch: Estrutura de controle que permite escolher entre várias opções com base no valor de uma expressão;
- case: Define uma possível opção ou caminho dentro do switch. Cada case é seguido por um valor específico que é comparado com a expressão do switch;
- break: Utilizado para encerrar a execução de um bloco case. Evita que o código "caia" nos casos seguintes;
- Importante: Cada case deve terminar com um break (ou outro comando de desvio) para evitar a execução de outros casos;
- Valores exclusivos: Os valores em case devem ser exclusivos e correspondentes ao tipo da expressão do switch;



Estrutura switch em java: case e break

```
// 11 - Switch Case e Break

// Dias da semana baseado em número
// 1 = Domingo, 7 = Sábado

int diaSemana = 4;

switch(diaSemana) {
    case 1:
        System.out.println(x:"Domingo");
        break;
    case 2:
        System.out.println(x:"Segunda-feira");
        break;
    case 3:
        System.out.println(x:"Terça-feira");
        break;
    case 4:
        System.out.println(x:"Quarta-feira");
        break;
    case 5:
        System.out.println(x:"Quinta-feira");
        break;
    case 6:
        System.out.println(x:"Sexta-feira");
        break;
    case 7:
        System.out.println(x:"Sábado");
        break;
}
```

Quarta-feira

Estrutura switch em Java: default

- default: O bloco default é executado quando nenhum dos valores especificados nos case corresponde à expressão do switch;
- Opcional: O uso de default é opcional, mas recomendado para capturar todos os casos não previstos;
- Posição: Normalmente é colocado no final do switch, mas pode aparecer em qualquer lugar dentro do bloco;
- Sem break necessário: Como default geralmente é o último bloco, não é necessário usar break, mas pode ser incluído se o default não for o último;
- Fornece um comportamento padrão ou uma mensagem de erro quando nenhum caso específico é atendido;

Estrutura switch em java: default

```
// 12 - Default

int n = 10;

switch (n) {
    case 1:
        System.out.println(x:"É 1");
        break;
    case 2:
        System.out.println(x:"É 2");
        break;
    default:
        System.out.println(x:"Valor não encontrado!");
}
```

Valor não encontrado!

```
switch (n) {
    case 1 -> System.out.println(x:"É 1");
    case 2 -> System.out.println(x:"É 2");
    default -> System.out.println(x:"Valor não encontrado!");
}
```

(Estrutura de Switch recomendada pelo VSCode)

switch expressions.

Switch sem brake

- fall-through: Sem break, o switch continua executando os blocos subsequentes, mesmo que o caso correspondente já tenha sido encontrado;
- Efeito inesperado: Pode levar à execução de múltiplos casos, causando resultados inesperados;
- Necessário break: Para interromper a execução após o bloco case correspondente, evitando o efeito de fall-through;
- Uso intencional: Em raros casos, o fall-through pode ser usado intencionalmente, mas é menos comum e deve ser bem documentado;
- Boa prática: Sempre incluir break para prevenir comportamentos indesejados, a menos que o fall-through seja intencional;

Switch sem brake

(Com a modernização do java o método de criar switch mudou e os breaks já não são mais necessários, pois são implícitos)

```
switch(diaSemana) {  
    case 1 -> System.out.println(x: "Domingo");  
    case 2 -> System.out.println(x: "Segunda-feira");  
    case 3 -> System.out.println(x: "Terça-feira");  
    case 4 -> System.out.println(x: "Quarta-feira");  
    case 5 -> System.out.println(x: "Quinta-feira");  
    case 6 -> System.out.println(x: "Sexta-feira");  
    case 7 -> System.out.println(x: "Sábado");  
  
    // 12 - Default  
  
    int n = 10;  
  
    switch (n) {  
        case 1 -> System.out.println(x: "É 1");  
        case 2 -> System.out.println(x: "É 2");  
        case 3 -> System.out.println(x: "É 3");  
        default -> System.out.println(x: "Valor não encontrado!");  
    }  
}
```

```
// 13 - Switch sem break (inútil!)

switch (1) {
    case 1:
        System.out.println(x:"Executou 1");
    case 2:
        System.out.println(x:"Executou 2");
    case 3:
        System.out.println(x:"Executou 3");
    default:
        System.out.println(x:"Executou 4");
}
// Todos os resultados incluindo o default são exibidos!
```

```
Executou 1
Executou 2
Executou 3
Executou 4
```


Quando Usar if vs. switch

- Usar if:
 - Ideal para expressões booleanas simples;
 - Perfeito para condições que envolvem operadores lógicos (&&, ||) e comparações entre diferentes variáveis;
 - Útil quando a condição depende de um intervalo de valores (ex.: $x > 10$);
 - Útil para comparar objetos, strings com equals(), ou outras condições não numéricas;
- Usar switch:
 - Melhor para escolher entre várias opções discretas baseadas em um único valor;
 - Ideal quando você está lidando com um conjunto limitado de valores, como enumerações ou dias da semana;
 - Facilita a leitura e a manutenção quando há muitas opções (mais de 3 ou 4 casos);
 - Funciona bem com expressões baseadas em inteiros, caracteres, strings ou enums;

Quando usar if vs switch

Exercícios

```
// Exercício 01

Scanner scanner = new Scanner(System.in);

System.out.println(x:"Digite o valor do produto: ");
double valor = scanner.nextDouble();

if (valor > 100){
    System.out.println(x:"O produto é caro!");
} else if (valor >= 50 && valor <= 100) {
    System.out.println(x:"O produto é médio!");
} else {
    System.out.println(x:"O produto é barato!");
}

scanner.nextLine();

// Exercício 2

System.out.println(x:"Digite o login: ");
String login = scanner.next();

System.out.println(x:"Digite a senha: ");
String senha = scanner.next();

if ("admin".equals(login) && "1234".equals(senha)){
    System.out.println(x:"Acesso Permitido");
} else {
    System.out.println(x:"Acesso Negado!");
}

scanner.close();
```

```
Digite o valor do produto:
100
O produto é médio!
Digite o login:
admin
Digite a senha:
1234
Acesso Permitido
```

```
// Exercício 3

System.out.println(x:"Insira um número: ");
int x = scanner.nextInt();

if ((x % 2) == 1){
    System.out.println(x:"O número é impar");
} else {
    System.out.println(x:"O número é par");
}

// Exercício 4

System.out.println(x:"Insira um número de 1 a 7");
int y = scanner.nextInt();
switch (y) {
    case 1 -> System.out.println(x:"Domingo / Final de semana");
    case 2 -> System.out.println(x:"Segunda-feira / Dia útil");
    case 3 -> System.out.println(x:"Terça-feira / Dia útil");
    case 4 -> System.out.println(x:"Quarta-feira / Dia útil");
    case 5 -> System.out.println(x:"Quinta-feira / Dia útil");
    case 6 -> System.out.println(x:"Sexta-feira / Dia útil");
    case 7 -> System.out.println(x:"Sábado / Final de semana");
    default -> System.err.println(x:"Insira um valor válido!");
}
scanner.close();
```

if (x % 2 == 1) é preferencial ao invés de if ((x % 2) == 1)

o switch pode ser usado dessa forma também:

```
case 7,8,9 -> System.out.println("Sábado / Final de semana");
```

```
Insira um número:
50
O número é par
Insira um número de 1 a 7
6
Sexta-feira / Dia útil
```

```

// Exercício 5

System.out.println(x:"Insira um valor: ");
int valor = scanner.nextInt();

if (valor >= 10 && valor <= 20){
    System.out.println(x:"Valor dentro do intervalo");
} else {
    System.out.println(x:"Valor fora do intervalo");
}

// Exercício 6

System.out.println(x:"Insira um letra: ");
// charAt(0) vai considerar apenas o dígito de posição 0! (primeiro)
char letra = scanner.next().toLowerCase().charAt(index:0);
switch (letra) {
    // Com Char usamos aspas simples ' '
    case 'a', 'e', 'i', 'o', 'u' -> System.out.println(x:"Vogal");
    default -> System.out.println(x:"Consoante");
}
scanner.close();

```

Lembrete: Char usa aspas simples 'aspas' e .charAt(0) vai usar o char de índice 0 no input, sendo 0 a primeira letra da palavra!

```

Insira um valor:
63
Valor fora do intervalo
Insira um letra:
p
Consoante

```

Condicionais ternárias

- Uma forma compacta de expressar uma condição if-else;
- Sintaxe: condição ? expressão1 : expressão2;
- Funcionamento: Avalia a condição; se for verdadeira, retorna expressão1, caso contrário, retorna expressão2;
- Ideal para atribuições simples e condições em linha;
- Limitação: Deve ser usado apenas em expressões simples para manter a legibilidade;

Condicionais ternarias

```
// 1 - Condicional ternário

System.out.println(x:"Digite um número: ");
int n = scanner.nextInt();

// Condição x > 5 ? true : false;
String resultado = (n % 2 == 0) ? "Par" : "Impar";

System.out.println(resultado);
```

```
Digite um número:
200
Par
```

If e else aninhado

- Estrutura onde um if ou else contém outro bloco if-else;
- Uso comum: Para testar múltiplas condições que dependem umas das outras;
- Útil para lidar com decisões complexas, mas pode prejudicar a legibilidade se usado em excesso;
- Sintaxe: Blocos if e else podem conter outros if-else, criando um encadeamento de condições;
- Boa prática: Mantenha o código claro e evite encadeamentos profundos;

If e else aninhado

```
// If encadeado

int idade = 25;
boolean cNH = false;

if (idade >= 18){
    if (cNH) {
        System.out.println(x:"Pode dirigir");
    } else {
        System.out.println(x:"Não tem CNH para poder dirigir");
    }
} else {
    System.out.println(x:"Não tem idade para poder dirigir");
}
```

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Digite a idade:");
        int idade = sc.nextInt();
        System.out.println("Digite se possui CNH (true/false):");
        boolean cNH = sc.nextBoolean();
        if (idade >= 18) {
            if (cNH) {
                System.out.println("Pode dirigir");
            } else {
                System.out.println("Não tem CNH para poder dirigir");
            }
        } else {
            System.out.println("Não tem idade para poder dirigir");
        }
    }
}
```

Não deixe o encadeamento grande!

Precedência de Operadores Lógicos

- A ordem em que os operadores lógicos e de comparação são avaliados em uma expressão;
- Ordem de Precedência:
 - 1. (): Parênteses têm a maior precedência e são avaliados primeiro;
 - 2. !: Operador NOT lógico tem a segunda maior precedência;
 - 3. &&: Operador AND lógico é avaliado antes de ||;
 - 4. ||: Operador OR lógico é avaliado por último;
- Compreender a precedência evita erros lógicos e garante que as expressões sejam avaliadas conforme esperado;
- Uso de Parênteses: Parênteses podem ser usados para alterar a ordem de avaliação, melhorar a clareza do código;

Precedência de operadores lógicos

```
// 3 - Precedência

boolean a = true;
boolean b = false;
boolean c = true;

// TRUE AND FALSE -> FALSE OR TRUE -> TRUE
boolean resultado2 = a && b || c; // true
System.out.println(resultado2);

// TRUE OR FALSE -> TRUE AND TRUE
boolean resultado3 = a || b && c; // true
System.out.println(resultado3);

// NOT (TRUE OR FALSE) -> NOT(TRUE) -> FALSE AND TRUE -> FALSE
boolean resultado4 = !(a || b) && c; // false
System.out.println(resultado4);

// (NOT TRUE OR FALSE) AND TRUE
// (FALSE OR FALSE) AND TRUE
// FALSE AND TRUE
// FALSE
boolean resultado5 = (!a || b) && c; // false
System.out.println(resultado5);
```

(ATENÇÃO!)

Projeto: Calculadora de IMC

```
System.out.println(x:"Digite seu peso em Kgs: ");
double peso = scanner.nextDouble();

System.out.println(x:"Digite sua altura em Metros: ");
double altura = scanner.nextDouble();

double imc = peso / (altura * altura);

if (imc < 18.5){
    System.out.println("Seu IMC é: " + imc + " Você é considerado: Abaixo do
    peso");
} else if (imc >= 18.5 && imc <= 24.9){
    System.out.println("Seu IMC é: " + imc + " Você é considerado: Peso normal");
} else if (imc >= 25 && imc < 29.9){
    System.out.println("Seu IMC é: " + imc + " Você é considerado: Sobrepeso");
} else {
    System.out.println("Seu IMC é: " + imc + " Você é considerado: Obeso");
}

scanner.close();
```

```
Digite seu peso em Kgs:
78,8
Digite sua altura em Metros:
1,64
Seu IMC é: 29.298036882807857 Você é considerado: Sobrepeso
```

Projeto: Classificação de Produto

```
// Catalogo
String produto1 = "Teclado";
String produto2 = "Mouse";
String produto3 = "Monitor";

double preco = 0;

System.out.println(x:"Digite o nome do produto: ");

String nomeProduto = scanner.nextLine();

// Verificação do Catalogo
if (produto1.equalsIgnoreCase(nomeProduto) || produto2.equalsIgnoreCase
(nomeProduto) || produto3.equalsIgnoreCase(nomeProduto)) {
    System.out.println(x:"Produto já existente. Atualizar o preço? | (sim/não)");
    String resposta = scanner.nextLine();
    if (resposta.equalsIgnoreCase(anotherString:"sim")){
        System.out.println(x:"Insira o novo preço: ");
        preco = scanner.nextDouble();
    } else {
        System.out.println(x:"Preço inalterado");
        scanner.close();
    }
} else {
    System.out.println(x:"Insira o preço do produto: ");
    preco = scanner.nextDouble();
}

// Classificação

if (preco < 50){
    System.out.println(x:"Classificação: Barato");
} else if (preco >= 50 && preco <= 100){
    System.out.println(x:"Classificação: Moderado");
} else {
    System.out.println(x:"Classificação: Caro");
}

// Output

System.out.println("Produto: " + nomeProduto + " . Preço: " + preco);
```

```
Digite o nome do produto:
Martelo
Insira o preço do produto:
50
Classificação: Moderado
Produto: Martelo . Preço: 50.0
```

```
Digite o nome do produto:
Teclado
Produto já existente. Atualizar o preço? | (sim/não)
sim
Insira o novo preço:
63
Classificação: Moderado
Produto: Teclado . Preço: 63.0
```

O que é uma função?

- Um **bloco de código** que realiza uma tarefa específica e pode ser chamado para ser executada;
- Divide o código em partes menores, tornando-o mais organizado e fácil de manter;
- Permite reutilizar código em diferentes partes do programa **sem precisar reescrever** as mesmas instruções;
- **Parâmetros e Retorno:** Pode receber dados de entrada (parâmetros) e retornar um resultado após a execução;
- As variáveis declaradas dentro de uma função **são locais** e não afetam o restante do programa;

O que é uma função

Criando a primeira função

- Normalmente uma função em Java é definida com um tipo de retorno, um nome e pode ou não receber parâmetros;
- A função pode ser criada sem parâmetros e sem retorno, ideal para tarefas simples que não requerem entrada ou saída;
- **Sintaxe Básica:** Consiste em um cabeçalho que inclui o tipo de retorno (void para sem retorno) e o corpo da função, onde o código é executado;
- **Chamando a Função:** A função é invocada pelo seu nome, e o código dentro dela é executado sempre que chamada;

Criando a primeira função

(void para funções sem retorno!!!)

```
// 1 - Criando a primeira função
saudacao();
// Funções tem () para diferenciar elas de variáveis e etc

}

// Perceba que criamos a variável fora do escopo do Main!
// PUBLIC = nível de acesso, STATIC = não precisa instanciar classe para executar,
// VOID = tipo de retorno
// Nome, (Argumentos) e {Bloco}
public static void saudacao() {
|   System.out.println(x:"Olá, está é uma função!");
}
```

Vamos entrar na questão de **STATIC** e etc logo depois!

Diferença entre a Função main e Outras Funções

- **Função main:**
 - É o ponto de entrada do programa, onde a execução começa;
 - Deve ter a assinatura exata `public static void main(String[] args);`;
 - Todo programa Java precisa de uma função main para ser executado;
 - A função main pode chamar outras funções e métodos dentro do programa;
- **Outras Funções:**
 - Criadas para dividir o código em partes menores e gerenciáveis;
 - Podem ter diferentes tipos de retorno, nomes, e receber parâmetros;
 - Podem ser chamadas várias vezes em diferentes partes do programa;
 - São executadas apenas quando chamadas pelo código, ao contrário do main, que é executado automaticamente;



Diferença entre a função main e outras funções

Argumentos em funções

- **Dados que você passa para uma função ao chamá-la**, permitindo que a função processe informações específicas;
- Parâmetros são **variáveis definidas na assinatura da função** para receber os argumentos;
- Funções podem receber nenhum, um ou vários argumentos, dependendo da tarefa que realizam;
- **Argumentos tornam as funções mais flexíveis e reutilizáveis** em diferentes contextos com dados diferentes;
- **Tipos de Argumentos:** Podem ser de qualquer tipo primitivo (int, double, etc.) ou objetos;

Argumentos em funções

Assinatura é o cabeçalho onde temos o 'public', 'main', 'void', etc, os argumentos são passados entre ()

```
// 2 - Parâmetros  
soma(a:2, b:10);  
soma(a:43, b:31);
```

```
0 resultado da é: 12  
0 resultado da é: 74
```

```
public static void soma(int a, int b) {  
    int resultado = a + b;  
    System.out.println("O resultado da é: " + resultado);  
}
```


Uso do return em funções

- Uma instrução que finaliza a execução de uma função e, opcionalmente, devolve um valor ao ponto onde a função foi chamada;
- Finalização de Função: Quando o return é executado, a função para de executar, e o controle é devolvido ao chamador;
- Tipos de Retorno: O return pode retornar valores de qualquer tipo, incluindo tipos primitivos, objetos, ou nenhum valor (void);
- O return permite que funções realizem cálculos ou operações e **enviem o resultado de volta** para ser utilizado em outras partes do programa;

Uso de return em funções

Podemos usar o **return** para parar a lógica de uma função e seguir o código do programa

```
// Note que aqui definimos que o retorno é um int e não um void!  
public static int dobrar(int n){  
    return n * 2;  
}
```

```
// 3 - Return  
  
dobrar(n:10); // Return não retorna código pro usuário e sim para uso posterior!  
  
int numero = 300;  
  
int numeroDobrado = dobrar(numero); // Dessa forma atribuímos o resultado em um int!  
System.out.println("O número dobrado é: " + numeroDobrado); // resultado: 600  
  
System.out.println(dobrar(n:20)); // resultado: 40
```


Funções com X sem retorno

- **Funções com Retorno:**

- Permitem que uma operação seja realizada e seu resultado seja utilizado em outras partes do programa;
- Cálculos, validações, e operações que produzem um resultado necessário para outras funções ou partes do código;
- Exemplo: Calcular a soma de dois números e retornar o resultado para ser exibido ou usado em outro cálculo;

- **Funções sem Retorno:**

- Executam uma ação sem precisar devolver um resultado, ideal para tarefas como exibição de dados ou alterações diretas no estado do programa;
- Exibir mensagens, modificar variáveis globais, ou realizar operações que não requerem um retorno;
- Exemplo: Exibir uma mensagem de boas-vindas ou atualizar o valor de uma variável de controle;

Funções com X sem retorno

Quando trabalhamos com retorno, podemos pegar esse 'retorno' e usar em outra função

Encapsulando retorno em variável

- O processo de armazenar o resultado de uma função em uma variável para uso posterior;
- Permite reutilizar o valor retornado por uma função em várias partes do código, aumentando a modularidade e a legibilidade;
- **Uso Comum:** Armazenar resultados de cálculos, verificações ou operações complexas para evitar chamadas repetidas à mesma função;
- Encapsular o retorno em uma variável **pode ajudar a simplificar o código** e reduzir a necessidade de executar novamente a função;

Encapsulando retorno em variável

Parâmetros de funções diferentes podem ter o mesmo nome (o 'n' se repete!)

```
public static String verificarPar(int n) {  
    if (n % 2 == 0){  
        return "O número " + n + " é par!";  
    } else {  
        return "O número " + n + " é impar!";  
    }  
}
```

```
// 4 - Retorno em variável  
String r1 = verificarPar(numero);  
  
String r2 = verificarPar(n:3);  
  
System.out.println(r1);  
System.out.println(r2);
```

Encapsulamos o retorno em r1 e r2

```
O número 300 é par!  
O número 3 é impar!
```

Se duplicarmos a função dobrar e colocar um retorno nela, podemos usar o resultado para realizar outras operações:

```
public static int soma2(int a, int b) {  
    return a + b;  
}
```

```
int x = dobrar(soma2(a:2, b:4));  
System.out.println(x);  
}
```

12
PS:

Dessa forma com o retorno da função soma2 efetuamos a função de dobrar e atribuímos ela á variável 'x'

Funções com if/else e Condicionais Complexas

- Estruturas de controle if/else dentro de funções permitem tomar decisões complexas baseadas em múltiplas condições;
- Condicionais Complexas: Permitem combinar múltiplas condições usando operadores lógicos (&&, ||) e comparações para determinar o fluxo de execução;
- **Uso Comum:** Verificações de múltiplos critérios, tomadas de decisão em processos complexos e validações de entradas de usuário;
- Funções com if/else organizam e centralizam a lógica de decisão, tornando o código mais modular e fácil de manter;



Funções com if/else e condicionais complexas

```
public static String verificarAcesso(  
    int idade,  
    boolean temCarteira,  
    boolean temHistoricoNegativo){  
  
    // Note o ! para se referir ao estado negativo do boolean no if!  
    if(idade >= 18 && temCarteira && !temHistoricoNegativo){  
        return "Acesso permitido, critérios atendidos";  
    } else if (idade >= 18 && temCarteira && temHistoricoNegativo) {  
        return "Acesso negado, histórico negativo detectado!";  
    } else {  
        return "Acesso negado, critérios não atendidos";  
    }  
}
```

```
// 5 - Funções com Condicionais  
String r1 = verificarAcesso(idade:19, temCarteira:true, temHistoricoNegativo:true);  
System.out.println(r1);  
  
String r2 = verificarAcesso(idade:25, temCarteira:false, temHistoricoNegativo:true);  
System.out.println(r2);
```

```
Acesso negado, histórico negativo detectado!  
Acesso negado, critérios não atendidos
```

Funções com switch

- O switch é uma estrutura de controle que permite a execução de diferentes blocos de código com base no valor de uma expressão;
- O switch dentro de uma função é útil **quando há múltiplas condições discretas (casos)** a serem verificadas, como valores inteiros, caracteres ou strings;
- **Benefícios:** Simplifica a lógica quando comparado a múltiplos if-else, tornando o código mais organizado e fácil de entender;
- **Boa Prática:** Sempre incluir um default para tratar valores inesperados ou casos não cobertos;

Funções com switch

```
public static String obterDiaDaSemana (int dia){  
    switch (dia) {  
        case 1 -> {  
            return "Segunda-feira";  
        }  
        case 2 -> {  
            return "Terça-feira";  
        }  
        case 3 -> {  
            return "Quarta-feira";  
        }  
        case 4 -> {  
            return "Quinta-feira";  
        }  
        case 5 -> {  
            return "Sexta-feira";  
        }  
        case 6 -> {  
            return "Sábado";  
        }  
        case 7 -> {  
            return "Domingo";  
        }  
        default -> {  
            return "Número Inválido!";  
        }  
    }  
}
```

O uso do 'break' é desnecessário em algumas situações, a IDE ajuda a identificar esses momentos!

```
// 6 - Funções com Switch
```

```
System.out.println(obterDiaDaSemana(dia:5));  
System.out.println(obterDiaDaSemana(dia:3));  
System.out.println(obterDiaDaSemana(dia:1));  
System.out.println(obterDiaDaSemana(dia:10));
```

```
Sexta-feira  
Quarta-feira  
Segunda-feira  
Número Inválido!
```


Funções com System.exit

- **O que é System.exit:**
 - O método `System.exit(int status)` encerra imediatamente a execução do programa, finalizando todas as operações em andamento;
 - O argumento `int status` indica o estado de término do programa, um valor de 0 geralmente indica uma saída bem-sucedida, enquanto valores diferentes de 0 indicam erros;
 - Como o `System.exit` termina o programa abruptamente, deve ser usado com cuidado, normalmente em situações de erro crítico ou quando não há mais nada a ser feito;
- **Casos de Uso:**
 - Tratamento de Erro: Em cenários onde o programa não pode continuar devido a um erro crítico;
 - Interrupção Controlada: Quando o programa atinge um estado em que deve ser encerrado imediatamente, como após confirmar a saída do usuário;

Funções com System.exit

```
public static void verificarAutenticacao(String usuario, String senha){  
    // Se usuário for diferente de 'admin' e senha de 'SenhaSegura'  
    if (!usuario.equals(anObject:"admin") && !senha.equals(anObject:"SenhaSegura")) {  
        System.out.println(x:"Autenticação falhou");  
        System.exit(status:1);  
    } else {  
        System.out.println(x:"Autenticação bem sucedida");  
    }  
}
```

```
// 7 - System.exit  
  
verificarAutenticacao(usuario:"admin", senha:"SenhaSegura");  
  
// Oi não será executado se os dados estiverem incorretos, pois a função irá  
retornar o System.exit(1)  
System.out.println(x:"Oi");
```

```
Autenticação bem sucedida  
Oi
```

Documentando funções

- **O que é Documentação de Função:**
 - Fornece informações detalhadas sobre o que a função faz, seus parâmetros, valor de retorno, e outros detalhes relevantes;
 - Utiliza o formato Javadoc (`/** ... */`) para gerar documentação automática e legível, que pode ser visualizada em IDEs e ferramentas de documentação;
 - Facilita a compreensão e manutenção do código, especialmente em projetos colaborativos ou de grande escala;
- **Componentes da Documentação:**
 - Descrição Geral: Explica o propósito da função e o que ela faz;
 - Parâmetros (`@param`): Descreve os parâmetros de entrada, incluindo seus tipos e o que representa;
 - Valor de Retorno (`@return`): Descreve o que a função retorna, se aplicável;
 - Exceções (`@throws`): Indica quais exceções a função pode lançar, se houver;

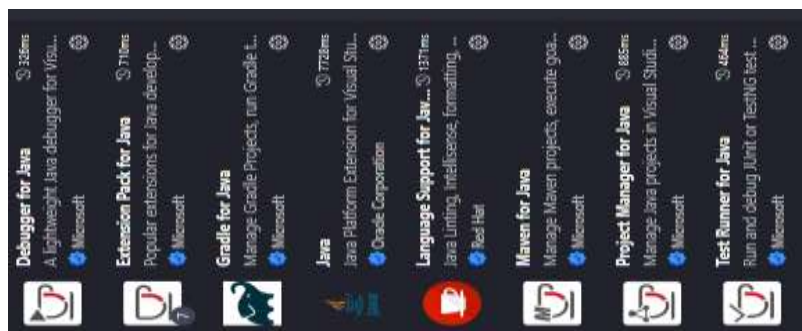
Documentando funções

```
// 8 - Documentação de Funções
```

```
System.out.println(calcularMedia(num1:5, num2:6, num3:7));
```

```
/**
 * Calcula a média de três números inteiros
 *
 * @param num1 O primeiro número/nota a ser enviado
 * @param num2 O segundo número/nota a ser enviado
 * @param num3 O terceiro número/nota a ser enviado
 * @return A média dos três números
 */
public static double calcularMedia(int num1, int num2, int num3){
    return (num1 + num2 + num3) / 3;
}
```

O VSCode (com as extensões) tem autocomplete para criar parcialmente a documentação de Javadoc!



Escopos em Java

- Escopo refere-se à visibilidade e ao tempo de vida de variáveis e objetos dentro de um programa;
- **Escopo Local:** Variáveis declaradas dentro de métodos, blocos if, for, ou while só são acessíveis dentro desses blocos;
- **Escopo de Classe (Global):** Variáveis declaradas fora de métodos, mas dentro de uma classe, são acessíveis por todos os métodos dessa classe;
- **Escopo de Parâmetro:** Parâmetros de função são tratados como variáveis locais dentro do escopo da função;
- **Encapsulamento:** O escopo ajuda a proteger variáveis e métodos de acessos indesejados fora do seu contexto apropriado;

Escopos em java

```
// 9 - Escopos

// Escopo local
int localVar = 10;

if (true) {
    System.out.println(localVar);
}

// Ainda podemos manipular a variável global!
System.out.println(globalVar);

escopoLocal();

// Não podemos acessar essa variável, pois ela está no escopo da
função que criamos!
// System.out.println(testeFuncao); <- ERRO
```

```
// Escopo global
// Static permite que o valor seja utilizado independente da classe
ter sido instanciada ou não!
static int globalVar = 20;
```

```
public static int escopoLocal() {  
    // As variáveis globais também podem ser utilizadas dentro de  
    // funções!  
    System.out.println(globalVar);  
    // Entretanto as variáveis locais não podem ser acessadas!  
    // System.out.println(localVar); <- ERRO  
    // Se passássemos a variável local como parâmetro para a função,  
    // tal como: escopoLocal(localVar) poderíamos acessá-la!  
  
    int testeFuncao = 1;  
    return testeFuncao;  
}
```

{ } Geralmente delimitam um escopo!

O que são Funções Built-in em Java?

- São funções já incorporadas na linguagem Java que **fornecem funcionalidades comuns e essenciais**. Elas são prontas para uso e não precisam ser definidas pelo programador;
- Estão **disponíveis automaticamente** sem necessidade de importação ou definição;
- Para que servem: manipulação de strings, operações matemáticas, conversão de dados, entre outras funcionalidades;
- Geralmente, as funções built-in são **altamente otimizadas** para desempenho;



O que são funções built-in em java

Funções Built-in de String

- **length()**: Retorna o comprimento de uma string, ou seja, o número de caracteres;
- **substring(int beginIndex, int endIndex)**: Extrai uma subsequência da string, começando no índice beginIndex e terminando em endIndex;
- **toUpperCase()**: Converte todos os caracteres da string para letras maiúsculas;
- **replace(char oldChar, char newChar)**: Substitui todas as ocorrências de um caractere especificado por outro;
- Vamos ver na prática!



Funções built-in de String

```
// 10 - Funções Built-in com Strings
String frase = "Ninguém merece..";

// Length
System.out.println(frase.length()); // resultado: 16

// Tal como int = inteiro
// String => S (Letra inicial maiuscula) = Classe / String é
// classe!

System.out.println(frase.substring(beginIndex:0,
endIndex:5)); // resultado: Ningu

// OS ESPAÇOS SÃO CONTABILIZADOS TAMBÉM

System.out.println(frase.toUpperCase());

System.out.println(frase.replace(target:"e",
replacement:"u")); // resultado: Ninguém murucu..

// Os dados podem ser armazenados em variáveis
String fraseModificada = frase.toUpperCase();
System.out.println(fraseModificada);
```


Funções Built-in de Números (Math)

- **Math.sqrt(double a):** Calcula a raiz quadrada de um número;
- **Math.pow(double a, double b):** Eleva um número a à potência b;
- **Math.abs(int a):** Retorna o valor absoluto de um número;
- **Math.max(int a, int b):** Retorna o maior de dois números;
- Vamos utilizá-las!

Funções built-in de números (math)

```
// 11 - Funções do Math

// Raiz quadrada
System.out.println(Math.sqrt(a:26));

// Potência
System.out.println(Math.pow(a:2, b:3));

// Valor absoluto
System.out.println(Math.abs(-10));
System.out.println(Math.abs(a:10));

// Valor máximo (maior)
System.out.println(Math.max(a:100, b:10));

// Atribuição a variáveis
double raizQuadrada = Math.sqrt(a:99);
System.out.println("A raiz quadrada de 99 é: " + raizQuadrada);
```

```
5.0990195135927845
8.0
10
10
100
A raiz quadrada de 99 é: 9.9498743710662
```

Existem incontáveis outras operações que podem ser feitas

Exercícios

```
// Exercício 1
double celsius = 23.0;
double fahrenheit = conversorFahrenheit(celsius);
System.out.println("A temperatura convertida em Fahrenheits é: " + fahrenheit);

// Exercício 2
System.out.println("O fatorial de 5 é: " + calcFatorial(n:5));

// Exercício 3
System.out.println("O número 15... " + parOuImpar(n:15));

// Exercício 4
System.out.println(notas(n:5));

// Exercício 5
verificarAcesso(n:18);

// Exercício 6
int[] numeros = {10, 5, 22, 44, 5};
System.out.println(encontrarMaior(numeros));

scanner.close();
```

```
// Funções

public static double conversorFahrenheit(double n){
    return (n * 1.8) + 32;
}

public static int calcFatorial(int n){
    if (n == 0 || n == 1){
        return 1;
    } else {
        // Reutilizando a função
        return n * calcFatorial(n-1);
    }
}

public static String parOuImpar(int n){
    if (n % 2 == 0){
        return "É par!";
    } else {
        return "É impar!";
    }
}
```

```

public static String notas(int n){
    if (n < 0 || n > 10){
        return "Nota inválida!";
    }
    return switch (n) {
        case 10, 9 -> "Nota " + n + " Resultado: A";
        case 8, 7 -> "Nota " + n + " Resultado: B";
        case 6, 5 -> "Nota " + n + " Resultado: C";
        case 4, 3 -> "Nota " + n + " Resultado: D";
        default -> "Nota " + n + " Resultado: F";
    };
}

public static void verificarAcesso(int n){
    if (n < 18){
        System.out.println(x:"Acesso negado!");
        System.exit(status:0);
    } else {
        System.out.println(x:"Acesso permitido!");
    }
}

// Vamos receber um array [] nomeados de numeros!
public static int encontrarMaior(int[] numeros){
    int maior = numeros[0];
    // Contador, Cond. execução, Incremento
    for (int i = 1; i < numeros.length; i++){
        // Se o número do array for maior que 'maior', ocorre a atribuição!
        if(numeros[i] > maior){
            maior = numeros[i];
        }
    }
    return maior;
}

```

(Veremos loops e arrays logo mais!)

```

A temperatura convertida em Fahrenheits é: 73.4
O fatorial de 5 é: 120
O número 15... É impar!
Nota 5 Resultado: C
Acesso permitido!
44

```

Funções recursivas

- Recursão é a técnica onde uma função chama a si mesma para resolver um problema que pode ser dividido em subproblemas menores e semelhantes ao original;
- Um caso base/cenário é essencial para terminar a recursão, sem ele, a função entraria em um loop infinito;
- Como Funciona: O problema é dividido em subproblemas menores até atingir o caso base, após o qual a solução começa a ser construída ao "subir" a pilha de chamadas;
- Pode ser ineficiente em termos de tempo e memória, especialmente para problemas grandes, devido à sobrecarga de chamadas de função;

Funções recursivas

(Isso pode ser visto em um exercício feito na página anterior!)

Não use em problemas grandes, pois uma vez que a função seja chamada incontáveis vezes pode dar caca

```
// 12 - Funções Recursivas

int soma = somaRecursiva(n:6); // resultado: 21
System.out.println(soma);
}

public static int somaRecursiva(int n ){
    if (n == 1){
        return 1;
    } else {
        // RECURSIVIDADE!
        return n + somaRecursiva(n - 1);
    }
}
```



Funções recursivas não são tão comuns!

Sobrecarga de Funções (Method Overloading)

- Sobrecarga de funções permite definir várias funções com o mesmo nome, desde que tenham assinaturas diferentes (número ou tipo de parâmetros);
- Vantagens:
 - Permite criar diferentes versões de uma função para lidar com diferentes tipos de dados ou diferentes quantidades de argumentos;
 - Mantém o código limpo e organizado, reutilizando o nome da função para tarefas relacionadas;
- Regras para Sobrecarga:
 - Número de Parâmetros: As funções devem diferir no número de parâmetros;
 - Tipo de Parâmetros: As funções podem ter o mesmo número de parâmetros, desde que os tipos sejam diferentes;
 - Tipo de Retorno: Não pode ser usado sozinho para diferenciar funções sobrecarregadas;



Sobrecarga de funções

```
// 13 - Method Overloading

System.out.println(soma(a:10, b:20));
System.out.println(soma(a:15, b:12, c:55));
System.out.println(soma(a:45.5, b:484.2, c:47.2));
```

```
public static int soma(int a, int b){
    return a + b;
}

public static int soma(int a, int b, int c){
    return a + b + c;
}

public static double soma(double a, double b, double c){
    return a + b + c;
}
```

Três funções com o mesmo nome recebendo diferentes argumentos (não é comum)

Funções Anônimas (Lambda Expressions)

- Lambdas são funções anônimas, ou seja, funções sem nome, que podem ser usadas para expressar brevemente pequenas operações ou blocos de código, especialmente em programação funcional;
- Introduzidas no Java 8, as lambdas são uma parte central da API Streams e permitem uma programação mais funcional;
- Sintaxe: Formato: (parâmetros) -> { corpo da função }
- Para expressões simples, o corpo da função pode ser uma única linha sem { };
- Vantagens:
 - Lambdas permitem escrever código mais conciso e legível;
 - São amplamente utilizadas em conjunto com Streams e interfaces funcionais como Runnable e Comparator;



Funções anônimas (Lambda expressions)

(Na verdade ela tem nome sim, é uma variável QUE PRECISA OBRIGATORIAMENTE TER “RUNNABLE” NO INÍCIO, para invocar a função temos de por ‘run’ na frente da função!)

```
// 14 - Funções Anônimas (Lambda Expressions)
// Por ser simples e de única linha, não precisamos por abaixo do
// método main, NÃO ESQUEÇA DO 'RUNNABLE'

Runnable tarefa = () -> System.out.println(x:"Minha função
anônima");

// Para invocar essa função precisamos do método 'run'
tarefa.run();

// (arg1, arg2) -> {}

// Um pouco mais complexo usando algumas bibliotecas para imprimir
// os nomes da lista usando funções anônimas

List<String> nomes = Arrays.asList(...a:"Ana", "Pedro", "Paulo");

// For each usa uma função anônima para exibir cada nome da lista
nomes.forEach(nome -> System.out.println(nome));
```

Funções anônimas são comuns de serem usadas!

Projeto: Conversor de Temperatura

```
Scanner scanner = new Scanner(System.in);

System.out.println(x:"Escolha o tipo de conversão!");
System.out.println(x:"1: Celsius para Fahrenheit");
System.out.println(x:"2: Fahrenheit para Celsius");

int input = scanner.nextInt();

    switch (input) {
        case 1 -> converterCparaF();
        case 2 -> converterFparaC();
        default -> System.out.println(x:"Opção inválida!");
    }
scanner.close();
}

public static void converterCparaF(){
    Scanner scanner = new Scanner(System.in);
    System.out.println(x:"Informe a temperatur em Celsius: ");
    double celsius = scanner.nextDouble();
    double fahrenheit = (celsius * 9/5) + 32;
    System.out.println(celsius + "C convertido é igual a: " +
    fahrenheit + "F");
    scanner.close();
}

public static void converterFparaC(){
    Scanner scanner = new Scanner(System.in);
    System.out.println(x:"Informe a temperatur em Fahrenheit: ");
    double fahrenheit = scanner.nextDouble();
    double celsius = (fahrenheit - 32) * 5/9;
    System.out.println(fahrenheit + "F convertido é igual a: " +
    celsius + "C");
    scanner.close();
}
```

```
Escolha o tipo de conversão!
1: Celsius para Fahrenheit
2: Fahrenheit para Celsius
1
Informe a temperatur em Celsius:
33
33.0C convertido é igual a: 91.4F
```


Mesmo projeto com lógica melhor evitando repetir o Scanner

```
Scanner scanner = new Scanner(System.in);

System.out.println(x: "Escolha o tipo de conversão:");
System.out.println(x: "1: Celsius para Fahrenheit");
System.out.println(x: "2: Fahrenheit para Celsius");
int escolha = scanner.nextInt();

System.out.println(x: "Informe a temperatura:");
double temperatura = scanner.nextDouble();
double resultado = 0;

switch (escolha) {
    case 1 -> {
        resultado = (temperatura * 9 / 5) + 32;
        System.out.println(temperatura + "°C convertido é igual a: " + resultado + "°F");
    }
    case 2 -> {
        resultado = (temperatura - 32) * 5 / 9;
        System.out.println(temperatura + "°F convertido é igual a: " + resultado + "°C");
    }
    default -> System.out.println(x: "Opção inválida!");
}

scanner.close();
```

Projeto: Contador de Palavras

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    String continuar = "s";

    while (continuar.equalsIgnoreCase("s")) {
        contarPalavras();
        System.out.println(x:"Deseja continuar? (s/n)");
        continuar = scanner.nextLine();
    }
    System.out.println(x:"Aplicação encerrada");
    scanner.close();
}

public static void contarPalavras(){
    Scanner scanner = new Scanner(System.in);
    System.out.println(x:"Digite uma frase: ");
    String frase = scanner.nextLine();

    // Transformar a frase em um Array de Strings
    // trim para não contar os espaços em branco como palavras
    // split para dividir a frase em palavras a cada regex
    // regex = expressão regular / \s+ é o símbolo de espaço
    // logo as palavras são divididas a cada
    String[] palavras = frase.trim().split(regex:"\\s+");

    // Java is cursed
    // ['Java', 'is', 'cursed'] = 3

    // Usar lenght para saber quantas palavras tem no array
    int numerosPalavras = palavras.length;

    System.out.println("A frase contém: " + numerosPalavras + "
    palavras");
}
```

```
Digite uma frase:
Olha, é uma vaca no pasto!
A frase contém: 6 palavras
Deseja continuar? (s/n)
n
Aplicação encerrada
```

Esse projeto contém conteúdo ainda não visto tal como Arrays, Loops manipulação dos Arrays!

O que são estruturas de repetição?

- Estruturas de repetição, ou loops, são comandos que permitem a execução repetida de um bloco de código enquanto uma condição específica for verdadeira;
- Para que servem:
 - Automatizar a execução de tarefas repetitivas, economizando tempo e esforço ao evitar a necessidade de escrever o mesmo código várias vezes;
 - Permitem iterar sobre coleções de dados, realizar operações com números sequenciais, e processar entradas de forma eficiente;
- As mais utilizadas:
 - for: Ideal para quando se sabe o número exato de iterações a serem realizadas. Permite o controle de variáveis de início, condição de continuidade e incremento;
 - while: Executa um bloco de código enquanto uma condição é verdadeira. Útil quando o número de iterações não é conhecido antecipadamente;
 - do-while: Similar ao while, mas garante que o bloco de código seja executado pelo menos uma vez, pois a condição é verificada após a execução;



O que são estruturas de repetição (loops)

Estrutura for

- for é uma estrutura de repetição que permite executar um bloco de código um número determinado de vezes;
- Ideal para quando se sabe o **número exato de iterações**;
- **Consiste em três partes:** inicialização, condição e incremento;
- Oferece controle sobre o fluxo de repetição com variáveis de início e incremento;

Estrutura for (loop for)

```

// 1 - For

// Loop de 1 a 5
// Variável de inicialização e contador = i, j, k...
// Condição de até quando ou quantas vezes o loop vai rodar
// Incremento que aumenta o contador até chegar na condição
for(int i = 1; i <= 5; i++) {
    System.out.println("Contador: " + i);
}

// Mostrar cada caractere de uma String
// Length em strings = o número de letras!
// Em arrays e Strings a primeira posição é 0, por isso começamos
com o contador em 0 aqui!
String palavra = "Java";
for(int i = 0; i < palavra.length(); i++){
    // charAt(i) = o caractere na posição i que é atualizado no
    loop!
    System.out.println("Caractere: " + palavra.charAt(i));
}

// Contagem Regressiva
for(int i = 5; i > 0; i--){
    System.out.println("Contador: " + i);
}

```

```

Contador: 1
Contador: 2
Contador: 3
Contador: 4
Contador: 5
Caractere: J
Caractere: a
Caractere: v
Caractere: a
Contador: 5
Contador: 4
Contador: 3
Contador: 2
Contador: 1

```

Estrutura de repetição while

- while é uma estrutura de repetição que executa um bloco de código repetidamente enquanto uma condição específica for verdadeira;
- Sintaxe: while (condição) { bloco de código }
- Execução contínua até que uma condição seja falsa, ideal para quando o número de iterações não é conhecido antecipadamente;

Estrutura de repetição while (loop while)

```
// 2 - While
// Note que aqui temos de declarar a variável i fora do escopo do
loop!
int i = 1;
while (i <= 5) {
    System.out.println("Contador While: " + i);
    i++;
}

int valor = 0;
while(valor != 7){
    // Isso vai gerar um número aleatório até 10!
    // Math sempre gera valores decimais então vamos transformar
    'valor' em int com type cast
    valor = (int)(Math.random() * 1000);
    System.out.println("Valor aleatório: " + valor);
}
```

```
Contador While: 1
Contador While: 2
Contador While: 3
Contador While: 4
Contador While: 5
Valor aleatório: 3
Valor aleatório: 0
Valor aleatório: 9
Valor aleatório: 7
```

Apesar de ter suas utilidades o loop For é bem mais usado!

Do while

- do-while é uma estrutura de repetição que garante que o bloco de código seja executado pelo menos uma vez, verificando a condição após a execução;
- Sintaxe: do { bloco de código } while (condição);
- Ideal quando é necessário executar o código ao menos uma vez antes de verificar a condição;

Do while (loop do while)

```
// 4 - Do While

int j = 10;

do {
    System.out.println("Contador Do While: " + j);
    j--;
} while (j > 0);
// Se 'j' fosse 0 ele rodaria uma vez independentemente da
// condição, uma vez que ela só é verificada após a execução do
// código!

int numero = 0;
do {
    numero = (int)(Math.random() * 10);
    System.out.println("Valor aleatório Do While: " + numero);
} while (numero != 1);
```

```
Contador Do While: 10
Contador Do While: 9
Contador Do While: 8
Contador Do While: 7
Contador Do While: 6
Contador Do While: 5
Contador Do While: 4
Contador Do While: 3
Contador Do While: 2
Contador Do While: 1
Valor aleatório Do While: 7
Valor aleatório Do While: 8
Valor aleatório Do While: 4
Valor aleatório Do While: 7
Valor aleatório Do While: 5
Valor aleatório Do While: 3
Valor aleatório Do While: 1
```

Do while não é tão usado, foque no loop For!

Break em loops

- **break** é uma instrução usada para interromper um loop imediatamente, mesmo que a condição de término original não tenha sido atingida;
- Frequentemente usado para sair de loops antecipadamente quando uma condição específica é satisfeita;
- Termina o loop atual e continua a execução do código após o loop;

Break em loops

```
// 5 - Break

// Break pode ser utilizado em qualquer estrutura de Loop!

for(int x = 0; x <= 10; x++){
    System.out.println("O valor de X é: " + x);

    if(x == 5){
        System.out.println(x:"Parando o Loop");
        break;
    }
}
```

```
O valor de X é: 0
O valor de X é: 1
O valor de X é: 2
O valor de X é: 3
O valor de X é: 4
O valor de X é: 5
Parando o Loop
```

Continue em loops

- **continue** é uma instrução que interrompe a iteração atual do loop e pula para a próxima, ignorando o restante do código dentro do bloco do loop para aquela iteração;
- Usado para **pular certas iterações** quando uma condição específica é atendida, sem interromper o loop completo;
- **Continua a execução do loop na próxima iteração**, ignorando as instruções após o continue na iteração atual;

Continue em loops

```
// 6 - Continue

for(int x = 10; x > 0; x--){
    if(x % 2 == 0){
        System.out.println(x + " É Par!");
        continue;
    }
    // Esse trecho não é executado e sim pulado nos números pares!
    System.out.println("Contador de X: " + x);
}
```

```
10 É Par!
Contador de X: 9
8 É Par!
Contador de X: 7
6 É Par!
Contador de X: 5
4 É Par!
Contador de X: 3
2 É Par!
Contador de X: 1
```

Nested loops

- Nested loops ocorrem quando um loop é colocado dentro de outro loop, permitindo que o loop interno seja executado completamente para cada iteração do loop externo;
- Frequentemente usados para manipulação de matrizes, tabelas e para iterar sobre estruturas de dados mais complexas;
- Podem ser menos eficientes e mais complexos de entender, exigindo cuidado para evitar loops infinitos ou comportamento inesperado;

Nested loops (Loops aninhados)

```
// 7 - Nested Loops

for(int m = 1; m <=3; m++){
    System.out.println(x:"EXTERNO");
    for(int n = 1; n <=3; n++){
        // Note que podemos manipular a variável 'm'
        System.out.println(m + " x " + n + " = " + (m * n));
    }
}

for(int o = 1; o <=10; o++){
    for(int p = 1; p <= o; p++){
        System.out.print(s:"*");
    }
    System.out.println();
}
```

```
EXTERNO
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
EXTERNO
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
EXTERNO
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
*
**
***
****
*****
*****
*****
*****
*****
*****
```

(ATENÇÃO!)

Nested Loops não são tão comuns, mas o exemplo das estrelas costumam aparecer em testes técnicos em entrevistas!

Exercícios

```
// Exercício 01
int soma = 0;
for (int x = 0; x <= 100; x++){
    soma += x;
}
System.out.println("A soma de 1 a 100 é: " + soma); // resultado: 5050

// Exercício 02
int x = 1;
while(x <= 20){
    if(x % 2 == 0){
        System.out.println(x + " é par!");
    }
    x++;
}

// Exercício 03
System.out.println(x:"Digite um número para saber se ele é primo: ");
int primo = scanner.nextInt();

boolean ePrimo = true;
// i começa em 2 porque numeros primos nao podem ser divididos por 1!
for(int i = 2; i < primo; i++){
    if (primo % i == 0){
        ePrimo = false;
        break;
    }
}
if(ePrimo){
    System.out.println("O número " + primo + " é primo!");
} else {
    System.out.println("O número " + primo + " NÃO é primo!");
}
```

Os exercícios 3 e 6 requerem uma atenção especial devido a sua lógica!

```
// Exercício 04

int opcao;
do {
    System.out.println(x:"Exercício 4 - Menu");
    System.out.println(x:"(1) Lanche");
    System.out.println(x:"(2) Batata");
    System.out.println(x:"(3) Suco");
    System.out.println(x:"(4) Sair");
    opcao = scanner.nextInt();
    System.out.println("A opção escolhida foi: " + opcao);
} while (opcao != 4);

// Exercício 05
int numFatorial = 12;
int fatorial = 1;

for(int i = 1; i <= numFatorial; i++){
    fatorial *= i;
}
System.out.println("O fatorial de " + numFatorial + " é igual a: " + fatorial);

// Exercício 06
int numeroParaContar = 1000;
int contador = 0;

while(numeroParaContar != 0){
    // Dividir por 10 remove o último dígito 1000 > 100 > 10 > 1 > 0 e o contador sobe, o
    // processo para quando o número dividido for 0!
    numeroParaContar = numeroParaContar / 10;
    contador++;
}
System.out.println("Número de dígitos: " + contador);
}
```

```
2 é par!
4 é par!
6 é par!
8 é par!
10 é par!
12 é par!
14 é par!
16 é par!
18 é par!
20 é par!
Digite um número para saber se ele é primo:
11
O número 11 é primo!
Exercício 4 - Menu
(1) Lanche
(2) Batata
(3) Suco
(4) Sair
4
A opção escolhida foi: 4
O fatorial de 12 é igual a: 479001600
Número de dígitos: 4
```


Loops com rótulos

- Rótulos (labels) em Java são usados para identificar blocos específicos de código, como loops. Eles permitem o controle direto sobre qual loop deve ser interrompido ou continuado, especialmente em loops aninhados;
- Rótulos são utilizados em conjunto com `break` e `continue` para sair ou pular diretamente para loops externos, em vez de apenas o loop interno;

Loops com rótulos (labels loops)

```
// 8 - Loops com Rótulos (Labels loops)

// Rótulos externos e internos

externo:
for(int i = 0; i < 3; i++){

    for(int j = 0; j < 3; j++){
        // Quando ambos contadores estiverem em 1, o loop externo é interrompido!
        // Usar somente break pararia apenas o loop interno!
        if (i == 1 && j == 1 ){
            break externo;
        }
        System.out.println("i " + i + " , j " + j);
    }
}

for(int i = 0; i < 3; i++){
    System.out.println("Externo " + i);

    interno:
    for(int j = 0; j < 3; j++){

        if (j == 2){
            System.out.println(x:"Parou interno");
            break interno; // o break interno é desnecessário, apenas break
                          funcionaria!
        }

        System.out.println("i " + i + " , j " + j);
    }
}
```

```
i 0 , j 0  
i 0 , j 1  
i 0 , j 2  
i 1 , j 0  
Externo 0  
i 0 , j 0  
i 0 , j 1  
Parou interno  
Externo 1  
i 1 , j 0  
i 1 , j 1  
Parou interno  
Externo 2  
i 2 , j 0  
i 2 , j 1  
Parou interno
```

(ATENÇÃO!)

Lógica meio pah

Erro Comum: Loops Off-by-One

- O erro "off-by-one" ocorre quando um loop executa uma iteração a mais ou a menos do que o necessário devido a um erro na configuração da condição de término;
- Causa: Erro na condição de comparação, como usar `<=` em vez de `<`, ou começar o loop no índice errado;
- Pode levar a resultados inesperados, como omissão de dados ou acesso a índices inválidos em arrays;

Erro comum: Loops off-by-one

```
// 9 - Off by one
// Executar 5 vezes
for(int i = 0; i < 5; i++){
    System.out.println("I: " + i);
}
// A omissão do = em '<=' faz o loop chegar apenas no número 4!
// Ao trabalhar com arrays, errar esse tipo de coisa pode fazer com que o programa
seja interrompido, uma vez que podemos tentar acessar índices não existentes!
```

```
I: 0
I: 1
I: 2
I: 3
I: 4
```

Projeto: Calculadora Simples

V1

```
Scanner scanner = new Scanner(System.in);

int opcao;
do {
    System.out.println(x:"Digite o primeiro número para realizar a operação: ");
    double n1 = scanner.nextDouble();
    System.out.println(x:"Digite o segundo número para realizar a operação: ");
    double n2 = scanner.nextDouble();
    System.out.println(x:"Calculadora Simples - Menu de Operações");
    System.out.println(x:"(1) - Soma");
    System.out.println(x:"(2) - Subtração");
    System.out.println(x:"(3) - Multiplicação");
    System.out.println(x:"(4) - Divisão");
    System.out.println(x:"(5) - Sair");
    System.out.println(x:"Escolha a operação!");
    opcao = scanner.nextInt();
    if (n1 == 0 || n2 == 0 && opcao == 4){
        System.out.println(x:"Não é possível dividir por zero, encerrando programa!");
        break;
    }
    switch (opcao) {
        case 1 -> System.out.println(n1 + " + " + n2 + " = " + (n1+n2));
        case 2 -> System.out.println(n1 + " - " + n2 + " = " + (n1-n2));
        case 3 -> System.out.println(n1 + " x " + n2 + " = " + (n1*n2));
        case 4 -> System.out.println(n1 + " / " + n2 + " = " + (n1/n2));
        default -> {
            System.out.println(x:"Encerrando");
            break;
        }
    }
} while (opcao != 5);
scanner.close();
```

```
Digite o primeiro número para realizar a operação:
50
Digite o segundo número para realizar a operação:
8
Calculadora Simples - Menu de Operações
(1) - Soma
(2) - Subtração
(3) - Multiplicação
(4) - Divisão
(5) - Sair
Escolha a operação!
3
50.0 x 8.0 = 400.0
Digite o primeiro número para realizar a operação:
|
```

V2 Com funções

```
Scanner scanner = new Scanner(System.in);
int opcao;
do {
    System.out.println(x:"Digite o primeiro número para realizar a operação: ");
    double n1 = scanner.nextDouble();
    System.out.println(x:"Digite o segundo número para realizar a operação: ");
    double n2 = scanner.nextDouble();
    System.out.println(x:"Calculadora Simples - Menu de Operações");
    System.out.println(x:"(1) - Soma");
    System.out.println(x:"(2) - Subtração");
    System.out.println(x:"(3) - Multiplicação");
    System.out.println(x:"(4) - Divisão");
    System.out.println(x:"(5) - Sair");
    System.out.println(x:"Escolha a operação!");
    opcao = scanner.nextInt();
    if (n1 == 0 || n2 == 0 && opcao == 4){
        System.out.println(x:"Não é possível dividir por zero, encerrando programa!");
        break;
    }
    switch (opcao) {
        case 1 -> System.out.println(n1 + " + " + n2 + " = " + soma(n1, n2));
        case 2 -> System.out.println(n1 + " - " + n2 + " = " + subtracao(n1, n2));
        case 3 -> System.out.println(n1 + " x " + n2 + " = " + multiplicacao(n1, n2));
        case 4 -> System.out.println(n1 + " / " + n2 + " = " + divisao(n1, n2));
        default -> {
            System.out.println(x:"Encerrando");
            break;
        }
    }
} while (opcao != 5);
scanner.close();
}

public static double soma(double a, double b){
    return a + b;
}
public static double subtracao(double a, double b){
    return a - b;
}
public static double multiplicacao(double a, double b){
    return a * b;
}
public static double divisao(double a, double b){
    return a / b;
}
```

Correção em ambas versões

```
if ((n1 == 0 || n2 == 0) && opcao == 4){
```

Projeto: Jogo de Adivinhação

```
Scanner scanner = new Scanner(System.in);

int alvo = (int) (Math.random() * 100);
int contagem = 0;
int palpite;

do {
    System.out.println(x:"Tente acertar o número de 1 a 100");
    System.out.println(x:"Qual o seu palpite: ");
    palpite = scanner.nextInt();
    contagem++;
    if (alvo < palpite){
        System.out.println(x:"O número aleatório é MENOR que seu palpite!");
    } else if (alvo > palpite){
        System.out.println(x:"O número aleatório é MAIOR que seu palpite!");
    }
} while (palpite != alvo);
if (palpite == alvo){
    System.err.println("Você achou o número aleatório! Foram necessárias " + contagem + " tentativas!");
    scanner.close();
}
```

```
Tente acertar o número de 1 a 100
Qual o seu palpite:
60
O número aleatório é MENOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
50
O número aleatório é MENOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
40
O número aleatório é MAIOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
45
O número aleatório é MAIOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
49
O número aleatório é MENOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
47
O número aleatório é MENOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
48
O número aleatório é MENOR que seu palpite!
Tente acertar o número de 1 a 100
Qual o seu palpite:
46
Você achou o número aleatório! Foram necessárias 8 tentativas!
```



```
int alvo = (int) (Math.random() * 100) + 1;
```

Correção: No código base o contador ia apenas até o número 99 com a adição de +1 ele chega ao número 100!

O que são arrays?

- Arrays são **estruturas de dados** que armazenam múltiplos valores do mesmo tipo em uma única variável;
- Arrays **podem armazenar tipos primitivos** (como int, double, char) ou objetos (como String ou classes personalizadas);
- Os elementos de um array são acessados por **índices, que começam em 0**;
- O tamanho de um array é definido na sua criação e não pode ser alterado;
- Arrays são usados para armazenar listas de dados, como números, nomes, ou qualquer coleção de elementos homogêneos;

O que são arrays

Sintaxe de arrays

- Arrays são declarados **especificando o tipo** dos elementos seguido de colchetes ([]);
- Arrays podem ser inicializados com valores específicos no momento da declaração ou instanciados com um **tamanho fixo**;
- Os elementos de um array são acessados usando **índices que começam em 0**;
- **Sintaxe:** tipo[] nomeArray = new tipo[tamanho]; ou tipo[] nomeArray = {valores};

Sintaxe de arrays

```
// 1 - Sintaxe

int[] numeros = {1,2,3,4,5,6,7,8};
// [] após o tipo de dado, seguido do nome, {elementos} separados
por vírgula!

// Primeiro elemento, lembre-se que comecem em 0!
System.out.println("Primeiro elemento: " + numeros[0]);

// Tamanho fixo, porém vazio
String[] frutas = new String[3];
// Podemos inserir elementos nos índices 0, 1, e 2!

// Atribuição
frutas[0] = "Maça";
frutas[1] = "Banana";
frutas[2] = "Manga";

// A impressão de arrays é feita com a classe 'Arrays.toString'
System.out.println(Arrays.toString(frutas));

double[] precos = {1.99, 1.45, 2.45};

// 2.45 -> 5.00
precos[2] = 5.00;

System.out.println(Arrays.toString(precos));
```

```
Primeiro elemento: 1
[Maça, Banana, Manga]
[1.99, 1.45, 5.0]
```

Naturalmente não há como inserir um int em um array de Strings e vice-versa

Loop em arrays

- Loops são usados para iterar sobre os elementos de um array, acessando e manipulando cada elemento individualmente;
- Você pode usar **for**, **while**, ou loops aprimorados (for-each) para iterar sobre arrays;
- Percorrer todos os elementos, somar valores, encontrar elementos específicos, modificar valores;

Loops em arrays

```
// 2 - Loops em Arrays

// Somar todos os elementos de um array
int soma = 0;
// 8 porque o array contém 8 elementos!
//     for(int i = 0; i < 8; i++){
// Normalmente não fazemos assim porque não sabemos o tamanho dos arrays
// use LENGTH, ele irá retornar a quantidade de elementos do array!
for(int i = 0; i < numeros.length; i++){
    soma += numeros[i];
}
System.out.println("A soma dos elementos no array é: " + soma);

// For Each (Estrutura de loops para arrays)
/* Não temos controle sobre a quantidade de suas iterações
Ele irá iterar de acordo com a quantidade de elementos no array
Nele temos de nomear o item, isso é o singular do elemento do array
Pessoas -> Pessoa
Frutas -> Fruta
Logo, para cada Fruta em Frutas, faça isso {}
*/
for(String fruta : frutas){
    System.out.println("A fruta da vez é: " + fruta);
}

// Encontrar o maior valor de um array
int [] valores = {10, 25, 8, 22, 1};
// Usamos o primeiro índice como base, os outros numeros serão comparados com este
int maiorValor = valores[0];

int j = 0;
// Enquanto 'j' for menor que o comprimento do array, faça isso {}
while(j < valores.length){
    // Se valores no índice 'j' for maior que 'maiorValor' atribua esse valor a ele
    if(valores[j] > maiorValor){
        maiorValor = valores[j];
    }
    // Incremento por fora
    j++;
}
System.out.println(maiorValor); // resultado: 25
```

```
A soma dos elementos no array é: 36
A fruta da vez é: Maça
A fruta da vez é: Banana
A fruta da vez é: Manga
25
```

Loop for-each em Arrays

- O loop for-each permite iterar sobre os elementos de um array de maneira simplificada, sem necessidade de gerenciar índices;
- Ideal para percorrer todos os elementos de um array quando não é necessário modificar o índice ou acessar elementos fora da sequência;
- Sintaxe: for (tipo variavel : array) { // bloco de código }

For each em arrays

```
// 3 - For Each
for(int numero : numeros){
    System.out.println("O número é: " + numero);
}

// Concatenar elementos de um array
String[] palavras = {"Java", "Is", "Cursed!"};
String frase = "";

for(String palavra : palavras){
    // Operador de atribuição também funciona com strings
    frase += palavra + " ";
}
System.out.println(frase);

// Verificar se valor esta presente no array
char[] letras = {'a', 'e', 'i', 'o', 'u'};
char letraProcurada = 'i';

for(char letra : letras){
    if (letra == letraProcurada){
        System.out.println("Encontramos a letra: " + letra);
        // Break também funciona no For Each
        break;
    }
}
```

```
O número é: 1
O número é: 2
O número é: 3
O número é: 4
O número é: 5
O número é: 6
O número é: 7
O número é: 8
Java Is Cursed!
Encontramos a letra: i
```

For x For-each

- **Usar for:**
 - Ideal quando você precisa acessar ou manipular o índice de cada elemento, como em casos onde o índice determina a lógica ou o comportamento do código;
 - Útil quando a iteração precisa pular elementos, ou quando o loop deve ser interrompido ou reiniciado em uma posição específica;
 - Recomendado quando você precisa modificar diretamente os elementos do array usando o índice;
- **Usar for-each:**
 - Ideal para percorrer todos os elementos de um array ou coleção sem a necessidade de modificar os elementos ou controlar o índice;
 - Melhor para código que precisa ser limpo e fácil de entender, onde a manipulação do índice não é necessária;
 - Evita erros relacionados ao índice, como `ArrayIndexOutOfBoundsException`, porque o for-each não expõe o índice diretamente;

Diferenças entre For e For-each

Ou seja quando usarmos o índice, `array[0]`, `array[1]` e etc, provavelmente usaremos loops **For**

O erro mencionado no final se refere principalmente a quando tentamos acessar um índice que não existe no array, como o **For-each** percorre automaticamente o array inteiro, esse erro não ocorre!

Loops com if

- Combinar loops com instruções if permite executar condições específicas dentro de cada iteração do loop;
- Usos: Filtrar elementos, interromper ou continuar a execução do loop com base em condições lógicas;
- Usar if dentro de loops adiciona lógica condicional, permitindo decisões dinâmicas em cada iteração;

Loops com if

```
// 4 - Loops com If
// Filtrar e somar numeros pares
// int[] numeros = {1,2,3,4,5,6,7,8};
int somaPares = 0;
for (int numero : numeros){
    if(numero % 2 == 0){
        somaPares += numero;
    }
}
System.out.println(somaPares); // resultado: 20

// Exibir valores maiores que um determinado valor
int[] nums = {12, 6, 18, 24, 48, 5};
int limite = 10;

for(int i = 0; i < nums.length; i++){
    if(nums[i] > limite){
        System.out.println("O seguinte valor é maior que " + limite + ": " + nums[i]);
    }
}

// Outra forma:
// for(int num : nums){
//     if (num > 10){
//         System.out.println("O seguinte valor é maior que " + limite + ": " + num);
//     }
// }

String[] linguagens = {"Java", "C", "Python", "PHP"};
String linguagemAlvo = "Python";

for(String linguagem : linguagens){
    // O VSCode recomenda usar .equals para comparar Strings!
    if(linguagem == linguagemAlvo){
        System.out.println(x:"Linguagem encontrada, parando o loop");
        break;
    } else {
        System.out.println(x:"Linguagem ainda não encontrada");
    }
}
```



```
20
O seguinte valor é maior que 10: 12
O seguinte valor é maior que 10: 18
O seguinte valor é maior que 10: 24
O seguinte valor é maior que 10: 48
Linguagem ainda não encontrada
Linguagem ainda não encontrada
Linguagem encontrada, parando o loop
```

Atualizações de valores em arrays

- Atualizar valores em um array significa modificar os elementos do array em índices específicos, seja através de loops ou diretamente;
- Usos: Alterar valores com base em cálculos, substituir elementos, ou aplicar operações em todos os elementos do array;
- Permite manipular dados armazenados em arrays para refletir mudanças dinâmicas ou realizar operações em massa;

Atualização de valores em arrays

```
// 5 - Atualizar Valores
for (int i = 0; i < numeros.length; i++){
    // Multiplica o numero do indice [i] por 2
    numeros[i] *= 2;
}

for(int numero : numeros){
    System.out.println("Numero dobrado: " + numero);
}

// FOR EACH NÃO PERMITE ALTERAÇÃO DIRETA NOS ELEMENTOS DO ARRAY (SIM, DE NOVO!)
// Replica errônea abaixo!
// for(int numero : numeros){
//     numero *= 2;
// }

// Manipulação do indice diretamente
numeros[5] = 1;

// Substituindo um valor String por outro, Maça está no índice 0!
for(int i = 0; i < frutas.length; i++){
    if(frutas[i].equals(anObject:"Maça")){
        frutas[i] = "Abacate";
    }
}

System.out.println(frutas[0]);
```

```
Numero dobrado: 2  
Numero dobrado: 4  
Numero dobrado: 6  
Numero dobrado: 8  
Numero dobrado: 10  
Numero dobrado: 12  
Numero dobrado: 14  
Numero dobrado: 16  
Abacate
```

Método toString de Arrays

- O método `toString` da classe `Arrays` é usado para converter um array em uma representação em `String`, permitindo a exibição direta do conteúdo do array;
- Uso Comum: Facilita a visualização dos elementos de um array em uma única linha, útil para debug e saída de dados;
- Sintaxe: `Arrays.toString(array)` converte o array em uma string formatada;

Método toString de arrays

É mais usado em debugs de arrays, quando queremos ver seu conteúdo!

Várias classes no Java possuem métodos utilitários, o `Arrays` é uma delas e precisa ser importado dessa forma:

```
import java.util.Arrays;
```

```
// 6 - Método toString  
String dadosNumericos = Arrays.toString(numeros);  
System.out.println(dadosNumericos); // [2, 4, 6, 8, 10, 1, 14, 16]  
  
String dadosFrutas = Arrays.toString(frutas);  
System.out.println(dadosFrutas); // [Abacate, Banana, Manga]  
  
int[] teste = new int[3];  
System.out.println(Arrays.toString(teste)); // [0, 0, 0]
```

Bem útil quando não queremos fazer loops para ver os valores de um array!

Maneiras de Adicionar Novos Itens a Arrays

- Arrays em Java têm tamanho fixo, portanto, para adicionar novos itens, é necessário criar um novo array ou usar estruturas como ArrayList;
- Métodos Comuns: Criar um novo array maior e copiar os elementos, usar ArrayList para manipulação dinâmica de elementos;

Maneiras de adicionar novos itens a arrays

ArrayList é um método relativamente novo, antigamente se criava um novo array na maioria das situações, ele permite adicionar novos elementos de forma mais prática

Note que:

```
java.util.ArrayList<String> listaFrutas = new  
java.util.ArrayList<>(Arrays.asList("Maça", "Banana", "Laranja"));
```

pode ser escrito da seguinte forma):

```
ArrayList<String> listaFrutas = new ArrayList<>(Arrays.asList("Maçã",  
"Banana", "Laranja"));
```

O primeiro método é apenas para não termos de importar as utilidades na aplicação lá
encima

Import java.util.ArrayList;

Em sistemas legado, arrayList talvez não funcione!

Quando usar arrayList para se referenciar a números inteiros (ints) use a palavra Integers!!!

```
// 7 - Adicionando novos itens a arrays

// Método 1: Criando um novo array e copiando os elementos com loops

// Criamos um array que tem como base o comprimento de 'numeros' + 1 índice
int [] novoArray = new int[numeros.length + 1];

// Loop para copiar os dados de 'numeros' para 'novoArray'
for (int i = 0; i < numeros.length; i++){
    novoArray[i] = numeros[i];
}

System.out.println(Arrays.toString(novoArray)); // [2, 4, 6, 8, 10, 1, 14, 16, 0]

// O ultimo elemento do array é o 0, se quissemos acessar e alterar ele, poderíamos
fazer da seguinte forma, note que não sabemos quantos elementos existem no array,
então só subtraímos 1 do comprimento total dele!

novoArray[novoArray.length - 1] = 7;
System.out.println(Arrays.toString(novoArray)); // [2, 4, 6, 8, 10, 1, 14, 16, 7]
```

```
// Método 2: Utilizando o arrayCopy para copiar os elementos

// Criação do array que irá guardar os elementos de 'frutas'
String[] novoFrutas = new String[frutas.length + 1];

// Sintaxe: Array antigo, posição inicial, Array novo, posição inicial, e limite da
cópia, como queremos copiar tudo usamos o comprimento do array 'frutas' como base!
System.arraycopy(frutas, srcPos:0, novoFrutas, destPos:0, frutas.length);

novoFrutas[novoFrutas.length - 1] = "Abacaxi";

System.out.println(Arrays.toString(novoFrutas)); // [Abacate, Banana, Manga, Abacaxi]

// Método 3: Usando ArrayList

// <Tipo de dado> nomeDoArray
java.util.ArrayList<String> listaFrutas = new java.util.ArrayList<>(Arrays.asList(...
a:"Maça", "Banana", "Laranja"));

// A impressão de ArrayLists não precisa de 'toString'
System.out.println(listaFrutas); // [Maça, Banana, Laranja]

// ArrayLists permitem usar métodos para realizar ações, tal como adicionar
elementos no array!
listaFrutas.add(e:"Manga");
System.out.println(listaFrutas); // [Maça, Banana, Laranja, Manga]
```

```
[2, 4, 6, 8, 10, 1, 14, 16, 0]
[2, 4, 6, 8, 10, 1, 14, 16, 7]
[Abacate, Banana, Manga, Abacaxi]
[Maça, Banana, Laranja]
[Maça, Banana, Laranja, Manga]
```

Reference Trap

- Reference trap ocorre quando duas variáveis apontam para o mesmo objeto na memória, causando modificações não intencionais ao objeto original ao alterar a cópia;
- Ao copiar arrays ou objetos, a nova variável pode compartilhar a referência ao mesmo espaço de memória, em vez de criar uma nova instância;
- Alterações em uma variável refletem na outra, pois ambas referenciam o mesmo objeto;

Problema de reference trap

Pode ser resolvido através da simples clonagem do array requisitado, dessa forma podemos alterar o valor do clone sem alterar o valor do array original

```
// 8 - Problema de Reference Trap
int[] arrayOriginal = {1,2,3};
// Vamos supor que já programamos bastante coisa e queremos reutilizar os valores
desse array
int[] arrayCopia = arrayOriginal;

arrayCopia[0] = 10;

// Perceba que o valor 10 foi inserido incorretamente no arrayOriginal
System.out.println(Arrays.toString(arrayOriginal)); // [10, 2, 3]
System.out.println(Arrays.toString(arrayCopia)); // [10, 2, 3]

// Podemos evitar isso através do processo de clonagem de arrays
int[] arrayClone = arrayOriginal.clone();

// Se alterarmos os valores do clone...
arrayClone[2] = 100;

// Não alteramos o valor do array que foi clonado!
System.out.println(Arrays.toString(arrayOriginal)); // [10, 2, 3]
System.out.println(Arrays.toString(arrayClone)); // [10, 2, 100]
```

Arrays 2D (matrizes)

- Arrays 2D são arrays de arrays, onde cada elemento do array principal é um outro array, permitindo a criação de estruturas de grade (matriz);
- Um array 2D é criado especificando o número de linhas e colunas;
- Os elementos são acessados usando dois índices: um para a linha e outro para a coluna;
- Elementos de um array 2D podem ser atribuídos diretamente ou através de loops;

Arrays 2D (matrizes)

```
// 9 - Arrays 2D (Matrizes)
// [[1, 2], [2, 3]]
// array[0][1] = 2

// Note o [][] para inicializar a matriz!
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.println(matriz[1][1]); // 5
System.out.println(matriz[2][2]); // 9
System.out.println(matriz[2][0]); // 7

// Criando um array 2D vazio e preenchendo-o, dessa forma criamos uma estrutura
similar a de cima!
int[][] tabela = new int[3][3];
```



```
// Sem loops
tabela[0][0] = 10;
tabela[1][1] = 20;
tabela[2][2] = 30;

// Impressão de matrizes
// int[] se refere ao tipo de dado, um array de inteiros!
for(int[] linha: tabela){
    System.out.println(Arrays.toString(linha));
    // [10, 0, 0]
    // [0, 20, 0]
    // [0, 0, 30]
}

// Com loops aninhados
int[][] grade = new int[4][5];

for(int m = 0; m < grade.length; m++){
    for (int n = 0; n < grade[m].length; n++){
        grade[m][n] = m * n;
    }
}

for(int[] linha: grade){
    System.out.println(Arrays.toString(linha));
    // [0, 0, 0, 0, 0]
    // [0, 1, 2, 3, 4]
    // [0, 2, 4, 6, 8]
    // [0, 3, 6, 9, 12]
    // Os valores que são multiplicados começam em 0!
}
```

```
[10, 0, 0]
[0, 20, 0]
[0, 0, 30]
[0, 0, 0, 0, 0]
[0, 1, 2, 3, 4]
[0, 2, 4, 6, 8]
[0, 3, 6, 9, 12]
```

(ATENÇÃO!)

A lógica dos loops pode parecer confusa!

Os exercícios a partir daqui serão nomeados dada a complexidade superior deles!

Exercícios

Mk1 com arrayList e usando **Collections**

Inversão de valores do array

```
// Exercício 01:

ArrayList<Integer> numeros = new ArrayList<>(Arrays.asList());
System.out.println(x:"Digite o primeiro numero: ");
int n1 = scanner.nextInt();
System.out.println(x:"Digite o segundo numero: ");
int n2 = scanner.nextInt();
System.out.println(x:"Digite o terceiro numero: ");
int n3 = scanner.nextInt();
numeros.add(n1);
numeros.add(n2);
numeros.add(n3);

System.out.println("Antes de inverter: " + numeros);

Collections.reverse(numeros);

System.out.println("Depois de inverter: " + numeros);
```

```
Digite o primeiro numero:
30
Digite o segundo numero:
66
Digite o terceiro numero:
84
Antes de inverter: [30, 66, 84]
Depois de inverter: [84, 66, 30]
```

Mk2 método do professor!

```
// mk2 (professor)

int[] arrayOriginal = {1, 2, 3, 4, 5};
int[] arrayInvertido = new int[arrayOriginal.length];

for(int i = 0; i < arrayOriginal.length; i++){
    arrayInvertido[i] = arrayOriginal[arrayOriginal.length - 1 - i];
}

System.out.println(Arrays.toString(arrayOriginal));
System.out.println(Arrays.toString(arrayInvertido));
```

```
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]
```

Encontrar elemento mais frequente no array

```
// Exercício 2:
int[] arraysInteiros = {1,2,3,4,5,6,4,4,5,5};
int maisFrequente = arraysInteiros[0];
int maxContagem = 1;
for(int i = 0; i < arraysInteiros.length; i++){

    int contagem = 0;

    for(int j = 0; j < arraysInteiros.length; j++){
        if(arraysInteiros[j] == arraysInteiros[i]){
            contagem++;
        }
    }
    if(contagem > maxContagem){
        maxContagem = contagem;
        maisFrequente = arraysInteiros[i];
    }
}
System.out.println("O mais frequente é: " + maisFrequente);
```

```
O mais frequente é: 4
```

Transpor uma matriz 2D (troca de linhas por colunas)

```
// Exercício 03:

int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6}
};

int[][] matrizTransposta = new int[matriz[0].length][matriz.length];

for(int i = 0; i < matriz.length; i++){
    for(int j = 0; j < matriz[0].length; j++){
        matrizTransposta[j][i] = matriz[i][j];
    }
}

for(int[] linha : matrizTransposta){
    System.out.println(Arrays.toString(linha));
    // Ao invés de 2 linhas e 3 colunas, temos 2 colunas e 3 linhas
}
```

```
[1, 4]
[2, 5]
[3, 6]
```

(ATENÇÃO!) Os exercícios estão ficando cabulosos :v

Foque em Loops com Matrizes!!!!

Substituir valores em um array com condição

```
// Exercício 04:
int[] antesSub = {-10, 20, -3, 4, -1, 20};

System.out.println(Arrays.toString(antesSub));
for(int i = 0; i < antesSub.length; i++){
    if(antesSub[i] < 0){
        antesSub[i] = 0;
    }
}
System.out.println(Arrays.toString(antesSub));
```

```
[ -10, 20, -3, 4, -1, 20]
[ 0, 20, 0, 4, 0, 20]
```

Remover elementos duplicados de um array

```
// Exercício 05:
int[] duplicados = {1, 1, 2, 2, 3, 4, 5};
System.out.println(Arrays.toString(duplicados));

// Array numérico sem qtd definida
ArrayList<Integer> semDuplicados = new ArrayList<>();

for(int numero: duplicados){
    // ! inverte, então se 'semDuplicados' NÃO conter o numero, ele o adiciona em
    // 'semDuplicados'
    if(!semDuplicados.contains(numero)){
        semDuplicados.add(numero);
    }
}

System.out.println(semDuplicados);
```

```
[1, 1, 2, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

Ordenação de arrays

- **Ordenação com Arrays.sort():**
 - Ordena arrays de tipos primitivos em ordem crescente;
 - Ordena arrays de objetos que implementam Comparable;
 - Pode usar Comparator para definir ordem personalizada;
- **Ordenação de Arrays Multidimensionais:**
 - Ordenação baseada em uma coluna específica;
 - Comparator pode ser utilizado para definir critérios complexos;

Ordenação de arrays

Método de impressão de matriz sem loops:

```
System.out.println(Arrays.deepToString(matriz)); // [[2, 3, 1],  
[4, 5, 1], [3, 8, 9]]
```



```

// 1 - Ordenação

// Array de inteiros

int[] numbers = {4, 5, 1, 2, 3, 7, 6};
System.out.println(Arrays.toString(numbers)); // [4, 5, 1, 2, 3, 7, 6]

// Esse método MODIFICA o array original!!!
Arrays.sort(numbers);
System.out.println(Arrays.toString(numbers)); // [1, 2, 3, 4, 5, 6, 7]

// Ordenação com Comparator
String[] names = {"Ana", "Carlos", "João", "Bob"};
Arrays.sort(names);
// Ele irá ordenar alfabeticamente!
System.out.println(Arrays.toString(names)); // [Ana, Bob, Carlos, João]

// Ordem inversa alfabeticamente dos nomes!
Arrays.sort(names, Comparator.reverseOrder());
System.out.println(Arrays.toString(names)); // [João, Carlos, Bob, Ana]

// Ordenação de matriz
int[][] matriz = {
    {4, 5, 1},
    {3, 8, 9},
    {2, 3, 1}
};

// ERRO -> Arrays.sort(matriz); / NÃO SE PODE USAR SORT EM ARRAYS 2D!
// Método correto:
// Em: a -> a[1] estamos escolhendo a coluna 1 como critério de ordenação!
Arrays.sort(matriz, Comparator.comparingInt(a -> a[1]));

for(int[] linha : matriz){
    System.out.println(Arrays.toString(linha));
    // Note que a coluna 1 está ordenada: 3, 5, 8!
    // [2, 3, 1]
    // [4, 5, 1]
    // [3, 8, 9]
}

```

Lembre-se usamos a coluna como critério de ordenamento da matriz!

Manipulação avançada de arrays

- Cópia de Arrays com `Arrays.copyOf()`:
 - Cria uma nova cópia de um array com tamanho especificado;
 - Pode ser utilizado para truncar ou expandir arrays;
- Preenchimento de Arrays com `Arrays.fill()`:
 - Preenche todos os elementos do array com um valor específico;
 - Útil para inicialização de arrays com valores padrão;
- Transformação de Arrays com `Arrays.stream()`:
 - Converte um array em um Stream para manipulação funcional;
 - Permite filtragem, mapeamento, redução e outras operações;

Manipulação avançada de arrays

Veremos o **stream** com mais profundidade mais a frente, ele permite usar métodos exclusivos!

```
// 2 - Manipulação avançada de arrays
int[] original = {1, 2, 2, 4};

// Cópia de arrays, original.length é o valor máximo que queremos copiar!
int[] copia = Arrays.copyOf(original, original.length);
System.out.println(Arrays.toString(copia)); // [1, 2, 2, 4]

// Fill ou preenchimento
int[] numeros = new int[5];
System.out.println(Arrays.toString(numeros)); // [0, 0, 0, 0, 0]

Arrays.fill(numeros, val:5);
System.out.println(Arrays.toString(numeros)); // [5, 5, 5, 5, 5]

// Transformação de array para stream / VEREMOS ISSO COM MAIS ATENÇÃO DEPOIS!
int[] values = {1, 2, 3, 4, 5};

// Soma dos elementos
int soma = Arrays.stream(values).sum();
System.out.println(soma); // 15
```

Array dinâmico

- Conceito de Arrays Dinâmicos:
 - Arrays em Java são de tamanho fixo; arrays dinâmicos ajustam o tamanho automaticamente;
 - **ArrayList** é a implementação mais comum de arrays dinâmicos;
- Adicionar Elementos com **ArrayList.add()**:
 - Permite adicionar elementos ao final do array;
 - Não requer redimensionamento manual;
- Remover Elementos com **ArrayList.remove()**:
 - Remove elementos pelo índice ou valor;
 - Ajusta automaticamente o tamanho do array;

Array dinâmico

```
// 3 - Arrays Dinâmicos

ArrayList<String> frutas = new ArrayList<>();
System.out.println(frutas); // []

frutas.add(e:"Banana");
frutas.add(e:"Maçã");
System.out.println(frutas); // [Banana, Maçã]

for(String fruta : frutas){
    System.out.println(fruta);
    // Banana
    // Maçã
}

// Remoção por nome
frutas.remove(o:"Banana");
System.out.println(frutas); // [Maçã]
// Remoção por índice
frutas.remove(index:0);
System.out.println(frutas); // []

frutas.add(e:"Maçã");

// Atribuição de índice em uma variável, sendo 0 o índice!
String frutaEspecifico = frutas.get(index:0);
System.out.println(frutaEspecifico); // Maçã

// ArrayList não permite print por índice diretamente!
// ERRO -> System.out.println(frutas[0]);
```

Projeto: Jogo da Forca

Mk1 (feito com ajuda rs)

```
String palavraSecreta = "macaco";
String[] palavraOculta = new String[palavraSecreta.length()];
Arrays.fill(palavraOculta, val: "_");
int tentativas = 3;

Scanner scanner = new Scanner(System.in);

while (tentativas > 0) {
    System.out.println(x:"Tente adivinhar a palavra oculta:");
    System.out.println(String.join(delimiter:" ", palavraOculta));
    System.out.println(x:"Digite uma letra:");
    String letra = scanner.nextLine();

    boolean acerto = false;
    for (int i = 0; i < palavraSecreta.length(); i++) {
        if (palavraSecreta.charAt(i) == letra.charAt(index:0)) {
            palavraOculta[i] = letra;
            acerto = true;
        }
    }

    if (!acerto) {
        tentativas--;
        System.out.println("Letra incorreta! Tentativas restantes: " + tentativas);
    }

    if (String.join(delimiter:"", palavraOculta).equals(palavraSecreta)) {
        System.out.println("Parabéns! Você acertou a palavra: " + palavraSecreta);
        break;
    }

    if (tentativas == 0) {
        System.out.println("Suas tentativas acabaram! A palavra era: " +
            palavraSecreta);
    }
}

scanner.close();
```

```
Tente adivinhar a palavra oculta:
_ _ _ _ _
Digite uma letra:
m
Tente adivinhar a palavra oculta:
m _ _ _ _
Digite uma letra:
a
Tente adivinhar a palavra oculta:
m a _ a _ _
Digite uma letra:
c
Tente adivinhar a palavra oculta:
m a c a c _
Digite uma letra:
o
Parabéns! Você acertou a palavra: macaco
```

Mk2 (Versão do professor)

```
// Mk2 (Versão do professor)
System.out.println(x:"Digite a palavra secreta:");
String palavraSecreta = scanner.next().toUpperCase();
char[] palavraOculta = new char[palavraSecreta.length()];
Arrays.fill(palavraOculta, val:'-');

int tentativas = 5;
boolean venceu = false;

while(tentativas > 0){
    // Retorna a representação em string de valores em char
    System.out.println("Palavra: " + String.valueOf(palavraOculta));
    System.out.println("Tentativas: " + tentativas);
    System.out.println(x:"Digite uma letra: ");
    // Dessa forma mesmo se o usuário digitar uma palavra inteira contaremos apenas
    // com a primeira letra e a tornaremos maiuscula!
    char letra = scanner.next().toUpperCase().charAt(index:0);
    boolean acertou = false;

    for(int i = 0; i < palavraSecreta.length(); i++){
        if(palavraSecreta.charAt(i) == letra){
            palavraOculta[i] = letra;
            acertou = true;
        }
    }
    // Se o usuário não acertar
    if(!acertou){
        tentativas--;
        System.out.println(x:"Letra incorreta!");
    } else {
        System.out.println(x:"Letra correta!");
    }

    // Se o usuário acertar a palavra com tentativas restantes
    if(String.valueOf(palavraOculta).equals(palavraSecreta)){
        venceu = true;
        break;
    }
}
```

```
// Condição de vitória
if(venceu){
    System.out.println("Parabéns, restavam-lhe apenas " + tentativas + "
    tentativas!");
} else {
    System.out.println(x:"Você perdeu!");
}
System.out.println("A palavra era: " + palavraSecreta);
scanner.close();
```

```
Digite a palavra secreta:
Boticario
Palavra: -----
Tentativas: 5
Digite uma letra:
b
Letra correta!
Palavra: B-----
Tentativas: 5
Digite uma letra:
o
Letra correta!
Palavra: BO-----O
Tentativas: 5
Digite uma letra:
ti
Letra correta!
Palavra: BOT-----O
Tentativas: 5
Digite uma letra:
i
Letra correta!
Palavra: BOTI---IO
Tentativas: 5
Digite uma letra:
c
Letra correta!
Palavra: BOTIC--IO
Tentativas: 5
Digite uma letra:
r
Letra correta!
Palavra: BOTIC-RIO
Tentativas: 5
Digite uma letra:
a
Letra correta!
Parabéns, restavam-lhe apenas 5 tentativas!
A palavra era: BOTICARIO
```


Projeto: Verificador de senha

Mk1

```
// Mk1
System.out.println(x:"Digite a senha para verificação!");
String senha = scanner.nextLine();

verificarSenha(senha);
scanner.close();
}

public static void verificarSenha(String senha){
    // Fraca <=5 sem numeros
    // Media <=8 com numeros
    // Forte > 8 com numeros
    if(senha.length() >= 8){
        System.out.println(x:"Sua senha é forte!");
    } else if (senha.length() < 5){
        System.out.println(x:"Sua senha é fraca!");
    } else {
        System.out.println(x:"Sua senha é mediana!");
    }
}
}
```

```
Digite a senha para verificação!
qwASZXCW22Ww
Sua senha é forte!
```

Mk2 (Versão do professor)

```
// Mk2 (versão do professor)
System.out.println(x:"Digite a senha: ");
String senha = scanner.next();
int forca = verificarForcaSenha(senha);

if(forca <= 2){
    System.out.println(x:"A sua senha é FRACA!");
} else if (forca == 3){
    System.out.println(x:"A sua senha é MEDIA!");
} else if (forca == 4){
    System.out.println(x:"A sua senha é SEGURA!");
} else {
    System.out.println(x:"A sua senha é MUITO FORTE!");
}
scanner.close();
```

```
public static int verificarForcaSenha(String senha){
    int forca = 0;

    if(senha.length() > 8){
        forca++;
    }
    // ELE USOU EXPRESSÕES REGULARES!
    // Letras maiúsculas de A a Z
    if(senha.matches(regex:".*[A-Z].*")){
        forca++;
    }
    // Letras minúsculas de A a Z
    if(senha.matches(regex:".*[a-z].*")){
        forca++;
    }
    // Numeros!
    if(senha.matches(regex:".*\\d.*")){
        forca++;
    }
    // Caracteres especiais (temos de especificar) os \\ FAZEM ESCAPE (PESQUISE)
    if(senha.matches(regex:(".*[!@#$%&*()\\-_.<>].*"))){
        forca++;
    }
    return forca;
}
```

Aqui temos o uso de **expressões regulares** que veremos mais a frente!

Digite a senha:
ScrewtheGov@@@
A sua senha é SEGURA!

Digite a senha:
teste123
A sua senha é FRACA!

Digite a senha:
SetTheWorldAFIRE<!>
A sua senha é SEGURA!

Digite a senha:
Viol3nceNeverEnd1ng@
A sua senha é MUITO FORTE!

O que é Orientação a Objetos?

- Orientação a Objetos (OOP) é um **paradigma de programação** que organiza o software em torno de objetos;
- Esses objetos **representam entidades do mundo real** e interagem entre si para resolver problemas;
- Cada objeto é uma **instância de uma classe**, que define os atributos e comportamentos do objeto;
- OOP facilita o **design modular** do software, tornando-o mais fácil de manter, reutilizar e expandir;
- Os principais pilares da OOP são **abstração, encapsulamento, herança e polimorfismo**;
- Benefícios da Orientação a Objetos
 - **Modularidade**: O código é organizado em classes e objetos, tornando-o mais fácil de entender e manter;
 - **Reuso de Código**: A herança e a modularidade facilitam a reutilização de partes do código em diferentes aplicações;
 - **Facilidade de Manutenção**: Alterações podem ser feitas em uma classe sem afetar outras partes do sistema, promovendo maior flexibilidade;
 - **Segurança**: O encapsulamento protege os dados e controla o acesso aos atributos e métodos de uma classe;

O que é orientação a objetos (poo ou oop)

A forma como temos trabalho até então se enquadra mais no conceito de código legado que existem nas empresas, é certo que projetos novos praticamente serão baseados em OOP

Terminologia de OO

- **Classe**
 - Um "molde" ou "blueprint" para criar objetos;
 - Define os atributos (propriedades) e métodos (comportamentos) de um objeto;
- **Objeto**
 - Instância de uma classe;
 - Exemplo: Um carro é um objeto da classe "Carro", com atributos como cor, modelo e métodos como acelerar, frear;
- **Atributo (ou Propriedade)**
 - São variáveis dentro da classe que armazenam os dados do objeto;
 - Exemplo: No objeto "Carro", atributos podem ser cor, modelo, ano;
- **Método**
 - Função definida dentro de uma classe que representa uma ação ou comportamento do objeto;
 - Exemplo: No objeto "Carro", métodos podem ser acelerar(), frear();
- **Instanciação**
 - Processo de criar um novo objeto a partir de uma classe;
 - Utiliza-se a palavra-chave **new** em Java para instanciar um objeto;

Terminologia de OO

Criando uma classe

- Uma classe é um modelo que define as propriedades e comportamentos de um objeto;
- Classes são criadas usando a palavra-chave `class`;
- O nome da classe deve ser um substantivo e começar com letra maiúscula;
- Dentro de uma classe, você pode definir atributos (propriedades) e métodos (comportamentos);
- Uma classe não faz nada por si só até que seja instanciada (criação de um objeto);

Criando uma classe

```
// O Java considera a pasta onde os arquivos são criados como um 'pacote'
// isso facilita a reutilização de classes e métodos dentro do mesmo pacote
// já que o Java organiza os arquivos baseados em sua localização no sistema
package secao17;

// Essa é a estrutura básica de uma classe, porém para ter utilidade ela precisa
// de atributos e propriedades
public class Carro {

    // Atributos ou Propriedades
    String marca;
    String modelo;
    int ano;

    // Métodos (funcionam de forma semelhante as funções, temos de definir o retorno, void
    para não gerar retorno)
    // As funções seguem de exemplo: PUBLIC STATIC VOID NOME (args)
    // os métodos são: VOID NOME (args)
    void acelerar(){
        System.out.println("O " + modelo + " está acelerando!");
    }

    // Acessando os atributos da classe
    void exibirInfo(){
        System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Ano: " + ano);
    }
}
```

Instanciando objetos

- Objetos são instâncias de uma classe;
- Para instanciar um objeto, usamos a palavra-chave **new**;
- A instância cria uma cópia do modelo da classe em memória;
- Cada objeto tem seus próprios valores de atributos, independentes de outros objetos;
- A instância de um objeto permite acessar métodos e atributos definidos na classe;
- A sintaxe básica para criar um objeto é:
 - `NomeDaClasse nomeDoObjeto = new NomeDaClasse();`
- Ao instanciar um objeto, podemos acessar seus métodos e definir valores para seus atributos;

Instanciando objetos

```
public class P00 {
    // Como vamos executar esse arquivo, precisamos do método MAIN
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        // 1 - Criar classe
        // Criação de Carro.java - FEITO

        // 2 - Instanciar a classe
        // NomeClasse nomeObjeto = new nomeClasse()
        Carro fusca = new Carro();

        // Atribuição de propriedades, não definimos que as propriedades são obrigatórias
        // neste caso definimos ela por vontade própria!
        // NOMEOBJETO.PROP e NOMEOBJETO.METODO()
        fusca.marca = "VW";
        fusca.modelo = "Fusca";
        fusca.ano = 1964;

        // Chamando os métodos
        fusca.acelerar();
        fusca.exibirInfo();

        Carro carro2 = new Carro();
        carro2.marca = "FIAT";
        // Os valores de propriedades podem ser iguais para diferentes objetos!
        carro2.modelo = "Fusca";
        carro2.acelerar();
        // Se não passarmos as propriedades, serão impressos os valores iniciais
        // de cada tipo de dado, sendo 0 para tipos numéricos e null para tipos de String
        carro2.exibirInfo();
    }
}
```

```
O Fusca está acelerando!
Marca: VW, Modelo: Fusca, Ano: 1964
O Fusca está acelerando!
Marca: FIAT, Modelo: Fusca, Ano: 0
```


Criando métodos

- Métodos são funções dentro de uma classe que definem o comportamento de um objeto;
- Eles permitem que objetos realizem ações ou operações;
- A sintaxe de um método inclui o tipo de retorno, o nome do método, e os parâmetros;
- Métodos podem ou não retornar um valor, dependendo da definição;
- Para criar um método, usamos a seguinte estrutura:

```
public TipoDeRetorno nomeDoMetodo(TipoParametro parametro) {  
    // Corpo do método  
}
```

- Se o método não retornar valor, usamos **void**;
- Os métodos são chamados a partir de uma instância de um objeto da classe;
- Os métodos podem receber parâmetros para trabalhar com dados e retornar resultados;

Criando métodos

classe

```
// 3 - Métodos  
double velocidadeAtual = 0;  
boolean motorLigado = false;  
  
void ligarMotor(){  
    if(!motorLigado){  
        motorLigado = true;  
        System.out.println(x:"Ligando motor.....");  
    } else {  
        System.out.println(x:"O motor já está ligado");  
    }  
}  
  
void aumentarVelocidade(double incremento){  
    if(motorLigado){  
        velocidadeAtual += incremento;  
        System.out.println("A velocidade atual é: " + velocidadeAtual);  
    } else {  
        System.out.println(x:"Primeiro precisa ligar o motor");  
    }  
}
```


objeto

```
// 3 - Métodos
// Dessa forma alteramos a propriedade do objeto, sendo eles booleans, ints, etc..

// Primeiro precisamos ligar o motor para acelerar
fusca.aumentarVelocidade(incremento:10.0);
// Tornamos motor ligado 'true'
fusca.ligarMotor();
System.out.println(fusca.motorLigado); // true
// Agora podemos acelerar
fusca.aumentarVelocidade(incremento:20);
fusca.aumentarVelocidade(incremento:50);
fusca.aumentarVelocidade(incremento:30);
// A velocidade é incrementada
}
```

```
Primeiro precisa ligar o motor
Ligando motor.....
true
A velocidade atual é: 20.0
A velocidade atual é: 70.0
A velocidade atual é: 100.0
```

O que é Encapsulation?

- Encapsulamento (encapsulation) é um dos pilares da Programação Orientada a Objetos (OOP);
- Ele consiste em esconder os detalhes internos de uma classe e expor apenas o necessário para o usuário da classe;
- A ideia principal é **proteger os dados de acessos indevidos** ou modificações diretas;
- Encapsulamento é obtido através do uso de modificadores de acesso, como **private**, **protected**, e **public**;
- Propriedades privadas só podem ser acessadas ou modificadas por métodos específicos da classe (**getters e setters**);
- Getters e setters fornecem uma maneira controlada de **acessar e modificar os atributos da classe**;
- Encapsulamento promove a integridade dos dados e facilita a manutenção do código;
- O objetivo é garantir que os atributos da classe só sejam alterados de maneira controlada e vál'

O que é encapsulation (Encapsulamento)

A forma como temos trabalhado na questão de definição de atributos é errônea, pois nada nos impede de escrever dados incorretos tais como esse:

```
carro2.marca = "FIAT";  
carro2.modelo = "IX35";
```

A Fiat não produz o IX35, a encapsulação serve pra prevenir que erros de atribuição como esse e outros (como por exemplo, definir o ano de fabricação de um fusca em 2040 kkkk) aconteçam

Os getters e setters usam lógica para controlar a modificação e validação dos dados dos atributos de um objeto, dessa forma o usuário que for criar objetos não vai sair simplesmente criando objetos com propriedades absurdas e ilógicas.

Veremos getters e setters mais adiante....

Criando propriedades

- Propriedades são atributos de uma classe que definem as características de um objeto;
- Cada propriedade tem um tipo de dado, como int, double, String, entre outros;
- As propriedades podem ser públicas, privadas ou protegidas, dependendo da visibilidade desejada;
- Propriedades geralmente são declaradas no início da classe;
- Propriedades públicas podem ser acessadas diretamente, enquanto propriedades privadas exigem métodos getters e setters;
- O valor das propriedades pode ser alterado diretamente (se público) ou por meio de métodos (se privado);
- Boas práticas de encapsulamento sugerem o uso de propriedades privadas com métodos públicos para acessar e modificar os valores;

Criando propriedades

Apesar de ser uma convenção, as vezes as propriedades **não** estão declaradas no início da classe

Para entender encapsulação temos de entender a visibilidade das propriedades de um objeto

Aqui veremos um breve exemplo de getters e setters (que veremos depois com mais detalhes) mas em suma **getters** 'pegam' dados, **setters** 'alteram dados'

Veremos também **this** (que também veremos numa aula mais a frente com mais detalhes) mas resumidamente, ela é usada para se referir ao próprio objeto

```
public class Pessoa {  
  
    // Declaramos a visibilidade como 'private' para que o  
    // método da própria classe modifique os dados!  
    private String nome;  
    private int idade;  
  
    // Setter para definir o nome do objeto  
    public void setNome(String nome){  
        //this.nomeDaClasse = nomeDoArgumento  
        // THIS -> OBJETO  
        this.nome = nome;  
    }  
  
    // Getter para retornar o nome do objeto (que por sua vez é privado)  
    // Note que ele retorna uma string sendo o nome do objeto 'Pessoa'  
    public String getNome(){  
        return nome;  
    }  
  
    public void setIdade(int idade){  
        this.idade = idade;  
    }  
  
    public int getIdade(){  
        return idade;  
    }  
}
```

```
// 4 - Criando propriedades
Pessoa alguem = new Pessoa();

// Não conseguimos declarar a propriedade de 'alguem' pois o atributo nome é PRIVADO
// (PROTEGIDO)
// ERRO -> alguem.nome = "João";
// Essa atribuição deve ser feito via método, isso é, deve haver um método nessa classe
// para definir o nome do objeto 'pessoa', os famosos getters e setters

// Usando setter para definir o nome do objeto 'Pessoa'
alguem.setNome(nome:"João");

// Usando getter para exibir o nome do objeto 'Pessoa'
System.out.println("O nome de alguem é: " + alguem.getNome());

// Mesma lógica pra idade!
alguem.setIdade(idade:24);
System.out.println("Essa pessoa tem: " + alguem.getIdade() + " anos!");
}
```

```
O nome de alguem é: João
Essa pessoa tem: 24 anos!
```

O this em objetos

- O this é uma palavra-chave em Java usada dentro de métodos de uma classe para se referir ao objeto atual;
- Ele é útil para distinguir entre os atributos do objeto e os parâmetros do método com o mesmo nome;
- O this é frequentemente usado em setters e construtores para referenciar os atributos do objeto;
- Sempre que um método de uma classe é chamado, o this se refere à instância do objeto que fez a chamada;
- O this também pode ser usado para encadear métodos (method chaining) ou chamar outros construtores;

O this em objetos

Veremos construtores mais a frente!

```
// Setter para definir o nome do objeto
public void setNome(String nome){
    //this.nomeDaClasse = nomeDoArgumento
    // THIS -> OBJETO
    this.nome = nome;
}
```

```
public void setIdade(int idade){
    this.idade = idade;
}
```

Em suma, se por acaso uma classe possui uma propriedade cor, e o argumento do setter desta propriedade também for cor, podemos usar this.cor (se referindo ao nome da propriedade da classe) = cor (se referindo ao argumento que o setter recebe) dessa forma apesar de ambos compartilharem do mesmo nome, não haverá problemas nas definições!

O que são Setters?

- Setters são métodos usados para alterar os valores das propriedades privadas de uma classe;
- Eles permitem o controle sobre como os atributos de um objeto são modificados;
- Setters seguem a convenção de nomeação: setNomePropriedade();
- Setters garantem que os valores dos atributos sejam válidos antes de serem atribuídos;
- Com setters, é possível adicionar lógica de validação ou manipulação dos dados antes de atualizar o valor;

O que são setters

Exemplos de validação

```
public class ContaBancaria {  
  
    private String titular;  
    private double saldo;  
  
    public void setTitular(String titular){  
        // Lógica para validar ou manipular  
        // Se titular é diferente de 'null' e NÃO está vazio...  
        if(titular != null && !titular.isEmpty()){  
            this.titular = titular;  
        } else {  
            System.out.println(x:"Nome do titular inválido!");  
        }  
    }  
  
    public void setSaldo(double saldo){  
        // O saldo não pode ser negativo!  
        if(saldo >= 0){  
            this.saldo = saldo;  
        } else {  
            System.out.println(x:"O valor precisa ser positivo!");  
        }  
    }  
  
    public void mostrarDados(){  
        System.out.println("Titular: " + titular + " / Saldo: " + saldo);  
    }  
}
```

```
// 5 - Setters

ContaBancaria ContaDaAna = new ContaBancaria();
ContaDaAna.setTitular(titular:null); // Nome do titular inválido!
ContaDaAna.setTitular(titular:""); // Nome do titular inválido!
ContaDaAna.setTitular(titular:"Ana"); // Execução correta

ContaDaAna.setSaldo(-11); // O valor precisa ser positivo!
ContaDaAna.setSaldo(saldo:40); // Execução correta

ContaDaAna.mostrarDados(); // Titular: Ana / Saldo: 40.0
```

O que são Getters?

- Getters são métodos usados para acessar os valores das propriedades privadas de uma classe;
- Eles permitem que outras classes ou métodos leiam os atributos, mas sem permitir modificá-los diretamente;
- Getters seguem a convenção de nomeação: getNomePropriedade();
- Um getter retorna o valor da propriedade correspondente e não deve alterar o estado do objeto;
- É uma prática comum usar getters para manter o princípio do encapsulamento;
- Getters ajudam a garantir que os atributos de um objeto sejam acessados de forma controlada;

O que são getters

Geram retornos

```
public String getTitular(){
    return titular;
}

// Manipulação do getter (retorno)
public String getSaldo(){
    return "R$ " + saldo;
}
```

```
System.out.println(ContaDaAna.getTitular()); // Ana
System.out.println(ContaDaAna.getSaldo()); // R$ 40.0
```


Lógica no Getter e Setter

- Getters e Setters podem conter lógica para validar, processar ou modificar os dados antes de serem lidos ou gravados;
- A lógica nos Setters permite validar os dados antes de armazená-los, garantindo integridade;
- A lógica nos Getters pode processar ou formatar os dados antes de retorná-los;
- Isso melhora a segurança e o controle sobre como os atributos são manipulados;
- Alterar dados com lógica nos Getters e Setters mantém o princípio do encapsulamento e previne valores inválidos;

Lógica no getter e setter

(Sim, de novo)

```
public String getProdutoInfo(){
    // Note que usamos os métodos getters e setters que fizemos
    return "Nome: " + this.getNome() + " , Preço: " + this.getPreco();
}

public void aplicarDesconto(double porcentagem){
    if(porcentagem > 0 && porcentagem <= 100){
        // Usamos o método abaixo para saber quantos reais equivalem o desconto dado.
        // e subtraímos esse valor do preço do objeto!
        double desconto = calcularDesconto(porcentagem);
        double precoFinal = this.preco - desconto;
        this.setPreco(precoFinal);
        System.out.println("Desconto de " + porcentagem + "% aplicado!");
        System.out.println(this.getProdutoInfo());
    } else {
        System.out.println(x:"Porcentagem Inválida!");
    }
}

// Método privado para somente ser usado aqui na classe!
private double calcularDesconto(double porcentagem){
    // O preço do objeto * porcentagem / por 100
    // Então se o produto custa 50 reais com desconto de 15% ele perde 7,5 reais de valor!
    return (this.preco * porcentagem) / 100;
}
```

getPreco tem um formatação interessante para valores contábeis!

```
// 6 - Lógica em Getters e Setters

Produto Camisa = new Produto();
Camisa.setNome(nome:"a"); // O nome enviado não atende aos critérios!
Camisa.setNome(nome:"Camisa Polo"); // Execução correta
System.out.println(Camisa.getNome()); // CAMISA POLO
Camisa.setPreco(-1); // O produto não pode ter um preço negativo!
Camisa.setPreco(preco:20.9999); // Execução correta
System.out.println(Camisa.getPreco()); // R$21,00
```

A lógica pode ser definida tanto para pegar ou expor informações dos objetos, a validação depende do sistema e da regra de negócios da empresa!

Método dentro de método

- Em Java, métodos podem chamar outros métodos dentro da mesma classe;
- Essa prática ajuda a modularizar o código, quebrando funcionalidades complexas em partes menores;
- Chamar um método dentro de outro promove a reutilização de código e facilita a manutenção;
- Ao combinar métodos, é possível realizar ações mais complexas e evitar duplicação de código;
- Métodos dentro de métodos ajudam a simplificar a lógica, tornando o código mais claro e legível;

Método dentro de método

Contêm maneiras interessantes de calcular porcentagens em descontos!

```
public String getProdutoInfo(){
    // Note que usamos os métodos getters e setters que fizemos
    return "Nome: " + this.getNome() + " , Preço: " + this.getPreco();
}

public void aplicarDesconto(double porcentagem){
    if(porcentagem > 0 && porcentagem <= 100){
        // Usamos o método abaixo para saber quantos reais equivalem o desconto dado.
        // e subtraímos esse valor do preço do objeto!
        double desconto = calcularDesconto(porcentagem);
        this.preco -= desconto;
        System.out.println("Desconto de " + porcentagem + "% aplicado!");
        System.out.println(this.getProdutoInfo());
    } else {
        System.out.println(x:"Porcentagem Inválida!");
    }
}

// Método privado para somente ser usado aqui na classe!
private double calcularDesconto(double porcentagem){
    // O preço do objeto * porcentagem / por 100
    // Então se o produto custa 50 reais com desconto de 15% ele perde 7,5 reais de
    // valor!
    return (this.preco * porcentagem) / 100;
}
```

```
// 7 - Método dentro de método
System.out.println(Camisa.getProdutoInfo()); // Nome: CAMISA POLO , Preço: R$21,00
Camisa.aplicarDesconto(porcentagem:10);
// Desconto de 10.0% aplicado!
// Nome: CAMISA POLO , Preço: R$18,90
```

O que são constructors?

- Construtores (constructors) são métodos especiais usados para inicializar objetos;
- Eles são chamados automaticamente quando um novo objeto é criado;
- O nome de um construtor deve ser o mesmo que o nome da classe;
- Construtores não têm um tipo de retorno (nem mesmo void);
- Podem ser usados para atribuir valores iniciais aos atributos de um objeto;
- Java fornece um construtor padrão se nenhum for definido, mas construtores personalizados permitem maior controle na criação do objeto;

O que são constructors? (Construtores)

```
public class Livro {
    // Imagine ter que declarar os getters e setters para todas essas propriedades...
    // Agora imagine um objeto com 10 ou mais propriedades....
    private String titulo;
    private String autor;
    private double preco;

    // Todos os objetos começarão com estes valores básicos, até que eles sejam definidos!
    public Livro(){
        this.titulo = "Título teste";
        this.autor = "Autor";
        this.preco = 12.99;
    }

    // Ou podemos fazer assim!
    public Livro(String titulo, String autor, double preco){
        this.titulo = titulo;
        this.autor = autor;
        this.preco = preco;
    }

    // Este código utiliza a sobrecarga de construtores para permitir diferentes formas de
    // inicializar um objeto.
    // O primeiro construtor define valores padrão para os atributos, enquanto o segundo
    // permite que
    // os valores sejam personalizados pelo desenvolvedor ao passar argumentos específicos.

    public void exibirInfo(){
        System.out.println("Título: " + titulo + ", Autor: " + autor + ", Preço: " +
            preco);
    }
}
```

```
// 8 - Construtores
Livro meuLivro = new Livro(titulo:"A busca onírica por Kadath", autor:"H P Lovecraft",
    preco:19.99);
Livro meuLivro2 = new Livro();
// SEM O CONSTRUTOR
meuLivro2.exibirInfo(); // Título: Título teste, Autor: Autor, Preço: 12.99
// COM O CONSTRUTOR
meuLivro.exibirInfo(); // Título: A busca onírica por Kadath, Autor: H P Lovecraft,
    Preço: 19.99
```

Exercícios

Criando a classe Celular

Mk1 (feito por mim)

```
public class Celular {
    private String marca;
    private String modelo;
    private int cargaBateria = 100;

    boolean ligado = false;

    public void setMarca(String marca){
        if(marca!= null && !marca.isEmpty()){
            this.marca = marca;
        } else {
            System.out.println(x:"Marca do celular invalida!");
        }
    }

    public void setModelo(String modelo){
        if(modelo!= null && !modelo.isEmpty()){
            this.modelo = modelo;
        } else {
            System.out.println(x:"Modelo do celular invalido!");
        }
    }

    public String mostrarInfo(){
        return "Marca: " + marca + ", Modelo: " + modelo + ", Bateria: " + cargaBateria;
    }

    void ligarCelular(){
        if(!ligado && cargaBateria > 0){
            ligado = true;
            System.out.println(x:"Ligando o celular.....");
        } else if (ligado){
            System.out.println(x:"O celular já está ligado");
        } else {
            System.out.println(x:"Não é possível ligar o celular. Bateria insuficiente!");
        }
    }
}
```

```

void desligarCelular(){
    if(!ligado){
        System.out.println(x:"O celular já está desligado");
    } else {
        ligado = false;
        System.out.println(x:"Desligando o celular");
    }
}

void usarCelular(){
    if(ligado && cargaBateria > 0){
        System.out.println(x:"Usando o celular!");
        cargaBateria -= 10;
        if(cargaBateria <= 0){
            cargaBateria = 0;
            System.out.println(x:"Bateria zerada! O celular será desligado");
            desligarCelular();
        }
    } else if (!ligado){
        System.out.println(x:"Celular está desligado!");
    } else {
        System.out.println(x:"Celular sem bateria!");
    }
}
}

```

```

// Exercício 01
Celular Semsunga = new Celular();
Semsunga.setMarca(marca:"Samsung");
Semsunga.setModelo(modelo:"Pocket 17");
System.out.println(Semsunga.mostrarInfo()); // Marca: Samsung, Modelo: Pocket 17,
Bateria: 100.0
Semsunga.desligarCelular(); // O celular já está desligado
Semsunga.ligarCelular(); // Ligando o celular....

Semsunga.usarCelular(); // Usando o celular!
System.out.println(Semsunga.mostrarInfo()); // Marca: Samsung, Modelo: Pocket 17,
Bateria: 90.0
Semsunga.ligarCelular(); // O celular já está ligado
Semsunga.desligarCelular(); // Desligando o celular
Semsunga.usarCelular(); // Celular está desligado!

```

Mk2 (feito pelo professor)

```
public class Celular {
    String marca;
    String modelo;
    int bateria = 100;

    void ligar(){
        System.out.println("O celular " + modelo + " está ligado!");
    }

    void desligar(){
        System.out.println("O celular " + modelo + " está desligado!");
    }

    void usar(int consumo){
        if(bateria - consumo >= 0){
            bateria -= consumo;
            System.out.println("O celular foi usado, bateria em: " + bateria + "%");
        } else {
            System.out.println(x:"Bateria insuficiente");
        }
    }
}
```

```
// Exercício 01
Celular meuCelular = new Celular();
meuCelular.marca = "Samsung";
meuCelular.modelo = "Pocket";

meuCelular.ligar(); // O celular Pocket está ligado!
meuCelular.desligar(); // O celular Pocket está desligado!
meuCelular.usar(consumo:50); // O celular foi usado, bateria em: 50%
meuCelular.usar(consumo:50); // O celular foi usado, bateria em: 0%
meuCelular.usar(consumo:50); // Bateria insuficiente
```


Classe aluno com encapsulamento e construtores

Mk1 (feito por mim)

```
public class Aluno {
    private String nome;
    private String matricula;
    private double notaFinal;

    public Aluno(String nome, String matricula, double notaFinal){
        this.nome = nome;
        this.matricula = matricula;
        this.notaFinal = notaFinal;
    }

    public void setAluno(String nome){
        if(nome != null && !nome.isEmpty()){
            this.nome = nome;
        } else {
            System.out.println(x:"Nome do aluno inválido!");
        }
    }

    public void setMatricula(String matricula){
        if(matricula != null && !matricula.isEmpty()){
            this.matricula = matricula;
        }
    }

    public void setNotaFinal(double notaFinal){
        if(notaFinal >= 0 && notaFinal <= 100){
            this.notaFinal = notaFinal;
        } else {
            System.out.println(x:"A nota final deve estar entre 0 e 100!");
        }
    }

    public void getInfo(){
        System.out.println("Aluno: " + nome + ", Matrícula: " + matricula + ", Nota Final: "
            + notaFinal);
    }
}
```

```
// Exercício 02
Aluno aluno = new Aluno(nome:"Fulano", matricula:"88ACL4", notaFinal:80);

Aluno aluno2 = new Aluno(nome:"Beltrano", matricula:"65AOE", notaFinal:60);

aluno.getInfo(); // Aluno: Fulano, Matrícula: 88ACL4, Nota Final: 80.0
aluno2.getInfo(); // Aluno: Beltrano, Matrícula: 65AOE, Nota Final: 60.0
aluno.setNotaFinal(notaFinal:100);
aluno.setAluno(nome:""); // Nome do aluno inválido!
aluno.getInfo(); // Aluno: Fulano, Matrícula: 88ACL4, Nota Final: 100.0
```

Mk2 (Feito pelo professor)

```
public class Aluno {
    private String nome;
    private int matricula;
    private double notaFinal;

    public Aluno(String nome, int matricula, double notaFinal){
        this.nome = nome;
        this.matricula = matricula;
        this.notaFinal = notaFinal;
    }

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        if(nome != null && !nome.isEmpty()){
            this.nome = nome;
        } else {
            System.out.println(x:"Nome inválido!");
        }
    }

    public double getNotaFinal(){
        return notaFinal;
    }

    public void setNotafinal(double notaFinal){
        if(notaFinal >= 0 && notaFinal <= 100){
            this.notaFinal = notaFinal;
        } else {
            System.out.println(x:"Nota inválida!");
        }
    }

    public void exibirInfo(){
        System.out.println("Aluno " + nome + ", Matrícula: " + matricula + ", Nota Final: "
            + notaFinal);
    }
}
```

```
// Exercício 02
Aluno aluno = new Aluno(nome:"Fulano", matricula:88775, notaFinal:80);
Aluno aluno2 = new Aluno(nome:"Beltrano", matricula:87456, notaFinal:100);

System.out.println(aluno.getNome()); // Fulano
System.out.println(aluno2.getNome()); // Beltrano

System.out.println(aluno.getNotaFinal()); // 80.0
System.out.println(aluno2.getNotaFinal()); // 100.0

aluno.setNotafinal(notaFinal:100);

aluno.exibirInfo(); // Aluno Fulano, Matrícula: 88775, Nota Final: 100.0
aluno2.exibirInfo(); // Aluno Beltrano, Matrícula: 87456, Nota Final: 100.0
```

Classe ContaCorrente com métodos e encapsulamento

Mk1 (feito por mim)

```
public class ContaCorrente {
    private String titular;
    private double saldo;
    private double limiteSaque;

    public ContaCorrente(String titular, double saldo, double limiteSaque){
        this.titular = titular;
        this.saldo = saldo;
        this.limiteSaque = limiteSaque;
    }

    public void exibirSaldo(){
        System.out.println("Titular: " + titular + ", Saldo: " + saldo);
    }

    public void depositar(double valor){
        if(valor > 0){
            saldo += valor;
            System.out.println("Valor depositado com sucesso, Saldo atual: " + saldo);
        } else {
            System.out.println(x:"Valor do depósito inválido!");
        }
    }

    public void sacar(double valor){
        if(valor <= limiteSaque && valor <= saldo){
            saldo -= valor;
            System.out.println("Valor sacado com sucesso, Saldo atual: " + saldo);
        } else if (valor > limiteSaque) {
            System.out.println(x:"O valor inserido supera o limite de saque!");
        } else {
            System.out.println(x:"Você não possui saldo o suficiente!");
        }
    }
}
```

```
// Exercício 03
ContaCorrente conta = new ContaCorrente(titular:"Ana", saldo:100, limiteSaque:200);
conta.exibirSaldo(); // Titular: Ana, Saldo: 100.0
conta.depositar(valor:140); // Valor depositado com sucesso, Saldo atual: 240.0
conta.depositar(valor:0); // Valor do depósito inválido
conta.exibirSaldo(); // Titular: Ana, Saldo: 240.0

conta.sacar(valor:500); // O valor inserido supera o limite de saque!
conta.sacar(valor:100); // Valor sacado com sucesso, Saldo atual: 140.0
conta.exibirSaldo(); // Titular: Ana, Saldo: 140.0
conta.sacar(valor:200); // Você não possui saldo o suficiente!
```

Classe ProdutoEletronico com método dentro de método

Mk1 (Feito por mim)

```
public class ProdutoEletronico {
    private String nome;
    private double preco;
    private boolean garantia;

    public ProdutoEletronico(String nome, double preco, boolean garantia){
        this.nome = nome;
        this.preco = preco;
        this.garantia = garantia;
    }

    public String getNome(){
        return nome;
    }

    public double getPreco(){
        return preco;
    }

    public boolean getGarantia(){
        return garantia;
    }

    public void setNome(String nome){
        if(nome != null && !nome.isEmpty()){
            this.nome = nome;
        } else {
            System.out.println(x:"Nome do produto inválido");
        }
    }

    public void setPreco(double preco){
        if(preco > 0){
            this.preco = preco;
        } else {
            System.out.println(x:"O produto não pode ter um valor negativo!");
        }
    }

    public void mostrarInfo(){
        System.out.println("Produto: " + this.getNome() + ", Preço: " + this.getPreco() + ",
        Garantia: " + this.getGarantia());
    }
}
```

```
    private double calcularDesconto(double porcentagem){
        return (this.preco * porcentagem) / 100;
    }

    public void aplicarDesconto(double porcentagem){
        if(porcentagem > 0 && porcentagem <= 100){
            double desconto = calcularDesconto(porcentagem);
            double precoFinal = this.preco - desconto;
            this.setPreco(precoFinal);
            System.out.println("Desconto de " + porcentagem + "% aplicado!");
            this.mostrarInfo();
        }
    }
}
```

```
// Exercício 04
ProdutoEletronico produto = new ProdutoEletronico(nome:"Escada", preco:100,
garantia:true);
produto.mostrarInfo(); // Produto: Escada, Preço: 100.0, Garantia: true
produto.aplicarDesconto(percentagem:30);
// Desconto de 30.0% aplicado!
// Produto: Escada, Preço: 70.0, Garantia: true
```

Mk2 (Feito pelo professor)

```
public class ProdutoEletronico {
    private String nome;
    private double preco;
    private int garantia;

    public ProdutoEletronico(String nome, double preco, int garantia){
        this.nome = nome;
        this.preco = preco;
        this.garantia = garantia;
    }

    public void aplicarDesconto(double percentagem){
        if(percentagem > 0 && percentagem < 100){
            double valorDesconto = calcularDesconto(percentagem);
            preco -= valorDesconto;
            System.out.println("Desconto aplicado, o preço agora é: " + preco);
        } else {
            System.out.println(x:"Porcentagem incorreta!");
        }
    }

    public double calcularDesconto(double percentagem){
        return (preco * percentagem) / 100;
    }

    public void exibirInfo(){
        System.out.println("Produto: " + nome + ", Preço: " + preco + ", Garantia (em meses)
        : " + garantia);
    }
}
```

```
// Exercício 04
ProdutoEletronico produto = new ProdutoEletronico(nome:"Bateria", preco:200,
garantia:12);
produto.aplicarDesconto(percentagem:40); // Desconto aplicado, o preço agora é: 120.0
produto.exibirInfo(); // Produto: Bateria, Preço: 120.0, Garantia (em meses): 12
```

Classe LivrosBiblioteca com lógica em setters e getters

Mk1 (Feito por mim)

```
public class LivroBiblioteca {
    private String titulo;
    private String autor;
    private boolean disponivel;

    public LivroBiblioteca(String titulo, String autor, boolean disponivel){
        this.titulo = titulo;
        this.autor = autor;
        this.disponivel = disponivel;
    }

    public void pegarEmprestado(){
        if(!disponivel){
            System.out.println(x:"O livro não está disponível!");
        } else {
            this.disponivel = false;
            System.out.println(x:"Livro pego com sucesso!");
        }
    }

    public void devolverLivro(){
        if(disponivel){
            System.out.println(x:"O livro já está disponível!");
        } else {
            this.disponivel = true;
            System.out.println(x:"Livro devolvido com sucesso");
        }
    }

    public void getInfo(){
        System.out.println("Título: " + titulo + ", Autor: " + autor + ", Disponível: " +
            disponivel);
    }
}
```

```
// Exercício 05
LivroBiblioteca livro = new LivroBiblioteca(titulo:"A Cidade Sem Nome",
autor:"Lovecraft", disponivel:true);
livro.getInfo(); // Título: A Cidade Sem Nome, Autor: Lovecraft, Disponível: true
livro.devolverLivro(); // O livro já está disponível!
livro.pegarEmprestado(); // Livro pego com sucesso!
livro.getInfo(); // Título: A Cidade Sem Nome, Autor: Lovecraft, Disponível: false
livro.pegarEmprestado(); // O livro não está disponível!
livro.devolverLivro(); // Livro devolvido com sucesso
livro.getInfo(); // Título: A Cidade Sem Nome, Autor: Lovecraft, Disponível: true
```

Mk2 (Feito pelo professor)

```
public class LivroBiblioteca {  
    private String titulo;  
    private String autor;  
    private boolean disponivel = true;  
  
    public String getTitulo(){  
        return titulo;  
    }  
  
    public void setTitulo(String titulo){  
        if(titulo != null && !titulo.isEmpty()){  
            this.titulo = titulo;  
        } else {  
            System.out.println(x:"Título inválido!");  
        }  
    }  
  
    public boolean verificarDisponibilidade(){  
        return disponivel;  
    }  
  
    public void pegarEmprestado(){  
        if(disponivel){  
            disponivel = false;  
            System.out.println("Livro: " + titulo + ", pego com sucesso!");  
        } else {  
            System.out.println(x:"Livro não disponível");  
        }  
    }  
  
    public void devolver(){  
        if(!disponivel){  
            disponivel = true;  
            System.out.println("Livro: " + titulo + ", devolvido com sucesso!");  
        } else {  
            System.out.println(x:"Livro já está disponível");  
        }  
    }  
  
    public void setAutor(String autor){  
        this.autor = autor;  
    }  
  
    public String getAutor(){  
        return autor;  
    }  
}
```



```
// Exercício 05
LivroBiblioteca livro = new LivroBiblioteca();
livro.setTitulo(titulo:"A busca onírica por Kadath");
livro.devolver(); // Livro já está disponível
livro.pegarEmprestado(); // Livro: A busca onírica por Kadath, pego com sucesso!
livro.pegarEmprestado(); // Livro não disponível
livro.devolver(); // Livro: A busca onírica por Kadath, devolvido com sucesso!
```

• Modificadores de acesso

- Os modificadores de acesso controlam a visibilidade dos membros de uma classe (atributos e métodos);
- **public**: O membro pode ser acessado de qualquer lugar (dentro ou fora do pacote);
- **private**: O membro só pode ser acessado dentro da própria classe;
- **protected**: O membro pode ser acessado dentro da classe, suas subclasses e classes do mesmo pacote;
- O uso correto dos modificadores de acesso é fundamental para aplicar o encapsulamento;
- Eles protegem os dados e métodos, garantindo o controle de como eles são acessados e modificados;

Modificadores de acesso

(Sim, já temos usados eles algumas vezes!)

```
public class Funcionario {

    public String nome;
    protected double salario;
    private String senha;

    // O constructor pode fazer as atribuições independentemente da visibilidade
    public Funcionario(String nome, double salario, String senha) {
        this.nome = nome;
        this.salario = salario;
        this.senha = senha;
    }

    public void exibirDados(){
        System.out.println("Nome: " + nome + ", Salário: " + salario + ", Senha: " + senha);
    }

    // Métodos protegidos funcionam como as propriedades protegidas
    protected void aumentarSalario(double porcentagem){
        this.salario += ((this.salario * porcentagem) / 100);
        System.out.println("O salário agora é de: " + salario);
    }

    private boolean verificarSenha(String tentativaSenha){
        // Vai verificar se a senha do objeto é igual a tentativaSenha
        return this.senha.equals(tentativaSenha);
    }

    public boolean autenticar(String tentativaSenha){
        // LÓGICA E BLABLABLABLA
        return verificarSenha(tentativaSenha);
    }

}
```

```

public class P002 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        // 1 - Níveis de Acesso
        Funcionario funcionario = new Funcionario(nome:"Fulano", salario:2000,
        senha:"teste123");

        funcionario.exibirDados(); // Nome: Fulano, Salário: 2000.0, Senha: teste123

        // Podemos alterar o nome porque ele é um atributo publico!
        funcionario.nome = "Creuso";
        // Podemos alterar o salário pois estamos usando sua classe e compartilhando o pacote
        funcionario.salario = 3000;
        // Não podemos alterar a senha pois esse atributo é privado e só pode ser manipulado
        através de getters e setters
        // APENAS ESSE RETORNA ERRO -> funcionario.senha = "teste999";
        funcionario.exibirDados(); // Nome: Creuso, Salário: 3000.0, Senha: teste123

        // O método abaixo é protegido
        funcionario.aumentarSalario(porcentagem:8); // O salário agora é de: 3240.0

        // O método abaixo é privado e retorna um erro (um vez que só pode ser usado na
        classe em si!)
        // ERRO -> System.out.println(funcionario.verificarSenha());

        // O método abaixo é publico e irá funcionar normalmente, ele usa um outro método
        privado em sua lógica!
        if(funcionario.autenticar(tentativaSenha:"teste123")){
            System.out.println(x:"Funcionário entrou no sistema");
        } else {
            System.out.println(x:"Senha incorreta!");
        }
        // Funcionário entrou no sistema
    }
}

```

O protected tem seus raros casos de uso, o uso de private e public é presente em quase todos os sistemas e deve ser o foco de aprendizagem!

Classes imutáveis

- Uma classe imutável é aquela cujas instâncias (objetos) não podem ser modificadas depois de criadas;
- Todos os atributos de uma classe imutável são declarados como `private` e `final`;
- Não há `setters`, e qualquer alteração nos atributos requer a criação de um novo objeto;
- Classes imutáveis garantem consistência e segurança no código, evitando mudanças inesperadas no estado do objeto;
- Exemplos de classes imutáveis nativas em Java incluem `String` e classes wrappers como `Integer` e `Double`;
- Classes imutáveis são particularmente úteis em programação multithread, pois eliminam a necessidade de sincronização;

Classes imutáveis são raras, sem alterações internas (Dentro da classe) e externas (Fora da classe)

```
public class PessoaImutavel {  
  
    private final String nome;  
    private final int idade;  
  
    public PessoaImutavel(String nome, int idade) {  
        this.idade = idade;  
        this.nome = nome;  
    }  
  
    public String getNome(){  
        return this.nome;  
    }  
  
    public int getIdade() {  
        return this.idade;  
    }  
}
```

```
// 2 - Classes Imutáveis  
PessoaImutavel joaquim = new PessoaImutavel(nome:"Joaquim", idade:23);  
System.out.println(joaquim.getNome()); // Joaquim  
System.out.println(joaquim.getIdade()); // 23  
  
/*  
Se tentarmos usar joaquim.nome = "Teste"; não funcionaria por causa do atributo  
ser private  
Se tentarmos usar this.nome = "Teste"; na classe 'PessoaImutavel' não funcionaria  
porque o atributo nome é declarado como final, logo não pode ser alterado, da mesma  
forma um setter pro nome e/ou qualquer atributo final não iria funcionar!  
*/
```

Encapsulamento de Arrays

- Encapsulamento é a prática de restringir o acesso direto aos atributos de uma classe, expondo-os apenas através de métodos controlados (getters e setters);
- Quando trabalhamos com arrays ou coleções, o encapsulamento é igualmente importante;
- Arrays e coleções contêm múltiplos dados, e expô-los diretamente pode permitir modificações não controladas, causando inconsistências;
- Para encapsular arrays ou coleções, usamos getters e setters, garantindo que acessos e modificações sejam feitos de forma controlada;
- É importante fornecer cópias ao expor arrays ou coleções, garantindo que o conteúdo original não seja alterado diretamente;

Encapsulamento de arrays

```
public class Turma {  
  
    private String[] alunos;  
  
    public Turma(String[] alunos) {  
        // Aqui criamos uma cópia de 'alunos' com base na quantidade de alunos, dessa forma  
        // não precisamos inserir um valor para definir a quantidade de alunos  
        this.alunos = Arrays.copyOf(alunos, alunos.length);  
    }  
  
    public String[] getAlunos(){  
        // Mais uma vez vamos usar apenas as cópias do array!  
        return Arrays.copyOf(alunos, alunos.length);  
    }  
  
    public void setAlunos(String[] alunos){  
        if(alunos.length > 0){  
            this.alunos = Arrays.copyOf(alunos, alunos.length);  
        }  
    }  
}
```

```
// 3 - Encapsulamento de Arrays  
String[] meusAlunos = {"Fulano", "Sicrano", "Beltrano"};  
  
// Passamos o array acima como critério para a criação do objeto Turma  
Turma novaTurma = new Turma(meusAlunos);  
  
// Impressão dos alunos no objeto 'novaTurma'  
System.out.println(Arrays.toString(novaTurma.getAlunos())); // [Fulano, Sicrano,  
Beltrano]  
  
// Precisamos colocar os atributos em variáveis obrigatoriamente  
String[] outrosAlunos = {"Ana", "Carlos", "João"};  
// Alteramos TODOS os alunos para os valores do array acima!  
novaTurma.setAlunos(outrosAlunos);  
System.out.println(Arrays.toString(novaTurma.getAlunos())); // [Ana, Carlos, João]
```

Projeto: Simulador de Loteria

```
public class Bilhete {

    private int[] numerosEscolhidos;
    private int[] resultadoSorteio;

    public Bilhete(int[] numerosEscolhidos) {
        this.numerosEscolhidos = numerosEscolhidos;
    }

    public void realizarSorteio(){
        Random random = new Random();
        // Atribuímos a variável a um array com 6 posições
        resultadoSorteio = new int[6];
        for(int i = 0; i < resultadoSorteio.length; i++){
            // Atribuímos um número aleatório ao índice i em resultadoSorteio
            resultadoSorteio[i] = random.nextInt(bound:60) + 1;
        }
        Arrays.sort(resultadoSorteio);
    }

    public int contarAcertos(){
        // Aqui eu fiz em loops aninhados na minha versão!
        int acertos = 0;
        for(int numerosEscolhido : numerosEscolhidos){
            for(int numeroSorteado : resultadoSorteio){
                if(numerosEscolhido == numeroSorteado){
                    acertos++;
                }
            }
        }
        return acertos;
    }

    public void exibirResultados(){
        System.out.println("Números escolhidos: " + Arrays.toString(numerosEscolhidos));
        System.out.println("Números sorteados: " + Arrays.toString(resultadoSorteio));

        int acertos = contarAcertos();

        System.out.println("Você teve: " + acertos + " acertos!");

        System.out.println();
    }
}
```

```
public class SimuladorLoteria {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        ArrayList<Bilhete> bilhetes = new ArrayList<>();

        // Solicitando bilhetes do usuário
        while (true) {
            System.out.println(x:"Digite 6 números para o seu bilhete, entre 1 e 60: ");

            int[] numerosEscolhidos = new int[6];

            // Usuário escolhe os números
            for(int i = 0; i < numerosEscolhidos.length; i++){
                System.out.println("Digite o número " + (i + 1) + ": ");
                numerosEscolhidos[i] = scanner.nextInt();
            }

            // Criamos o bilhete com os números escolhidos
            Bilhete bilhete = new Bilhete(numerosEscolhidos);

            // Adicionamos o bilhete a arrayList (o usuário pode ter vários bilhetes)
            bilhetes.add(bilhete);

            // Questiona se o usuário quer continuar
            System.out.println(x:"Deseja registrar outro bilhete? (s/n) ");
            String resposta = scanner.next();

            if(resposta.equals(anObject:"n")){
                break;
            }
        }

        // Realizar o sorteio para cada bilhete
        System.out.println(x:"Realizando o sorteio...");

        for(Bilhete bilhete : bilhetes){
            bilhete.realizarSorteio();
            bilhete.exibirResultados();
        }
        scanner.close();
    }
}
```

```
Digite 6 números para o seu bilhete, entre 1 e 60:
Digite o número 1:
23
Digite o número 2:
45
Digite o número 3:
35
Digite o número 4:
17
Digite o número 5:
5
Digite o número 6:
54
Deseja registrar outro bilhete? (s/n)
s
Digite 6 números para o seu bilhete, entre 1 e 60:
Digite o número 1:
34
Digite o número 2:
21
Digite o número 3:
23
Digite o número 4:
5
Digite o número 5:
6
Digite o número 6:
45
Deseja registrar outro bilhete? (s/n)
n
```

```
Realizando o sorteio...
Números escolhidos: [23, 45, 35, 17, 5, 54]
Números sorteados: [5, 10, 22, 29, 41, 51]
Você teve: 1 acertos!

Números escolhidos: [34, 21, 23, 5, 6, 45]
Números sorteados: [1, 9, 12, 17, 25, 47]
Você teve: 0 acertos!
```


Projeto: Sistema de Cadastro de Alunos

```
public class Aluno {

    private String nome;
    private String matricula;
    private double[] notas;

    public Aluno(String nome, String matricula, int numeroDeNotas) {
        this.nome = nome;
        this.matricula = matricula;
        // Criamos um array com base no número de notas que iremos receber
        this.notas = new double[numeroDeNotas];
    }

    // Passando Scanner como argumento
    public void adicionarNotas(Scanner scanner){
        System.out.println("Digite as notas do aluno: " + nome + ": ");
        for(int i = 0; i < notas.length; i++){
            System.out.println("Digite a nota " + (i + 1) + ": ");
            notas[i] = scanner.nextDouble();
        }
    }

    public double calcularMedia(){
        double soma = 0;
        for(double nota : notas){
            soma += nota;
        }
        // Retorna a soma das notas dividida pela quantidade de notas
        return soma / notas.length;
    }

    // Critério de sucesso do aluno
    public void exibirResultado(){
        double media = calcularMedia();

        System.out.println("Nome: " + nome);
        System.out.println("Matrícula: " + matricula);
        System.out.println("Média das notas: " + media);
        if(media >= 7.0){
            System.out.println(x:"Aluno APROVADO");
        } else {
            System.out.println(x:"Aluno REPROVADO");
        }
    }
}
```

```
public class SistemaCadastroAluno {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Solicitar o número de alunos a serem cadastrados
        System.out.println(x:"Quantos alunos deseja cadastrar? ");
        int numeroDeAlunos = scanner.nextInt();

        // Criação do array de alunos
        Aluno[] alunos = new Aluno[numeroDeAlunos];

        // Loop de cadastro de alunos
        for(int i = 0; i < numeroDeAlunos; i++){
            System.out.println("Cadastro do aluno " + (i + 1) + ": ");

            System.out.println(x:"Nome: ");
            String nome = scanner.next();

            System.out.println(x:"Matrícula: ");
            String matricula = scanner.next();

            System.out.println(x:"Quantidade de notas: ");
            int numeroDeNotas = scanner.nextInt();
            scanner.nextLine();

            Aluno aluno = new Aluno(nome, matricula, numeroDeNotas);

            // Adicionar as notas
            aluno.adicionarNotas(scanner);

            // Armazenar o aluno no array
            alunos[i] = aluno;
        }
        // Exibindo resultados
        System.out.println(x:"Resultado dos alunos: ");
        for(Aluno aluno : alunos){
            aluno.exibirResultado();
            System.out.println();
        }
        scanner.close();
    }
}
```

```
Resultado dos alunos:
Nome: Carlos
Matrícula: a1
Média das notas: 9.5
Aluno APROVADO

Nome: Fulano
Matrícula: a2
Média das notas: 7.0
Aluno APROVADO

Nome: Beltrano
Matrícula: a3
Média das notas: 7.0
Aluno APROVADO

Nome: Sicrano
Matrícula: a4
Média das notas: 5.0
Aluno REPROVADO

Nome: Zicrano
Matrícula: a6
Média das notas: 7.0
Aluno APROVADO
```

Neste projeto temos a inclusão de Scanner como parâmetro de uma classe!

Object Composition

- Composição de Objetos é um princípio da Programação Orientada a Objetos (POO), onde um objeto é composto por outros objetos;
- Na composição, um objeto maior contém outros objetos menores como atributos, combinando suas funcionalidades para formar comportamentos mais complexos;
- A composição oferece uma alternativa à herança, favorecendo a reutilização de código sem criar dependências rígidas entre classes;

Object Composition (Composição de objetos)

Podemos acessar os métodos de um objeto mesmo se ele estiver dentro de outro objeto, note que acessamos o método `exibirInfo()` de motor, apesar de estarmos construindo um método dentro da classe `Carro`!

```
// 1- Object Composition (Composição de objetos)
Motor motor1 = new Motor(tipo:"V8", potencia:450);

// Note que usamos o motor criado acima para criar o carro abaixo!
Carro carro1 = new Carro(marca:"Ford", modelo:"Mustang", motor1);

carro1.exibirInfo();
// Carro com marca: Ford, Modelo: Mustang <- ORIUNDO DA CLASSE CARRO
// Motor: V8, Potência: 450 cavalos <- ORIUNDO DA CLASSE MOTOR

// Encadeamento de objetos, só é possível porque 'motor' é um atributo publico na
// classe 'Carro'! (não seria possível sua execução se fosse 'private')
carro1.motor.exibirInfo(); // Motor: V8, Potência: 450 cavalos
```

```
public class Motor {
    private String tipo;
    private int potencia;

    public Motor(String tipo, int potencia){
        this.tipo = tipo;
        this.potencia = potencia;
    }

    public void exibirInfo(){
        System.out.println("Motor: " + tipo + ", Potência: " + potencia + " cavalos");
    }
}
```

```

public class Carro {
    private String marca;
    private String modelo;
    // Usando a classe motor, Motor = classe, motor = nome do atributo
    public Motor motor;

    public Carro(String marca, String modelo, Motor motor){
        this.marca = marca;
        this.modelo = modelo;
        // Propriedade de object composition
        this.motor = motor;
    }

    public void exibirInfo() {
        System.out.println("Carro com marca: " + marca + ", Modelo: " + modelo);
        // Vamos usar o método exibirInfo() de motor para exibir os dados dele
        motor.exibirInfo();
    }
}

```

```

Carro com marca: Ford, Modelo: Mustang
Motor: V8, Potência: 450 cavalos
Motor: V8, Potência: 450 cavalos

```

Herança

- Herança é um dos pilares da Programação Orientada a Objetos (POO);
- Permite que uma classe herde atributos e métodos de outra classe;
- Cria uma relação entre as classes, onde a subclasse é um tipo especializado da superclasse;
- Herança promove o reuso de código, facilitando a criação de novas classes sem duplicação de lógica;

Herança

Uma vez que usamos extends para definir uma classe pai para uma subclasse, somos **OBRIGADOS** a usar todas as propriedades da classe pai, e suas propriedades herdadas tem que conter a palavra **SUPER** (veremos mais tarde mais detalhadamente)

As subclasses tem acessos a tudo da superclasse (classe pai), entretanto a classe pai não tem acesso aos métodos e dados das subclasses!

```
public class Animal {  
    // Usamos protect porque poderemos alterar esse valor através das subclasses  
    // (classes geradas usando essa como 'pai')  
    protected String nome;  
  
    public Animal(String nome) {  
        this.nome = nome;  
    }  
  
    public void emitirSom(){  
        System.out.println(nome + " está emitindo som");  
    }  
}
```

```
// Usamos extends para definir qual é a classe pai!  
public class Cachorro extends Animal {  
    // Não precisamos definir suas propriedades, pois suas características  
    // São obrigatoriamente herdadas da classe pai, podemos até inserir novas  
    // características, mas todas as herdadas devem ter 'super' no construtor!  
    public Cachorro(String nome){  
        // Referência ao nome da classe Animal!  
        super(nome);  
    }  
  
    public void latir(){  
        System.out.println(nome + " está latindo!");  
    }  
}
```

```
// 2 - Herança
Cachorro bethooven = new Cachorro(nome:"Bethooven");

// Método da classe pai 'animal'
bethooven.emitirSom(); // Bethooven está emitindo som

// Método da subclasse 'cachorro'
bethooven.latir(); // Bethooven está latindo!

Animal leao = new Animal(nome:"Gato horripelmente grande");
leao.emitirSom(); // Gato horripelmente grande está emitindo som
// leao.latir(); <- ERRO, leao não é um objeto 'cachorro' então não pode latir!
```

```
Bethooven está emitindo som
Bethooven está latindo!
Gato horripelmente grande está emitindo som
```


A classe Object

- Object é a superclasse de todas as classes em Java;
- Todas as classes em Java, direta ou indiretamente, herdam da classe Object;
- Ela define métodos comuns que podem ser usados em qualquer classe, como toString(), equals(), hashCode(), e getClass();
- Esses métodos podem ser sobrescritos (overridden) nas classes que você criar para fornecer comportamentos específicos;

A classe Object

Breve introdução a **overriding** ou **sobrescrita** (veremos ela depois!)

```
// 3 - A classe Object
Pessoa carlos = new Pessoa(nome:"Carlos", idade:24);
Pessoa fulano = new Pessoa(nome:"Fulano", idade:20);

// Hash mostrando o alocamento da classe na memória
System.out.println(carlos.toString()); // secao20.Pessoa@41629346

// Comparação direta
System.out.println(carlos.equals(fulano)); // false

// Retornar um valor hash pro objeto
System.out.println(carlos.hashCode()); // 1096979270

// Fizemos a sobrescrita do método toString na classe 'Pessoa'
System.out.println(carlos.toString()); // Nome: Carlos, Idade: 24
```

```
public class Pessoa {
    private String nome;
    private int idade;

    public Pessoa(String nome, int idade){
        this.nome = nome;
        this.idade = idade;
    }

    // Override - Sobrescrita
    // Temos de declarar ela (veremos mais a frente!)
    // NÃO É UMA BOA PRÁTICA FAZER ISSO COM MÉTODOS DO JAVA!
    @Override
    public String toString(){
        return "Nome: " + nome + ", Idade: " + idade;
    }
}
```

Overriding

- Overriding (ou Sobrescrita) é um recurso da Programação Orientada a Objetos (POO) que permite a uma subclasse fornecer uma implementação específica para um método que já está definido na superclasse;
- O método sobrescrito na subclasse deve ter a mesma assinatura (mesmo nome e parâmetros) que o método da superclasse;
- A palavra-chave `@Override` é usada para indicar que um método está sendo sobrescrito;
- Overriding permite que a subclasse modifique ou especialize o comportamento herdado de uma superclasse;

Overriding (sobrescrita)

Se a função “pai” retorna um double, a versão sobrescrita dela na subclasse tem de **OBRIGATORIAMENTE** retornar um double também!

```
// 4 - Override ou Sobrescrita
// Cada objeto possui uma versão de calcularArea()
// Porém todas retornam o double obrigatório que
// é oriundo da classe pai 'Forma'!
Quadrado q1 = new Quadrado(lado:4);
Circulo c1 = new Circulo(raio:3.2);

System.out.println(q1.calcularArea()); // 16.0
System.out.println(c1.calcularArea()); // 32.169908772759484
```

```
public class Forma {

    public double calcularArea(){
        // Especificamos que o retorno é em double, logo
        // todas as subclasses tem de retornar um double
        // quando fazem overriding nessa função!
        System.out.println(x:"Calculando area...");
        return 0;
    }
}
```

```
public class Circulo extends Forma {  
    private double raio;  
  
    public Circulo(double raio) {  
        this.raio = raio;  
    }  
  
    @Override  
    public double calcularArea(){  
        return Math.PI * Math.pow(raio, b:2);  
    }  
}
```

```
public class Quadrado extends Forma {  
    private double lado;  
  
    public Quadrado(double lado) {  
        this.lado = lado;  
    }  
  
    @Override  
    public double calcularArea(){  
        return Math.pow(lado, b:2);  
    }  
}
```

Método super

- super é uma palavra-chave em Java usada para referenciar a superclasse de uma subclasse;
- Através de super, uma subclasse pode:
 - Chamar o construtor da superclasse;
 - Acessar métodos e atributos da superclasse que foram sobrescritos na subclasse;
- O uso de super é comum quando a subclasse deseja reutilizar parte do comportamento da superclasse;

Método super

```
// 5 - Super
Funcionario funcionario = new Funcionario(nome:"Carlos", salario:3000);
funcionario.exibirDetalhes(); // Nome do funcionário: Carlos

Gerente gerente = new Gerente(nome:"Marcos", salario:5000, bonusAdicional:1000);
gerente.exibirDetalhes(); // Nome do funcionário: Marcos
                        // Bonus adicional de: 1000.0

System.out.println(funcionario.calcularBonus()); // 300.0
System.out.println(gerente.calcularBonus()); // 1500.0
```

```
public class Funcionario {
    protected String nome;
    protected double salario;

    public Funcionario(String nome, double salario) {
        this.nome = nome;
        this.salario = salario;
    }

    public void exibirDetalhes(){
        System.out.println("Nome do funcionário: " + nome);
    }

    public double calcularBonus(){
        return salario * .1;
    }
}
```

```
public class Gerente extends Funcionario {
    private double bonusAdicional;

    public Gerente(String nome, double salario, double bonusAdicional) {
        super(nome, salario);
        this.bonusAdicional = bonusAdicional;
    }

    // Como o bonûs do gerente é maior, vamos dar um Override na função já criada!
    // Ela retorna a função de calculo de bonus criada em funcionário e
    // soma ela com o bonus adicional presente aqui na classe
    @Override
    public double calcularBonus(){
        return super.calcularBonus() + bonusAdicional;
    }

    // Podemos retornar quaisquer funções da classe pai
    @Override
    public void exibirDetalhes(){
        super.exibirDetalhes();
        System.out.println("Bonus adicional de: " + bonusAdicional);
    }
}
```

O que é Abstraction?

- Abstração é um dos pilares fundamentais da Programação Orientada a Objetos (POO);
- Ela consiste em **ocultar os detalhes complexos de uma implementação**, expondo apenas as funcionalidades essenciais para o usuário;
- Abstração ajuda a **simplificar o uso de objetos**, escondendo a complexidade interna e focando apenas nas operações relevantes;
- Em Java, abstração é alcançada com o uso de **classes abstratas e interfaces**;

O que é abstraction (abstração)

Classes abstratas

- Uma classe abstrata em Java é uma classe que **não pode ser instanciada diretamente**;
- Ela serve como um **"modelo"** para outras classes, definindo métodos que as subclasses devem implementar
- Classes abstratas podem conter métodos:
 - **Abstratos** (sem implementação), que devem ser implementados pelas subclasses;
 - **Concretos** (com implementação), que podem ser herdados ou sobrescritos;
- A principal função de uma classe abstrata é **fornecer uma estrutura comum para subclasses**, garantindo que elas implementem certos métodos;

Classes abstratas

ELAS NÃO INSTANCIAM OBJETOS, SERVEM DE MOLDE PARA QUE SUAS SUBCLASSES TENHAM UMA ESTRUTURA PREDEFINIDA, MAS DEVEM ESTAR NAS INSTANCIAS DOS OBJETOS

Métodos abstratos devem ser herdados, método não abstratos tem caráter opcional, seu método construtor também é obrigatório, e os métodos abstratos herdados são manipulados com **override ou sobrescrita**

Podemos instanciar os objetos de uma classe abstrata utilizando as subclasses que foram criadas com ela!

```
// 6 - Classe Abstrata
InstrumentoMusical violao = new Violao(nome:"Violão");
InstrumentoMusical bateria = new Bateria(nome:"Bateria");

violao.exibirDetalhes(); // Instrumento: Violão
bateria.exibirDetalhes(); // Instrumento: Bateria
violao.tocar(); // Tocando as cordas do: Violão
bateria.tocar(); // Batendo nos tambores da: Bateria
```

```
// Definimos ela como 'Abstract class'!
abstract class InstrumentoMusical {
    protected String nome;

    public InstrumentoMusical(String nome) {
        this.nome = nome;
    }

    // Criação de métodos abstratos
    // public abstract 'retorno' 'nome do método()'

    // Abstrato: Todas as subclasses devem implementar esse método
    // Utilizando override!
    public abstract void tocar();

    // Concreto: Pode ser herdado
    public void exibirDetalhes(){
        System.out.println("Instrumento: " + nome);
    }
}
```

```
// Não tem herança propriamente dita
// Mas precisa de extends!
public class Violao extends InstrumentoMusical{
    public Violao(String nome){
        super(nome);
    }

    // Implementação de métodos abstratos, sempre com OVERRIDE!
    @Override
    public void tocar() {
        System.out.println("Tocando as cordas do: " + nome);
    }
}
```

```
// Mesmo exemplo de Violao.java
public class Bateria extends InstrumentoMusical{
    public Bateria(String nome){
        super(nome);
    }

    @Override
    public void tocar() {
        System.out.println("Batendo nos tambores da: " + nome);
    }
}
```


Interfaces

- Uma interface em Java é um contrato que define um conjunto de métodos que uma classe deve implementar;
- Ela não fornece a implementação desses métodos, apenas suas assinaturas;
- Uma classe pode implementar múltiplas interfaces, permitindo maior flexibilidade em comparação à herança simples;
- As interfaces são ideais para definir comportamentos que podem ser compartilhados por classes não relacionadas;
- Desde Java 8, interfaces podem conter métodos concretos com a palavra-chave default;

Interfaces

```
// 7 - Interfaces
// Note que tal como as classes abstratas, primeiro usamos o nome da
// Superclasse 'Pagamento' e só depois usamos os nomes das subclasses
// sendo elas CartaoCredito e TransferenciaBancaria!
Pagamento cartao = new CartaoCredito();
Pagamento trans = new TransferenciaBancaria();

cartao.processarPagamento(valor:100); // Pagamento de R$100.0, via cartão de crédito.
cartao.exibirRecibo(valor:100); // Recibo do pagamento com valor de R$100.0

trans.processarPagamento(valor:150); // Pagamento de R$150.0, via transferência bancária.
trans.exibirRecibo(valor:150); // Recibo do pagamento com valor de R$150.0
```

```
// Declaramos ela como 'interface'
interface Pagamento {

    // Nela temos métodos abstratos e default

    // Métodos abstratos - OBRIGATÓRIOS PARA IMPLEMENTAR
    void processarPagamento(double valor);

    // Métodos default - IMPLEMENTAÇÃO OPCIONAL!
    default void exibirRecibo(double valor){
        System.out.println("Recibo do pagamento com valor de R$" + valor);
    }
}
```

```
// Ao contrário do 'extends' em herança, nas interfaces
// usamos o termo 'extends'
public class CartaoCredito implements Pagamento {

    @Override
    public void processarPagamento(double valor) {
        System.out.println("Pagamento de R$" + valor + ", via cartão de crédito.");
    }
}
```

```
// Mais uma vez, usamos 'implements' ao usar interfaces!
public class TransferenciaBancaria implements Pagamento{

    @Override
    public void processarPagamento(double valor) {
        System.out.println("Pagamento de R$" + valor + ", via transferência bancária.");
    }
}
```

Padrão parecido com abstração, métodos abstratos devem ser obrigatoriamente implementados, métodos default são opcionais, a estrutura dos métodos e argumentos deve ser respeitada!

Os métodos apesar de receberem suas assinaturas com propriedades e etc só receberão tratamento pros dados quando seus devidos métodos forem implementados, por isso que métodos default precisam ter a lógica completa, uma vez que eles não são obrigatoriamente implementados!

Múltiplas Interfaces

- Em Java, uma classe pode implementar várias interfaces, permitindo que ela herde comportamentos de diferentes fontes;
- Diferente da herança, onde uma classe pode herdar de apenas uma superclasse, com interfaces a classe pode "herdar" comportamentos de várias interfaces;
- Isso oferece maior flexibilidade ao criar classes que precisam combinar comportamentos de diferentes domínios;

Multiplas Interfaces

```
// 8 - Múltiplas Interfaces
Documento doc = new Documento(documento:"Arquivo de texto");
doc.imprimir(); // Imprimindo o documento
doc.salvar(); // Salvando o documento
doc.instrucaoParaSalvar(); // Instruções!
```

```
interface Imprimivel {
    void imprimir();
}

interface Salvavel {
    void salvar();
}

// Método default de qualquer interface é opcional na implementação!
default void instrucaoParaSalvar(){
    System.out.println(x:"Instruções!");
}

}

public class Documento implements Imprimivel, Salvavel {
    private String documento;

    public Documento(String documento) {
        this.documento = documento;
    }

    @Override
    public void imprimir(){
        System.out.println(x:"Imprimindo o documento");
    }

    @Override
    public void salvar(){
        System.out.println(x:"Salvando o documento");
    }
}
```

Default methods em Interfaces

- Default Methods são métodos concretos (com implementação) dentro de interfaces;
- Introduzidos no Java 8, eles permitem adicionar novas funcionalidades a interfaces existentes sem quebrar a compatibilidade com classes que já as implementam;
- Com métodos default, você pode fornecer uma implementação padrão que pode ou não ser sobrescrita pelas classes que implementam a interface;

Default methods em interfaces

```
// 9 - Default Methods nas Interfaces
CalculadoraAvancada calc = new CalculadoraAvancada();
System.out.println(calc.somar(a:12, b:20)); // 32
System.out.println(calc.multiplicar(a:2, b:5)); // pipipipopopo 10
```

```
interface Calculadora {

    int somar(int a, int b);

    // Pode ser utilizado sem implementação
    default int multiplicar(int a, int b){
        return a * b;
    }

}
```

```
public class CalculadoraAvancada implements Calculadora{

    @Override
    public int somar(int a, int b) {
        return a + b;
    }

    // Se quisermos podemos sobrescrever métodos DEFAULT também!
    @Override
    public int multiplicar(int a, int b){
        System.out.println(x:"pipipipopopo");
        return a * b;
    }

}
```

Classe Abstrata x Interface

- **Classe Abstrata:**
 - Uma classe que não pode ser instanciada diretamente;
 - Pode conter métodos abstratos (sem corpo) e métodos concretos (com corpo);
 - Permite a existência de atributos e construtores;
 - Serve para definir um comportamento comum, que pode ser herdado por subclasses;
 - Uma classe pode herdar apenas uma única classe abstrata;
- **Interface:**
 - Um contrato que define métodos que uma classe deve implementar;
 - Antes do Java 8, continha apenas métodos abstratos. A partir do Java 8, pode ter métodos default e static com implementação;
 - Não pode ter construtores ou atributos de instância, apenas constantes;
 - Uma classe pode implementar múltiplas interfaces ao mesmo tempo;

Diferença entre classe abstrata X interface

O que é Polimorfismo?

- **Polimorfismo** é um dos pilares da Programação Orientada a Objetos (POO);
- O termo significa "muitas formas" e permite que uma única interface (ou tipo) seja usada para diferentes tipos de objetos;
- Em Java, o polimorfismo permite que um objeto de uma subclasse seja tratado como um objeto de sua superclasse, e também que métodos da superclasse sejam sobrescritos pelas subclasses;
- Existem dois tipos de polimorfismo:
 - **Polimorfismo de Sobrescrita (Override):** Quando uma subclasse fornece sua própria implementação de um método herdado da superclasse;
 - **Polimorfismo de Sobrecarga (Overload):** Quando vários métodos têm o mesmo nome, mas com assinaturas diferentes (não será o foco aqui);

O que é polimorfismo?

```
// 10 - Polimorfismo
InstrumentoMusical violino = new Violino(nome:"Violino");
violino.exibirDetalhes(); // Instrumento: Violino
violino.tocar(); // Tocando o Violino
```

```
public class Violino extends InstrumentoMusical {
    public Violino(String nome){
        super(nome);
    }

    @Override
    public void tocar(){
        System.out.println("Tocando o " + nome);
    }
}
```



Note o override

Exercícios

Composição de Objetos

```
// Exercício 01
Endereco adress = new Endereco(rua:"In time", cidade:"Somewhere", numero:1700);
Pessoa2 alguem = new Pessoa2(nome:"Fulano", idade:22, adress);
alguem.exibirInfo();
// Nome: Fulano, Idade: 22
// Rua: In time, Cidade: Somewhere, Número: 1700
```

```
public class Endereco {
    private String rua;
    private int numero;
    private String cidade;

    public Endereco(String rua, String cidade, int numero) {
        this.rua = rua;
        this.cidade = cidade;
        this.numero = numero;
    }

    public String exibirInfo(){
        return "Rua: " + rua + ", Cidade: " + cidade + ", Número: " + numero;
    }
}
```

```
public class Pessoa2 {
    private String nome;
    private int idade;
    private Endereco endereco;

    public Pessoa2(String nome, int idade, Endereco endereco) {
        this.nome = nome;
        this.idade = idade;
        this.endereco = endereco;
    }

    public void exibirInfo(){
        System.out.println("Nome: " + nome + ", Idade: " + idade);
        System.out.println(endereco.exibirInfo());
    }
}
```


Herança e sobrescrita de métodos (Overriding)

```
// Exercício 02
Veiculo v01 = new Veiculo(nome:"Algo genérico!");
v01.acelerar(); // O veículo Algo genérico!, está acelerando!
Veiculo v02 = new Moto(nome:"PCX");
v02.acelerar(); // A moto PCX, está acelerando!
Veiculo v03 = new Carro(nome:"Kwid");
v03.acelerar(); // O carro Kwid, está acelerando!
```

```
public class Veiculo {
    protected String nome;

    public Veiculo(String nome){
        this.nome = nome;
    }

    public void acelerar(){
        System.out.println("O veículo " + nome + ", está acelerando!");
    }
}
```

```
public class Carro extends Veiculo{

    public Carro(String nome) {
        super(nome);
    }

    @Override
    public void acelerar(){
        System.out.println("O carro " + nome + ", está acelerando!");
    }
}
```

```
public class Moto extends Veiculo{

    public Moto(String nome) {
        super(nome);
    }

    @Override
    public void acelerar(){
        System.out.println("A moto " + nome + ", está acelerando!");
    }
}
```

Uso do método Super

```
// Exercício 03
Cachorro fureebis = new Cachorro(nome:"Fureebis", som:"auau", raca:"Caramelo");
fureebis.exibirDetalhes(); // Nome do animal: Fureebis, Som do animal: auau, Raça
do Cachorro: Caramelo
```

```
public class Animal {
    protected String nome;
    protected String som;

    public Animal(String nome, String som){
        this.nome = nome;
        this.som = som;
    }
}
```

```
public class Cachorro extends Animal{
    private String raca;
    public Cachorro(String nome, String som, String raca) {
        super(nome, som);
        this.raca = raca;
    }

    public void exibirDetalhes(){
        System.out.println("Nome do animal: " + nome + ", Som do animal: " + som + ", Raça
do Cachorro: " + raca);
    }
}
```

Classe abstrata e interface

```
// Exercício 04
FuncionarioTempoIntegral fti = new FuncionarioTempoIntegral(nome:"Fulano",
salarioBase:1000);
FuncionarioMeioPeriodo fmp = new FuncionarioMeioPeriodo(nome:"Beltrano",
salarioPorHora:50, horasTrabalhadas:5);

System.out.println("Salário Fulano: " + fti.calcularSalario()); // Salário Fulano:
1000.0
System.out.println("Salário Beltrano: " + fmp.calcularSalario()); // Salário
Beltrano: 250.0

fti.adicionarBeneficio(beneficio:"Trabalhar até morrer!");
// Benefício adicionado para tempo integral: Trabalhar até morrer!
fmp.adicionarBeneficio(beneficio:"Benefício de não ter benefício kkkk");
// Benefício adicionado para meio-período: Benefício de não ter benefício kkkk
```

```
abstract class Funcionario {
    protected String nome;

    public Funcionario(String nome) {
        this.nome = nome;
    }

    public abstract double calcularSalario();
}
```

```
public class FuncionarioMeioPeriodo extends Funcionario implements Beneficios{

    private double salarioPorHora;
    private int horasTrabalhadas;

    public FuncionarioMeioPeriodo(String nome, double salarioPorHora, int horasTrabalhadas){
        super(nome);
        this.salarioPorHora = salarioPorHora;
        this.horasTrabalhadas = horasTrabalhadas;
    }

    @Override
    public double calcularSalario(){
        return salarioPorHora * horasTrabalhadas;
    }

    @Override
    public void adicionarBeneficio(String beneficio){
        System.out.println("Benefício adicionado para meio-período: " + beneficio);
    }
}
```

```
public class FuncionarioTempoIntegral extends Funcionario implements Beneficios{

    private double salarioBase;

    public FuncionarioTempoIntegral(String nome, double salarioBase){
        super(nome);
        this.salarioBase = salarioBase;
    }

    @Override
    public double calcularSalario(){
        return salarioBase;
    }

    @Override
    public void adicionarBeneficio(String beneficio){
        System.out.println("Beneficio adicionado para tempo integral: " + beneficio);
    }
}
```

```
interface Beneficios {
    void adicionarBeneficio(String beneficio);
}
```

Implementação de múltiplas interfaces e polimorfismo

Podemos usar **instanceof** para saber se um objeto é instância de alguma coisa!

```
// Exercício 05
Pilotavel meuHidroAviao = new Hidroaviao();
Navegavel meuBarco = new Barco();
Pilotavel meuAviao = new Aviao();

meuAviao.pilotar(); // Pilotando o avião!
meuBarco.navegar(); // Navegando com o barco!
meuHidroAviao.pilotar(); // Pilotando o hidroavião!

operarVeiculo(meuAviao); // Este veículo é pilotável
operarVeiculo(meuHidroAviao); // Este veículo é pilotável e Esse veículo é navegável
operarVeiculo(meuBarco); // Esse veículo é navegável
}
public static void operarVeiculo(Object veiculo){
    // Usamos instanceof pra saber se ele faz parte de um determinado objeto!
    if(veiculo instanceof Pilotavel){
        System.out.println(x:"Este veículo é pilotável");
    }
    if (veiculo instanceof Navegavel){
        System.out.println(x:"Esse veículo é navegável");
    }
}
}
```

```
public class Barco implements Navegavel{

    @Override
    public void navegar() {
        System.out.println(x:"Navegando com o barco!");
    }

}
```

```
public class Aviao implements Pilotavel{

    @Override
    public void pilotar() {
        System.out.println(x:"Pilotando o avião!");
    }

}
```

```
public class Hidroaviao extends Barco implements Pilotavel{

    @Override
    public void pilotar() {
        System.out.println(x:"Pilotando o hidroavião!");
    }

}
```

Métodos e classes final

- O que é final em Java?
 - A palavra-chave final pode ser usada em classes, métodos e variáveis;
 - Quando aplicada a classes e métodos, impede modificações e sobrescritas;
- Classes final
 - Uma classe marcada como final não pode ser estendida;
 - Uso: Quando você quer garantir que a implementação de uma classe não seja alterada por subclasses;
- Métodos final
 - Um método marcado como final não pode ser sobrescrito por subclasses;
 - Uso: Para proteger comportamentos essenciais que não devem ser alterados;
- Quando usar final?
 - Classes final: Use quando quiser impedir que uma classe seja herdada;
 - Métodos final: Use quando quiser garantir que métodos importantes não sejam sobrescritos;

Métodos e classes final

```
// 1 - Classes com Final
ContaBancaria conta = new ContaBancaria(saldo:500);
System.out.println("Saldo: " + conta.getSaldo());
```

```
// O final vai impedir que ela seja herdada!
public final class ContaBancaria {
    private double saldo;

    public ContaBancaria(double saldo){
        this.saldo = saldo;
    }

    public double getSaldo(){
        return saldo;
    }
}
```

```
// Não conseguimos estender uma classe com final (ContaBancaria)
//public class NovaConta extends ContaBancaria{

//}
```

```
public class Veiculo {  
    public final void ligarMotor(){  
        System.out.println(x:"Ligando o motor");  
    }  
}
```

```
public class Carro extends Veiculo{  
  
    // Não conseguimos fazer sobrescrita de um método FINAL!  
    // @Override  
    // public final void ligarMotor(){  
    //     System.out.println("O veículo está ligando o motor!");  
    // }  
  
}
```

CLASSE COM FINAL NÃO PODE SER HERDADA

MÉTODO COM FINAL NÃO PODE SER SOBRESCRITO

Reflection API

- O que é Reflexão (Reflection API)?
 - Reflexão é uma API que permite inspecionar e modificar o comportamento de classes, métodos e atributos em tempo de execução;
 - Com a Reflection API, você pode obter informações sobre uma classe, como seus métodos, construtores e campos, sem conhecer a classe em tempo de compilação;
- Principais Usos da Reflexão
 - Inspeção de classes e seus membros (métodos, construtores e campos);
 - Invocação de métodos de forma dinâmica;
 - Acessar e modificar campos privados e protegidos;
 - Criação de instâncias de classes em tempo de execução;
- Como Usar a Reflexão?
 - Obter a classe: Use `Class.forName()` ou `objeto.getClass()`;
 - Obter métodos, campos e construtores: Utilize os métodos da classe `Class` como `getMethods()`, `getFields()`
 - Invocar métodos: Use `Method.invoke()` para chamar métodos de forma dinâmica;

Reflection API

```
// 2 - Reflection API
// Envelopamos ele num try-catch para ele retornar o erro caso haja algum
// Usamos <?> Porque não sabemos o tipo de classe!
try {
    Class<?> classePessoa = Class.forName(className:"secao20.Avançando.Pessoa");
    // No construtor passamos o tipo de dados que temos nos parâmetros

    Constructor<?> construtor = classePessoa.getConstructor(...parameterTypes:String.class, int.class);

    // Aqui instanciamos o objeto pessoa com os parâmetros 'joão' e '25'
    Object pessoa = construtor.newInstance(...initargs:"João", 25);

    // Aqui atribuímos o método 'dizerOla' em uma variável
    Method metodoDizerOla = classePessoa.getMethod(name:"dizerOla");

    // Aqui chamamos o método usando o objeto 'pessoa' que criamos antes!
    metodoDizerOla.invoke(pessoa);
} catch (Exception e) {
    e.printStackTrace();
}
```

```

public class Pessoa {
    private String nome;
    private int idade;

    public Pessoa(String nome, int idade){
        this.nome = nome;
        this.idade = idade;
    }

    public void dizerOla(){
        System.out.println(x:"Olá, mundo!");
    }
}

```

```

Olá, mundo!
PS C:\Users\

```

Podemos pegar campos e mudar a acessibilidade deles também!

```

// Podemos pegar campos também (campo nome)
Field campoNome = classePessoa.getDeclaredField(name:"nome");

// E mudar sua acessibilidade também!
campoNome.setAccessible(flag:true);

// E seus dados também, aqui mudamos de 'João' para 'Maria'
campoNome.set(pessoa, value:"Maria");

```

O que são Exceções? (erros)

- Exceções são eventos inesperados que interrompem a execução normal de um programa, geralmente causados por erros de lógica ou condições imprevistas;
- Em Java, todas as exceções são objetos que herdam da classe `Throwable`;
- As exceções podem ser verificadas (checked) ou não verificadas (unchecked):
 - **Verificadas:** São verificadas pelo compilador e devem ser tratadas ou declaradas explicitamente. Ex: `IOException`;
 - **Não verificadas:** Ocorrências em tempo de execução, não são obrigatórias de tratar. Ex: `NullPointerException`;
- O tratamento de exceções permite **capturar e lidar com erros**, mantendo a estabilidade do programa e prevenindo encerramentos abruptos;

O que são exceções (erros)

`IOException` – IO quer dizer in-out ou entrada e saída de dados!

Tempo de compilação vs Tempo de execução

- **Tempo de Compilação:** Quando o código é verificado e convertido em bytecode pelo compilador
 - Erros detectados: Sintaxe, tipos, declarações faltantes;
 - Exemplo: Esquecer um ponto e vírgula ou declarar uma variável incorretamente;
- **Tempo de Execução:** Quando o programa já compilado é executado pela máquina virtual
 - Erros detectados: Lógica, exceções não verificadas, como `NullPointerException`;
 - Exemplo: Divisão por zero ou acesso a um índice inexistente de array;



Tempo de compilação e Tempo de execução

Bloco try-catch

- O bloco try-catch permite capturar e tratar exceções que ocorrem durante a execução do programa;
- O código que pode gerar uma exceção é colocado dentro do bloco try;
- O bloco catch captura a exceção e executa o código de tratamento;
- É possível ter múltiplos blocos catch para tratar diferentes tipos de exceções;
- Usar try-catch evita que o programa encerre inesperadamente em caso de erro;

Bloco try-catch

Try {código} catch(tipoDeErro e) {mensagemDeErro}

Note que temos de saber o tipo de erro e endereçar o catch de forma correta, ao tentar dividir por 0 temos uma exceção aritmética por exemplo, mas podemos só usar 'exception' como um indicador de erro genérico

O 'e' é o objeto de erro, ele possui incontáveis métodos que podem auxiliar, tal como o getMessage() que vai exibir a mensagem de erro pro usuário (veremos isso depois)

É importante dizer que o objeto de erro 'e' é genérico e pode ter qualquer nome, mas costuma ser 'e' ou 'error'

```
// 1 - try catch
// Dividindo por zero
try {
    int a = 0;
    int b = 10;
    int resultado = b / a;
    System.out.println(resultado);
    // Note o tipo de erro
} catch (ArithmeticException e) {
    System.out.println(x:"Dividir por 0 não é possível!");
}

// Acessando indice não existente
try {
    int[] numeros = {1,2,3};
    System.out.println(numeros[3]);
} catch (Exception e) {
    System.out.println(x:"Erro genético não identificado!");
    System.out.println("Msg: " + e.getMessage()); // Msg: Index 3 out of bounds for
length 3
}
```

Bloco finally

- O bloco finally é usado junto com try-catch para garantir que o código seja executado, independentemente de uma exceção ter sido lançada ou não;
- O código no bloco finally sempre será executado, mesmo que haja um retorno antecipado no try ou catch;
- É comumente usado para liberar recursos, como fechar arquivos, conexões de banco de dados ou liberar memória;
- Um bloco finally pode ser usado com ou sem um bloco catch;

Bloco Finally

```
// 2 - finally
// com catch
try {

    int[] numeros = {1,2,3};
    System.out.println(numeros[3]);
    // Podemos também definir qual o tipo de erro e suas tratativas
    // não precisamos trabalhar sempre com erros genéricos
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println(x:"Erro genérico não identificado, finally!");
    System.out.println("Msg: " + e.getMessage()); // Msg: Index 3 out of bounds for
length 3
} finally {
    System.out.println(x:"Executou o finally!");
}
```

```
// com try
try {

    int[] numeros = {1,2,3};
    System.out.println("Acessando índice existente: " + numeros[2]);
    // Podemos também definir qual o tipo de erro e suas tratativas
    // não precisamos trabalhar sempre com erros genéricos
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println(x:"Erro genérico não identificado, finally!");
    System.out.println("Msg: " + e.getMessage());
} finally {
    System.out.println(x:"Executou o finally!");
}
```

```
// sem catch
try {

    int[] numeros = {1,2,3};
    System.out.println("Acessando indice existente sem catch: " + numeros[1]);
} finally {
    System.out.println(x:"Executou o finally!");
}
```

Exceções Verificadas vs. Não Verificadas

- **Exceções Verificadas:**
 - São verificadas em tempo de compilação;
 - O compilador exige que você trate ou declare essas exceções com try-catch ou throws;
 - Exemplo: IOException, SQLException;
- **Exceções Não Verificadas:**
 - Ocorrem em tempo de execução;
 - O compilador não exige que sejam tratadas;
 - Geralmente indicam erros de lógica no código;
 - Exemplo: NullPointerException, ArrayIndexOutOfBoundsException;
- **Diferença Principal:**
 - Verificadas: Tratamento obrigatório;
 - Não verificadas: Tratamento opcional, mas pode ser necessário para evitar falhas;

Exceções verificadas vs não verificadas

```
// 3 - verificadas e não verificadas
// Vamos ver um pouco de tratamento de arquivos, veremos isso
// com mais detalhes mais a frente no curso

// verificada / não permite compilar
try{
    BufferedReader reader = new BufferedReader(new FileReader(fileName:"arquivo.
txt"));
    String linha = reader.readLine();
    System.out.println(linha);
} catch (Exception e){
    System.out.println("Erro ao ler arquivo " + e.getMessage());
    // Erro ao ler arquivo arquivo.txt (O sistema não pode encontrar o arquivo
    especificado)
}

// não verificadas / permite compilar
String texto = null;
System.out.println(texto.length());
// RETORNA ERRO
```


Exceções são Objetos da Classe Throwable

- A palavra-chave throw é usada para lançar uma exceção explicitamente em um bloco de código;
- Pode ser usada para lançar exceções verificadas e não verificadas;
- Permite criar exceções personalizadas ou lançar exceções existentes em cenários específicos;
- Após lançar uma exceção com throw, o fluxo de execução é interrompido e a exceção deve ser tratada ou propagada;
- Sintaxe: throw new ExcecaoTipo("mensagem de erro");



Exceções são objetos da classe throwable

```
// 4 - Exceções com throw
try{
    validarIdade(idade:20);
    validarIdade(idade:10);
} catch (Exception e){
    System.out.println("Erro: " + e.getMessage());
    // Erro: Idade deve ser maior que 18
}
}

// Função para a atividade 4
public static void validarIdade(int idade){
    if (idade < 18){
        throw new IllegalArgumentException(s:"Idade deve ser maior que 18");
    }
    // Isso não será impresso se a idade for menor que 18
    System.out.println("Idade válida: " + idade);
}
```

Basicamente lançamos exceções com suas respectivas mensagens e tipos

Criando Exceções Customizadas

- Exceções customizadas permitem criar erros específicos para o contexto da sua aplicação;
- Para criar uma exceção personalizada, você herda de `Exception` (verificada) ou `RuntimeException` (não verificada);
- Você pode adicionar mensagens de erro personalizadas e comportamentos específicos;
- As exceções personalizadas melhoram a legibilidade do código e a tratativa de erros específicos;

Criando exceções customizadas

```
package secao21;  
  
// Temos de usar 'extends exception'!  
// Toda classe que herda de exception tem que enviar  
// a mensagem obrigatoriamente! getMessage()  
public class SaldoInsuficienteException extends Exception{  
  
    public SaldoInsuficienteException(String mensagem){  
        super(mensagem);  
    }  
  
}
```

Aqui criamos a exceção, a mensagem que ela herda é a 'e.getMessage()'

```
public class Banco {  
    private double saldo;  
  
    public Banco(double saldo) {  
        this.saldo = saldo;  
    }  
  
    public void sacar(double valor) throws SaldoInsuficienteException{  
        if(valor > saldo){  
            // Exception  
            throw new SaldoInsuficienteException("Saldo insuficiente para sacar " + valor);  
        }  
        saldo -= valor;  
        System.out.println("Saque de: " + valor + ", realizado com sucesso, Saldo atual: " +  
            saldo);  
    }  
}
```

Criamos uma classe que aplica a exceção em um caso específico

```
// 5 - exceções personalizadas
Banco minhaConta = new Banco(saldo:5000);

try{
    minhaConta.sacar(valor:6000);
} catch(SaldoInsuficienteException e ){
    System.out.println("Erro: " + e.getMessage());
    // Erro: Saldo insuficiente para sacar 6000.0
}
```

Instanciamos o objeto e vemos a exceção em funcionamento!

Uso do throws em Métodos

- A palavra-chave throws é usada na declaração de um método para indicar que ele pode lançar uma ou mais exceções;
- Métodos que lançam exceções verificadas devem declarar essas exceções usando throws;
- Exceções não verificadas **não precisam** ser declaradas com throws;
- O throws delega o tratamento da exceção para quem invoca o método, em vez de tratá-la diretamente dentro do método;
- Sintaxe: public void metodo() throws ExcecaoTipo { ... }

Uso do throws.+ em métodos

```
// Função para a atividade 6
// Aqui temos duas exceções, a primeira é quando o arquivo não é encontrado
// a segunda é para quando o caminho do arquivo não é encontrado
public static void processarArquivo(String caminho) throws FileNotFoundException,
IOException{
    if(caminho == null || caminho.isEmpty()){
        throw new IOException("Caminho inválido");
    }
    File arquivo = new File(caminho);
    if(!arquivo.exists()){
        throw new FileNotFoundException("Arquivo não encontrado");
    }
    System.out.println("Arquivo encontrado com sucesso!");
}
```

```
// 6 - throws em métodos
// Note o encadeamento de catches, se tirarmos os tratamentos
// o código não rodará, quando se herda uma função ou classe com
// throws, é obrigatório tratar os erros dela também!
try {
    processarArquivo("/var/www/arquivo.txt");
} catch (FileNotFoundException e) {
    System.out.println("Erro: " + e.getMessage());
} catch (IOException e){
    System.out.println("Erro: " + e.getMessage());
}
```

Encadeamento de Exceções

- Encadeamento de exceções permite associar uma exceção a outra, para rastrear a causa original de um erro;
- Útil para identificar a causa raiz de uma exceção ao longo de múltiplos níveis de métodos;
- A exceção primária (externa) pode "encapsular" a exceção original (causa interna) usando o método `initCause()` ou passando a causa no construtor da exceção;
- Facilita a depuração ao preservar o histórico completo das exceções que ocorreram;
- Sintaxe: `new Excecao("Mensagem", causa)` ou `excecao.initCause(causa)`;

Encadeamento de exceções

```
// Função para atividade 7
public static void abrirArquivo(String caminho){
    try {
        if(caminho == null){
            throw new NullPointerException(s:"Caminho nulo");
        }
        throw new FileNotFoundException(s:"Arquivo não encontrado");
    } catch (FileNotFoundException e) {
        // Criação da exceção com sua mensagem
        NullPointerException npe = new NullPointerException(s:"Erro ao processar arquivo");
        // Definimos que a causa da exceção acima é a exceção 'FileNotFoundException'
        npe.initCause(e);
        // Lançamos a exceção criada
        throw npe;
        // Encadeamos uma exceção como causa da outra e vice-versa
    }
}
```

```
// 7 - Encadeamento de exceções
try {
    abrirArquivo(caminho:null);
} catch (Exception e) {
    System.out.println("Mensagem: " + e.getMessage());
    System.out.println("Causa: " + e.getCause());
}
```

```
Mensagem: Caminho nulo
Causa: null
```

(ATENÇÃO!!!)

Boas práticas

- **Use exceções de forma adequada:** Não abuse de exceções para controle de fluxo normal do programa;
- **Especifique o tipo de exceção:** Capture exceções específicas sempre que possível, evitando capturar a exceção genérica `Exception`;
- **Trate exceções localmente:** Trate a exceção no ponto mais próximo onde o erro pode ser recuperado;
- **Limpeza de recursos:** Use `finally` para garantir que recursos sejam liberados corretamente;
- **Encapsule exceções internas:** Utilize encadeamento de exceções para preservar a causa original;
- **Evite exceções silenciosas:** Não capture exceções sem fornecer uma mensagem de log ou saída apropriada;
- **Lance exceções específicas:** Crie exceções customizadas quando necessário para representar melhor os erros de domínio;

Boas práticas em exceções

Não troque if-elses por try catch

Exceções multicatch

- O multicatch permite capturar várias exceções em um único bloco catch;
- Introduzido no Java 7, simplifica o código e evita duplicação ao capturar diferentes tipos de exceções com a mesma lógica de tratamento;
- Reduz o número de blocos catch, melhorando a legibilidade e manutenção do código;
- Sintaxe: catch (Excecao1 | Excecao2 e) { ... };

Exceções multicatch

```
// 8 - multicatch
try {
    processarArquivo(caminho:null);
} catch (NullPointerException | IOException e) {
    System.out.println("Erro Multicatch : " + e.getMessage());
} catch (MeuErro e){
    System.out.println(x:"Erro de exemplo");
}
```

Erros separados por |

Relançando exceção

- O re-lançamento de exceções permite capturar uma exceção, realizar alguma ação (ex.: log, limpeza de recursos) e, em seguida, **lançar novamente a exceção**;
- Útil para delegar o **tratamento completo da exceção para outro método ou camada da aplicação**;
- **Quando usar?** Quando é necessário adicionar informações ao erro, executar alguma lógica intermediária ou realizar a limpeza de recursos;
- Para preservar a pilha de execução (stack trace) original, é importante relançar a exceção sem modificá-la;
- **Sintaxe:** throw e; no bloco catch após o tratamento desejado;

Relançando exceção

```
// Função para atividade 9
public static void processarDados(String dados) throws Exception{
    try {
        if(dados == null){
            throw new NullPointerException(s:"Os dados são nulos");
        }
    } catch (Exception e) {
        System.out.println(x:"Tratamento, criação de log");
        throw e; // <----
    }
}
```

esse throw

```
// 9 - Relançando as exceções
try {
    processarDados(dados:null);
} catch (Exception e) {
    System.out.println(x:"Outra coisa");
    System.out.println("Pilha de execução: " + e.getStackTrace());
}
```

Joga pra esse throw

O que é Manipulação de Arquivos em Java?

- Manipulação de arquivos em Java envolve a leitura, escrita, e gerenciamento de arquivos e diretórios no sistema de arquivos;
- Java fornece várias APIs para trabalhar com arquivos, como `FileReader`, `FileWriter`, `BufferedReader`, e a API `java.nio.file`;
- É possível trabalhar com arquivos de texto (como `.txt`) e arquivos binários (como imagens, vídeos);
- A manipulação de arquivos permite armazenar e recuperar dados, realizar operações de entrada/saída (I/O) e até serializar objetos;
- O uso adequado dessas APIs garante segurança e eficiência no gerenciamento de arquivos, evitando problemas como corrupção de dados e vazamentos de recursos;



O que é manipulação de arquivos em java?

Lendo arquivos com Java

- A leitura de arquivos em Java pode ser feita usando várias classes da API, como `FileReader`, `BufferedReader`, e a API `java.nio.file`;
- `FileReader` é uma classe básica para ler dados de arquivos de texto, caracter por caracter;
- `BufferedReader` é uma classe que otimiza a leitura de arquivos, permitindo ler linhas inteiras de uma vez, o que melhora o desempenho;
- A manipulação adequada dos arquivos inclui sempre fechar o arquivo após a leitura, ou usar o `try-with-resources` para garantir o fechamento automático;
- `Try-with-resources` é um try catch com argumentos no try, que neste caso serão os arquivos, e eles fecham automaticamente no fim da execução;

Lendo arquivos em java

Vamos fazer os exercícios seguindo o modo “correto”, com try catch com recursos ou try-with-resources, ele garante que independentemente de erro ou sucesso, os recursos serão sempre fechados

```
// 1 - Leitura de Arquivos
/* Usualmente usariamos o caminho da pasta, porém para critério de economia de tempo, vamos instanciar
uma variavel com o diretório atual!
*/
// Vamos concatenar pra chegar até a pasta certa!
String currentDir = System.getProperty(key:"user.dir") + "\\secao22\\";
System.out.println(currentDir);
// Acima temos dita variável, podemos concatenar os arquivos que queremos abrir com ela! e abaixo é o
resultado que esperamos!
System.out.println(x:"C:\\Users\\Carlos\\Desktop\\Programação 2024 e Redes\\Java HdC\\secao22\\");
// Usamos duas contra barras \\ pra dar 'escape' tipo "\\n", o uso de somente uma iria retornar
inconsistências!
// Windows -> \
// Linux -> /

// FileReader = caracter por caracter
try (FileReader reader = new FileReader(currentDir + "arquivo.txt")) {

    // Essa variável servirá para iterar sobre os caracteres do documento, e mais a frente sofrerá um
    casting para char (do contrário teremos apenas o código ASCII de cada letra)
    int character;

    // O -1 indica o fim do arquivo, ou seja enquanto o caracter não chegar no fim do arquivo ele irá
    continuar!
    while((character = reader.read()) != -1){
        // Use apenas 'print', 'println' colocaria cada letra em uma linha!
        System.out.print((char)character);
    }
} catch (Exception e) {
    System.out.println("Erro ao ler arquivo " + e.getMessage());
}
}
```

```
// BufferedReader = linha por linha
// Note como ele recebe uma instância do FileReader pra ler o arquivo!
try (BufferedReader reader = new BufferedReader(new FileReader(currentDir + "arquivo.txt"))) {
    // Aqui lemos a linha inteira
    String linha;
    // E usamos o null para indentificar o fim do arquivo!
    while((linha = reader.readLine()) != null){
        System.out.println(linha);
    }
} catch (Exception e) {
    System.out.println("Erro ao ler arquivo " + e.getMessage());
}
}
```

Upon a hill lived a lonely man, with his heart drowned in sorrows, and his mind living in a endless dream
Upon a hill lived a lonely man, with his heart drowned in sorrows, and his mind living in a endless dream

Escrita de arquivos em Java

- A escrita de arquivos em Java pode ser feita utilizando classes como `FileWriter` e `BufferedWriter`;
- `FileWriter` é uma classe básica que permite escrever dados de texto em arquivos, caracter por caracter;
- `BufferedWriter` é uma classe que melhora a eficiência, permitindo escrever grandes quantidades de dados de uma vez, otimizando o desempenho;
- É possível sobrescrever um arquivo existente ou adicionar novos dados ao final do arquivo (modo `append`);
- O uso de `try-with-resources` garante o fechamento automático dos arquivos após a operação, prevenindo vazamentos de recursos;

Escrita de arquivos em java

```
// 1 - Leitura de arquivos
/* Usualmente usaríamos o caminho da pasta, porém para critério de economia de tempo, vamos instanciar
uma variavel com o diretório atual!
*/
// Vamos concatenar pra chegar até a pasta certa!
String currentDir = System.getProperty(key:"user.dir") + "\\secao22\\";
System.out.println(currentDir);
// Acima temos dita variável, podemos concatenar os arquivos que queremos abrir com ela! e abaixo é o
resultado que esperamos!
System.out.println(x:"C:\\Users\\Carlos\\Desktop\\Programação 2024 e Redes\\Java HdC\\secao22\\");
// Usamos duas contra barras \\ pra dar 'escape' tipo "\\n", o uso de somente uma iria retornar
inconsistências!
// Windows -> \
// Linux -> /

// FileReader = caracter por caracter
try (FileReader reader = new FileReader(currentDir + "arquivo.txt")) {

    // Essa variável servirá para iterar sobre os caracteres do documento, e mais a frente sofrerá um
    casting para char (do contrário teremos apenas o código ASCII de cada letra)
    int character;

    // O -1 indica o fim do arquivo, ou seja enquanto o caracter não chegar no fim do arquivo ele irá
    continuar!
    while((character = reader.read()) != -1){
        // Use apenas 'print', 'println' colocaria cada letra em uma linha!
        System.out.print((char)character);
    }
} catch (Exception e) {
    System.out.println("Erro ao ler arquivo " + e.getMessage());
}

System.out.println();
```

```

// BufferedReader = linha por linha
// Note como ele recebe uma instância do FileReader pra ler o arquivo!
try (BufferedReader reader = new BufferedReader(new FileReader(currentDir + "arquivo.txt"))) {
    // Aqui lemos a linha inteira
    String linha;
    // E usamos o null para indentificar o fim do arquivo!
    while((linha = reader.readLine()) != null){
        System.out.println(linha);
    }
} catch (Exception e) {
    System.out.println("Erro ao ler arquivo " + e.getMessage());
}

// 2 - Escrever em arquivos
// Se o arquivo não existe, ele será criado!

// FileWriter
try (FileWriter writer = new FileWriter(currentDir + "saida.txt")) {
    writer.write(str:"And quietly there, he rot \n");
    writer.write(str:"Waiting for the day, that never came \n");
} catch (Exception e) {
    System.out.println("Erro ao escrever no arquivo " + e.getMessage());
}

// BufferedWriter, note o uso de FileWriter!
try (BufferedWriter writer = new BufferedWriter(new FileWriter(currentDir + "saida2.txt"))) {
    writer.write(str:"But the world continued");
    writer.newLine(); // Cria uma nova linha, evitando o uso de \n
    writer.write(str:"And no one mourned him \n");
} catch (Exception e) {
    System.out.println("Erro ao escrever no arquivo " + e.getMessage());
}

// O uso de append adiciona conteudo, usar write de novo substituiria os dados antigos, é necessário
habilitar ele, tal como podemos ver na linha abaixo!
try (BufferedWriter writer = new BufferedWriter(new FileWriter(currentDir + "saida2.txt", append:true))) {
    writer.append(csq:"In the end, we all rot");
} catch (Exception e) {
    System.out.println("Erro ao escrever no arquivo " + e.getMessage());
}

```


Serialização de Objetos (Serializable)

- Serialização é o processo de converter um objeto em um fluxo de bytes para armazená-lo em um arquivo ou transmiti-lo pela rede;
- Deserialização é o processo inverso: recriar o objeto a partir do fluxo de bytes;
- Para serializar um objeto em Java, a classe deve implementar a interface `Serializable`;
- Vantagem: Permite salvar o estado do objeto e restaurá-lo posteriormente;
- Importante: Os campos que não devem ser serializados devem ser marcados como `transient`;
- Sintaxe: `ObjectOutputStream` para serializar e `ObjectInputStream` para deserializar;
- Caso de uso:
 - Um sistema de e-commerce que mantém os dados do carrinho de compras de um usuário em cache. Quando o usuário retorna ao site, o carrinho pode ser deserializado e rapidamente recuperado, permitindo que o estado do carrinho de compras seja restaurado.

Serialização de objetos (serializable)

```
// Para a serialização de objetos é necessário importar essa interface!
import java.io.Serializable;

public class Pessoa implements Serializable {
    // Um identificador único!
    public static final Long serialVersionUID = 1L;

    private String nome;
    private int idade;

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
}
```

```
// 3 - Serialização de objetos
Pessoa pessoa = new Pessoa(nome:"Someone", idade:66);

System.out.println(pessoa.getNome());

// Início do processo de serialização
/*
oss é o nome de uma variável, qualquer nome serve, usamos oss para referenciar o proprio
ObjectOutputStream

arquivos serializados tem a extensão .ser
*/
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(currentDir + "pessoa.ser"))) {

    oos.writeObject(pessoa);
    System.out.println(x:"Objeto serializado com sucesso");

} catch (Exception e) {
    System.out.println("Erro ao serializar objeto " + e.getMessage());
}
}
```

```
// Deserialização de objetos
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(currentDir + "pessoa.ser"))) {

    // Criação da variável para deserializar o objeto, note o tipecast que usamos para converter o objeto
    para sua classe 'natural'
    Pessoa pessoa2 = (Pessoa) ois.readObject();
    System.out.println("Nome: " + pessoa2.getNome());
    System.out.println("Idade: " + pessoa2.getIdade());

} catch (Exception e) {
    System.out.println("Erro ao deserializar objeto " + e.getMessage());
}

// SERIALIZAR = OUTPUT
// DESERIALIZAR = INPUT
```


Manipulação de Arquivos Binários em Java

- Arquivos binários contêm dados que **não são diretamente legíveis como texto** (ex.: imagens, vídeos, executáveis);
- Para manipular arquivos binários, usamos classes como `FileInputStream` e `FileOutputStream`, que leem e escrevem bytes diretamente;
- Essas classes funcionam com dados em nível de byte, permitindo ler e escrever qualquer tipo de arquivo;
- O uso adequado dessas classes inclui sempre fechar os streams após a leitura/escrita, ou usar `try-with-resources` para garantir o fechamento automático;
- Exemplo: Copiar ou modificar arquivos binários (imagens, PDFs, etc.);



Manipulação de arquivos binários em java

```
// 4 - Manipulação de binários
// Cópia de imagem
try (
    // Colocamos os dois argumentos assim para evitar 'resource leaks'
    FileInputStream fis = new FileInputStream(currentDir + "imagem.jpg");
    FileOutputStream fos = new FileOutputStream(currentDir + "copia_imagem.jpg");
) {
    // Acima ditamos a fonte dos bytes e o destino dele, abaixo vamos passar os bytes do original para o
    // arquivo novo!

    int byteData;
    // 0 != -1 indica o final do arquivo!
    while((byteData = fis.read()) != -1){
        fos.write(byteData);
    }
    // Enquanto houver bytes no arquivo fonte, eles serão escritos no arquivo copia
    System.out.println(x:"Arquivo copiado com sucesso");
} catch (Exception e) {
    System.out.println("Erro ao copiar arquivo " + e.getMessage());
}
```

```
// Cópia de vídeo, usamos Buffered para arquivos maiores
try (
    BufferedInputStream bis = new BufferedInputStream(new FileInputStream(currentDir + "video.mkv"));
    BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(currentDir + "copia_video.
    mkv"));
) {

    // Por ser um arquivo maior usaremos um array de bytes
    byte[] buffer = new byte[1024]; // Buffer 1kb em 1kb, buffers menores garantem integridade caso haja
    instabilidade na rede, dessa forma os pacotes perdidos não corrompem o
    int bytesLidos;

    while((bytesLidos = bis.read(buffer)) != -1){
        bos.write(buffer, off:0, bytesLidos);
        // Argumentos: dados, intervalo, e o número de bytes a serem escritos
    }
    System.out.println(x:"Vídeo copiado com sucesso");
} catch (Exception e) {
    System.out.println("Erro ao copiar vídeo " + e.getMessage());
}
```

Cópia de vídeo acima!

Manipulação de Imagens

- A manipulação de imagens em Java é feita através das classes da API `java.awt` e `java.awt.image`, que permitem editar, desenhar e modificar imagens;
- O `BufferedImage` é a classe principal para armazenar imagens na memória, permitindo manipulação pixel a pixel;
- A classe `Graphics2D` fornece métodos para desenhar na imagem, como adicionar texto, formas e até outras imagens;
- Exemplos de manipulação:
 - Inserir texto ou marcas d'água;
 - Redimensionar ou rotacionar a imagem;
 - Alterar cores ou aplicar filtros;

Manipulação de Imagens

```
// 5 - Manipulação de imagens
// Vamos inserir um texto no centro de imagem.jpg
try {

    BufferedImage imagem = ImageIO.read(new File(currentDir + "imagem.jpg"));
    if(imagem == null){
        System.out.println("A imagem não pode ser carregada");
        return;
    }
    Graphics g2d = imagem.createGraphics();

    // Definindo propriedades do texto
    g2d.setFont(new Font(name:"Arial", Font.BOLD,size:50));
    FontMetrics fm = g2d.getFontMetrics();
    String texto = "Marca d'água";

    // Centralizar texto na imagem
    int larguraTexto = fm.stringWidth(texto);
    int alturaTexto = fm.getHeight();

    // Posicionamento, feito através de calculo x = hori y = vert
    int x = (imagem.getWidth() - larguraTexto) / 2;
    int y = (imagem.getHeight() - alturaTexto) / 2 + fm.getAscent();

    // Desenhar retângulo
    g2d.setColor(Color.black);
    g2d.fillRect(x - 10, y - fm.getAscent(), larguraTexto + 20, alturaTexto);

    // Desenhar o texto sobre o retângulo
    g2d.setColor(Color.red);
    // Usamos o x e y que calculamos previamente!
    g2d.drawString(texto, x, y);
    // Liberação de recursos
    g2d.dispose();

    // Salvar a imagem
    File outputFile = new File(currentDir + "imagem_com_texto.png");
    ImageIO.write(imagem, formatName:"png", outputFile);
    System.out.println("Imagem modificada com sucesso");

} catch (Exception e) {
    System.out.println("Erro ao processar imagem " + e.getMessage());
}
```

Manipulação de Diretórios e Arquivos

- `java.nio.file` é a API moderna para manipulação de arquivos e diretórios em Java;
- A API oferece métodos eficientes e simplificados para operações como criação, exclusão, movimentação, e cópia de arquivos e diretórios;
- Classes principais:
 - `Files`: Contém métodos estáticos para operações com arquivos e diretórios (criar, copiar, mover, etc.);
 - `Paths`: Representa caminhos de arquivos e diretórios de forma multiplataforma;
- Vantagens:
 - Melhor suporte a sistemas de arquivos modernos;
 - Manuseio de exceções mais consistente com a classe `IOException`;
 - Funcionalidades adicionais, como operações atômicas e manipulação de metadados de ar

Manipulação de diretórios e arquivos

```
// 6 - Manipulação de arquivos e diretórios

// Criando um diretório
Path caminhoDiretorio = Paths.get(currentDir + "diretorioNovo");

try {
    if(!Files.exists(caminhoDiretorio)){
        Files.createDirectories(caminhoDiretorio);
        System.out.println("Diretório criado com sucesso: " +
            caminhoDiretorio.toString());
    } else {
        System.out.println(x:"Diretório já existe!");
    }
} catch (Exception e) {
    System.out.println("Erro ao criar diretório " + e.getMessage());
}

/*
Perceba que nunca instanciamos Paths ou Files!
Nós conseguimos usar os métodos pois eles são static!
*/
```

```
// Criar, copiar e mover arquivos

Path caminhoArquivoOriginal = Paths.get(currentDir + "arquivo_criado.
txt");
Path caminhoArquivoCopia = Paths.get(currentDir + "arquivo_criado_copia.
txt");
Path caminhoArquivoMovido = Paths.get(currentDir, ...
more:"diretorioNovo", "arquivo_movido.txt");

try {
    // Criar
    if(!Files.exists(caminhoArquivoOriginal)){
        Files.createFile(caminhoArquivoOriginal);
        System.out.println("Arquivo criado com sucesso: " +
            caminhoDiretorio.toString());
    } else {
        System.out.println(x:"Arquivo já existe!");
    }

    // Copiar
    // O if é só pra evitar um erro de sintaxe no contexto geral do
    código!
    if(!Files.exists(caminhoArquivoCopia)){
        // É a linha abaixo que importa!
        Files.copy(caminhoArquivoOriginal, caminhoArquivoCopia);
    }

    // Mover
    Files.move(caminhoArquivoCopia, caminhoArquivoMovido);

} catch (Exception e) {
    System.out.println("Erro ao tentar manipular arquivo" + e.getMessage
());
}
```

Arquivos temporários

- Arquivos Temporários são arquivos criados para armazenar dados temporários e que normalmente são excluídos após o uso;
- Java oferece o método `Files.createTempFile()` para criar arquivos temporários de forma simples e segura;
- Quando usar?
 - Armazenar dados intermediários que não precisam ser mantidos após o processamento;
 - Evitar o uso excessivo de memória em operações temporárias grandes (ex: upload de arquivos);
 - Em operações que requerem um espaço de trabalho temporário durante o processamento;
- Por que usar?
 - Garantia de que o arquivo será armazenado no local apropriado para arquivos temporários no sistema;
 - Arquivos temporários podem ser automaticamente excluídos quando o programa termina;



Arquivos temporários

```
// 7 - Arquivos temporários
try {
    Path arquivoTemporario = Files.createTempFile(prefix:"meuTempFile",
    suffix:".txt");

    System.out.println("Arquivo criado em: " + arquivoTemporario.
    toAbsolutePath());

    Files.writeString(arquivoTemporario, csq:"Brincadeira bicho!");

    String conteudo = Files.readString(arquivoTemporario);

    System.out.println("Conteudo do arquivo temporário: " + conteudo);

    Files.deleteIfExists(arquivoTemporario);
} catch (Exception e) {
    System.out.println("Erro ao criar arquivo temporário" + e.getMessage
    ());
}
```


Manipulação de Arquivos Comprimidos (ZIP)

- Java oferece suporte nativo para arquivos ZIP através da API `java.util.zip`;
- Arquivos ZIP são usados para compactar e descompactar arquivos e diretórios, economizando espaço de armazenamento e facilitando a transferência de múltiplos arquivos;
- Classes principais:
 - `ZipOutputStream`: Usada para criar e gravar arquivos ZIP;
 - `ZipInputStream`: Usada para ler e extrair arquivos de um arquivo ZIP;
 - `ZipEntry`: Representa uma entrada (arquivo ou diretório) dentro do arquivo ZIP;
- Aplicações comuns:
 - Agrupar múltiplos arquivos para envio;
 - Reduzir o tamanho de arquivos para economizar espaço;
 - Arquivos temporários ou backups compactados;

Manipulação de arquivos comprimidos (ZIP)

```
// Arquivos ZIP
// Compressão
Path arquivoOriginal = Paths.get(currentDir + "arquivo.txt");
Path arquivoZip = Paths.get(currentDir + "arquivo_comprimido.zip");
// Try with resources
try (
    ZipOutputStream zos = new ZipOutputStream(new FileOutputStream(
        arquivoZip.toFile()));
    FileInputStream fis = new FileInputStream(arquivoOriginal.toFile());
) {
    // Cria uma entrada ZIP para o arquivo
    ZipEntry zipEntry = new ZipEntry(arquivoOriginal.getFileName().toString());
    zos.putNextEntry(zipEntry);

    // Lê o conteúdo do arquivo e grava no ZIP (buffer)
    byte[] buffer = new byte[1024];
    int bytesRead;

    while((bytesRead = fis.read(buffer)) != -1){
        zos.write(buffer, off:0, bytesRead);
    }

    // Fechar a entrada do ZIP
    zos.closeEntry();
    System.out.println(x:"Arquivo compactado com sucesso!");
} catch (Exception e) {
    System.out.println("Erro ao compactar arquivo " + e.getMessage());
}
```

```

// Descompressão
Path arquivoZipado = Paths.get(currentDir + "arquivo_comprimido.zip");
Path destino = Paths.get(currentDir + "descompactado");
// Try with resources
try (
    ZipInputStream zis = new ZipInputStream(new FileInputStream(arquivoZipado.
        toFile()));
) {

    ZipEntry zipEntry;

    // Criar o diretório se não existir
    if(!Files.exists(destino)){
        Files.createDirectories(destino);
    }

    // Iteração sobre os arquivos ZIP
    while((zipEntry = zis.getNextEntry()) != null){

        Path caminhDestino = destino.resolve(zipEntry.getName());

        // Try normal
        try(
            FileOutputStream fos = new FileOutputStream(caminhDestino.toFile())
        ){
            byte[] buffer = new byte[1024];
            int bytesRead;

            while((bytesRead = zis.read(buffer)) != -1){
                fos.write(buffer, off:0, bytesRead);
            }
        }
        System.out.println("Arquivo descompactado " + caminhDestino);
        zis.closeEntry();
    }

} catch (Exception e) {
    System.out.println("Erro ao descompactar arquivo " + e.getMessage());
}

```


Leitura e escrita de arquivos CSV

- CSV (Comma-Separated Values) é um formato amplamente usado para armazenar dados tabulares de forma simples e legível;
- Em Java, arquivos CSV podem ser manipulados manualmente ou usando bibliotecas como OpenCSV;
- Leitura Manual: Simples de implementar, ideal para arquivos pequenos e estruturados de forma básica;
- Uso de OpenCSV: Biblioteca que facilita a leitura e escrita de arquivos CSV com suporte a operações avançadas como escape de caracteres e manipulação de cabeçalhos;



Leitura e escrita de arquivos CSV

```
// 9 - Manipulação de CSV
// Leitura
String arquivoCSV = currentDir + "dados.csv";
String linha;
String separador = ",";

// Try with resources
try (BufferedReader br = new BufferedReader(new FileReader(arquivoCSV))) {
    // Funcionará enquanto não houver mais linhas
    while((linha = br.readLine()) != null){
        // Dividir a string / explodir a string em array
        String[] dados = linha.split(separador);
        System.out.println("Nome: " + dados[0] + ", Idade: " + dados[1] + ",
            Cidade: " + dados[2]);
    }
} catch (Exception e) {
    System.out.println("Erro ao ler csv: " + e.getMessage());
}
```

```
// Escrita
String arquivoEscritaCSV = currentDir + "dadosEscrita.csv";
// Try with resources
try (FileWriter writer = new FileWriter(arquivoEscritaCSV)) {

    // Inserção de linha a linha, respeitando o separador (,) e quebrando a linha
    // no final
    writer.append(csq:"Nome,Idade,Cidade\n");
    writer.append(csq:"Isabel,22,Rio de Janeiro\n");
    writer.append(csq:"Maria,45,Floripa\n");
    writer.append(csq:"Igor,15,Guarulhos\n");

    System.out.println(x:"Conteudo gravado no CSV com sucesso");
} catch (Exception e) {
    System.out.println("Erro ao escrever no csv: " + e.getMessage());
}
```

O que são Generics?

- Generics são uma funcionalidade que permite criar classes, interfaces e métodos que operam com tipos de dados parametrizados;
- Introduzidos no Java 5, os Generics ajudam a criar código mais flexível e reutilizável, sem a necessidade de definir tipos de dados específicos;
- **Objetivo:** Permitir que uma estrutura de dados ou método trabalhe com diferentes tipos de dados, proporcionando segurança de tipos durante a compilação;
- Generics evitam o uso excessivo de casting e garantem que tipos incorretos não sejam atribuídos a coleções ou métodos;
- Usados amplamente em classes de coleções como **List**, **Set**, e **Map**, garantindo que os elementos armazenados sejam do tipo correto;
- **Benefícios:**
 - Segurança de tipos: Erros de tipo são detectados em tempo de compilação;
 - Reutilização de código: Um código genérico pode ser usado para diferentes tipos sem duplicação;
 - Legibilidade: O código se torna mais claro e previsível ao definir explicitamente o tipo de dados;

O que são generics?

Criando classes genéricas

- Classes Genéricas permitem que uma classe opere com tipos de dados diferentes de maneira parametrizada;
- Uma classe genérica utiliza parâmetros de tipo (como **<T>**, **<E>**, **<K, V>**) que podem ser substituídos por tipos específicos durante a criação de objetos;
- **Vantagens:**
 - Maior flexibilidade e reutilização do código sem a necessidade de duplicação para tipos diferentes;
 - Segurança de tipos em tempo de compilação, prevenindo erros relacionados a tipos;
- **Sintaxe:** A classe genérica é declarada com um parâmetro de tipo dentro de colchetes angulares (**<>**), como **<T>**, onde T pode ser qualquer nome que representa um tipo de dado;
- Classes genéricas são amplamente utilizadas nas coleções do Java (ex: **List<T>**, **Map<K, V>**);
- Exemplo de Uso:
 - Uma classe genérica pode manipular qualquer tipo de dados, seja Integer, String, ou até objetos mais complexos, proporcionando grande versatilidade;

Criando classes genéricas

```
// Usamos o <T> para declarar a classe como genérica
public class Caixa<T> {

    // Conteudo pode ser qualquer tipo de dado!
    private T conteudo;

    // Um setter usando T
    public void adicionar(T conteudo){
        this.conteudo = conteudo;
    }

    // Um getter usando T
    public T obter(){
        return conteudo;
    }
}
```

T só se torna realmente algo quando é instanciado, isso é, quando se usa Classe<TipoDeDado>, como no exemplo abaixo!

```
// 1 - Classes genéricas
// Ao instanciar uma classe genérica temos de definir finalmente o tipo de dado
// como no exemplo abaixo, <Integer>
Caixa<Integer> caixaInteira = new Caixa<>();
// Podemos adicionar '100' pois é um tipo de inteiro
caixaInteira.adicionar(conteudo:100);
// Tentar adicionar qualquer outro tipo de dado resultaria num erro
//caixaInteira.adicionar("conteudo");
System.out.println(caixaInteira.obter()); // 100

Caixa<String> caixaString = new Caixa<>();
caixaString.adicionar(conteudo:"Fulano");
//caixaString.adicionar(20);
System.out.println(caixaString.obter()); // Fulano
```

Criando métodos genéricos

- Métodos Genéricos permitem que um método opere com diferentes tipos de dados, de forma parametrizada, semelhante a classes genéricas;
- A principal diferença é que o parâmetro de tipo é declarado no nível do método e pode ser usado em qualquer método dentro da classe, sem tornar a classe inteira genérica;
- Sintaxe: O parâmetro de tipo é declarado entre <> antes do tipo de retorno do método;
- Exemplo: <T> T metodoGenerico(T parametro);
- Vantagens:
 - Maior flexibilidade: O mesmo método pode manipular diferentes tipos de dados sem precisar duplicar código;
 - Segurança de tipos: Verificação em tempo de compilação garante que os tipos usados sejam consistentes;
- Métodos genéricos são muito usados em algoritmos de ordenação, busca, e manipulação de coleção
- O uso de métodos genéricos ajuda a evitar o uso excessivo de casting, tornando o código mais robusto;

Criando métodos genéricos

```
// Obter maior valor entre 2 valores de forma genérica e implementando o 'Comparable'

// O uso da classe em <T> tem o propósito de permitir o uso de 'compareTo' em dados genéricos!
public static <T extends Comparable<T>> T obterMaior(T valor1, T valor2){
    return (valor1.compareTo(valor2) > 0) ? valor1 : valor2;
}

// Percorrer arrays independentemente do tipo

// Note o uso do array de valores genéricos T[]
public static <T> void imprimirArray(T[] array){
    for(T elemento : array){
        System.out.print(elemento + ", ");
    }
}
```

```
// 2 - Métodos genéricos
System.out.println("Maior valor entre 5 e 10: " + obterMaior(valor1:5,
valor2:10)); // 10

System.out.println("Maior valor entre Java e Python: " + obterMaior(valor1:"Java",
valor2:"Python")); // Python

String[] letras = {"a", "b", "c", "d"};
Integer[] numeros = {12, 24, 35, 31, 6};
Boolean[] bools = {true, false, true, false, false};

imprimirArray(letras);
imprimirArray(numeros);
imprimirArray(bools);

// a, b, c, d, 12, 24, 35, 31, 6, true, false, true, false, false,
```


Tipos delimitados (bounded types)

- Tipos delimitados (Bounded Types) permitem restringir o tipo genérico a um subtipo ou supertipo específico, proporcionando maior controle sobre o tipo de dado aceito por uma classe ou método genérico;
- Ao usar bounded types, você pode impor que o tipo genérico herde de uma classe específica ou implemente uma interface;
- Sintaxe:
 - Para restringir um tipo genérico a uma classe ou interface, usamos o operador `extends`;
 - Exemplo: `<T extends Number>` – Isso significa que `T` deve ser uma subclasse de `Number` (como `Integer`, `Double`, etc.);
- Vantagens:
 - Permite reutilizar código genérico com maior flexibilidade e segurança, mas restringindo tipos apropriados;
 - Oferece segurança de tipos em tempo de compilação;



Tipos delimitados (Bounded types)

```
// Dessa forma T só poderá ser tipos numéricos!
public class Comparador<T extends Number> {

    public T obterMaior(T numero1, T numero2){
        // O Double é o tipo numero mais bem 'aceito', usando esse simples 'parse' resolve
        // o problema em tempo de compilação que a linha geraria, note que esse parse não
        // substitui a declaração de tipo na inicialização do método!
        if(numero1.doubleValue() > numero2.doubleValue()){
            return numero1;
        } else {
            return numero2;
        }
    }
}
```

```
// 3 - Tipos delimitados (Bounded types)
Comparador<Integer> comparadorInteiros = new Comparador<>();

System.out.println("Maior número entre 20 e 40: " + comparadorInteiros.obterMaior(
    numero1:20, numero2:40)); //

// Como restringimos T para dados numéricos, a linha abaixo retornaria um erro,
// uma vez que o comparador não pode ser inicializado como String!
// Comparador<String> comparadorString = new Comparador<>();
```

Coringas (Wildcards)

- Coringas (ou Wildcards) são utilizados para tornar os tipos genéricos mais flexíveis ao permitir que parâmetros genéricos aceitem diferentes tipos sem especificá-los diretamente;
- Um Coringa é representado pelo símbolo ?, e pode ser usado para definir tipos genéricos que não precisam ser exatamente um tipo específico;
- Tipos de Coringas:
 - Coringa Sem Limitação (?): Aceita qualquer tipo;
 - Coringa com Extensão (? extends T): Aceita T ou qualquer subtipo de T;
 - Coringa com Supertipo (? super T): Aceita T ou qualquer supertipo de T;

Coringas (Wildcards)

```
// Coringas
// Veremos Lists (ou Collections) em mais detalhes depois
public static void imprimirLista(List<?> lista){
    for(Object elemento : lista){
        System.out.println(elemento);
    }
}

// Coringa com bounded types
// Note que limitamos os tipos para 'Number' ou numéricos!
public static double somarNumeros(List<? extends Number> lista){
    double soma = 0;
    for(Number numero : lista){
        // Mais um parse para evitar erro de compilação!
        soma += numero.doubleValue();
    }
    return soma;
}

// Coringa aceitando apenas tipos numéricos específicos, dessa forma aceitamos apenas
// números inteiros!
public static void adicionarNumeros(List<? super Integer> lista){
    for(int i = 1; i <= 5; i++){
        lista.add(i);
    }
}
```

```
// 4 - Coringas (Wildcards) - Veremos mais sobre Listas depois!
List<Integer> numeros2 = List.of(e1:1,e2:2,e3:3,e4:4,e5:5,e6:6);
List<String> palavras2 = List.of(e1:"Teste", e2:"Java", e3:"Exemplo");

// Como o método 'imprimirLista' é genérico, podemos usar ele para imprimir ambos
// exemplos acima e/ou qualquer tipo de dado
imprimirLista(numeros2);
imprimirLista(palavras2);

// Coringa com bounded type
System.out.println(somarNumeros(numeros2)); // 21.0
// Como limitamos os valores genéricos a numerais, o uso de Strings retorna erro!
// System.out.println(somarNumeros(palavras2));

// Coringa aceitando apenas tipos numéricos específicos, aqui ele somente aceitará
// <Number> ou <Integer>, qualquer outro tipo de dado numérico retornará um erro!
List<Number> numeros3 = new ArrayList<>();
adicionarNumeros(numeros3);

// Note que se declarássemos <Integer> acima, abaixo no loop for teríamos que
// declarar Integer também! o uso de <Number> requer o uso do Object!
for(Object numero : numeros3){
    System.out.println(numero);
}
```

O que são Collections?

- Collections são estruturas de dados que permitem armazenar, organizar e manipular grupos de objetos em Java;
- A Java Collections Framework oferece uma biblioteca padrão para manipulação de coleções de dados, como listas, conjuntos e mapas;
- Principais Interfaces:
 - **List**: Uma coleção ordenada de elementos que pode conter duplicatas (ex: ArrayList);
 - **Set**: Uma coleção que não permite elementos duplicados (ex: HashSet);
 - **Map**: Uma coleção de pares chave-valor, onde as chaves são únicas (ex: HashMap);
- Componentes Importantes:
 - **Iteradores**: Permitem navegar pelos elementos de uma coleção de forma segura e eficiente;
 - **Métodos utilitários**: Como sort, reverse, e shuffle, facilitam operações comuns em coleções;
- Obs: veremos alguns exemplos com Collections, e possivelmente teremos uma seção de Collections no curso =)

O que são collections?

São arrays bombados!

Generics com Coleções (Collections)

- Generics são amplamente utilizados nas coleções do Java, como List, Set, e Map, para garantir segurança de tipos e evitar erros em tempo de execução;
- Usando Generics em coleções, é possível especificar o tipo de objetos que uma coleção pode armazenar, tornando o código mais previsível e seguro;
- Coleções Genéricas evitam a necessidade de casting explícito ao recuperar elementos e permitem que o compilador verifique se os tipos de dados inseridos são consistentes;
- Principais Coleções Genéricas:
 - **List<T>**: Armazena uma lista ordenada de elementos do tipo T;
 - **Set<T>**: Armazena uma coleção de elementos únicos do tipo T;
 - **Map<K, V>**: Armazena pares chave-valor, onde K é o tipo da chave e V é o tipo do valor;

Generics com coleções (Collections)


```

// 5 - Generics com collections
// Lista de números inteiros
List<Integer> listaInteiros = new ArrayList<>();
listaInteiros.add(e:20);
listaInteiros.add(e:145);
listaInteiros.add(e:54);
// Essa lista não aceitará números com dígitos decimais, ex: 3.14 e qualquer outro
de dado que não seja 'Integer'
for (Number numeral : listaInteiros ){
    System.out.println(numeral); // 20 145 54
}

// Criação de set
Set<String> conjuntoDePalavras = new HashSet<>();
conjuntoDePalavras.add(e:"Pamonha");
conjuntoDePalavras.add(e:"Iguana");
conjuntoDePalavras.add(e:"Iguana");
for (String palavra : conjuntoDePalavras){
    System.out.println(palavra); // Pamonha Iguana
}
// Note que o output não retornou a palavra 'Iguana' repetidamente, isso ocorre
porque sets aceitam apenas elementos únicos não repetidos!

// Criação de map
// Definimos a chave e o valor, a chave sendo 'String' e o valor sendo 'Integer'
Map<String, Integer> mapaDeIdades = new HashMap<>();
// Usamos 'put' invés de 'add' para inserir dados
mapaDeIdades.put(key:"Carlos", value:24);
mapaDeIdades.put(key:"Ana", value:20);
// O print de map é diferente:
for(Map.Entry<String, Integer> entrada : mapaDeIdades.entrySet()){
    System.out.println(entrada.getKey() + " tem " + entrada.getValue() + " anos");
    // Note o uso dos getters, com map sempre trabalhamos com chave e valores!
}

```

Uso de Generics com Interfaces

- Interfaces Genéricas permitem que o tipo de dados usado pela interface seja definido quando a interface é implementada;
- Isso proporciona flexibilidade e reutilização, permitindo que várias classes implementem a mesma interface genérica com diferentes tipos;
- Sintaxe:
 - Uma interface genérica é declarada com um parâmetro de tipo (<T>), similar a uma classe genérica;
 - Exemplo: `public interface Exemplo<T>;`
- Exemplo de Uso:
 - Uma interface genérica que define métodos comuns para lidar com vários tipos de dados, e classes que implementam essa interface com tipos específicos;

Uso de generics com interfaces

```
// Note o uso de <T> para definir a interface como genérica!  
public interface Armazenamento<T> {  
  
    void salvar(T item);  
    T recuperar();  
}
```

```
public class ArmazenamentoNumeros implements Armazenamento<Integer> {  
  
    public Integer numero;  
  
    @Override  
    public void salvar(Integer item) {  
        this.numero = item;  
    }  
  
    @Override  
    public Integer recuperar() {  
        return numero;  
    }  
}
```

```
public class ArmazenamentoTextos implements Armazenamento<String> {  
  
    public String texto;  
  
    @Override  
    public void salvar(String item) {  
        this.texto = item;  
    }  
  
    @Override  
    public String recuperar() {  
        return texto;  
    }  
}
```

```
// 6 - Generics com interfaces
```

```
Armazenamento<String> aTexto = new ArmazenamentoTextos();  
aTexto.salvar(item:"Teste");  
System.out.println(aTexto.recuperar()); // Teste  
  
Armazenamento<Integer> aInteiro = new ArmazenamentoNumeros();  
aInteiro.salvar(item:10);  
System.out.println(aInteiro.recuperar()); // 10
```

Restrição Múltipla de Tipos em Generics

- Restrição Múltipla de Tipos permite que um parâmetro genérico seja limitado a múltiplas classes e interfaces, garantindo que ele atenda a várias condições simultaneamente;
- Sintaxe:
 - **T extends ClassA & InterfaceB:** Define que o tipo T deve ser uma subclasse de ClassA e também deve implementar a InterfaceB;
 - **Apenas uma classe pode ser especificada** (pois o Java não suporta herança múltipla), mas você pode listar várias interfaces;
- Exemplo de Uso:
 - Um método genérico que exige que o tipo seja uma subclasse de uma classe específica e, ao mesmo tempo, implemente uma ou mais interfaces;



```
public class Animal {  
    public void mover(){  
        System.out.println(x:"O animal está se movendo...");  
    }  
}
```

```
public interface Nadador {  
    void nadar();  
}
```

```
public interface Voador {  
    void voar();  
}
```

```
public class Pato extends Animal implements Voador, Nadador{  
  
    @Override  
    public void nadar() {  
        System.out.println(x:"O pato está nadando");  
    }  
  
    @Override  
    public void voar() {  
        System.out.println(x:"O pato está voando");  
    }  
}
```

```
// Note que aqui omitimos o implements e colocamos um & em seu lugar!
public class CriaturaGenerica<T extends Animal & Nadador & Voador>{

    private T criatura;

    public CriaturaGenerica(T criatura){
        this.criatura = criatura;
    }

    public void usarHabilidades(){
        criatura.mover();
        criatura.voar();
        criatura.nadar();
    }
}
```

```
// 7 - Restrição múltipla de tipos
Pato pato = new Pato();
CriaturaGenerica<Pato> criaturaPato = new CriaturaGenerica<>(pato);
criaturaPato.usarHabilidades();
// A única criatura capaz de ser uma criatura genérica é o pato, pois sua classe
condiz com os requisitos que são descritos na classe criaturaGenerica, sendo eles
1: a classe deve estender a classe animal, 2 - deve implementar as interfaces
Voador e Nadador!
```

Detalhes acima!!!

Introdução às Collections

- O que são Collections?
 - Collections são estruturas de dados que permitem armazenar, organizar e manipular grupos de objetos em Java
 - Elas facilitam a manipulação de conjuntos de dados, como listas, conjuntos, filas e mapas;
- Importância das Collections na Programação
 - Proporcionam uma forma eficiente e flexível de manipular grandes quantidades de dados;
 - As Collections eliminam a necessidade de criar estruturas de dados manualmente;
 - Oferecem uma série de métodos úteis para ordenar, buscar, filtrar, adicionar e remover dados;
- Java Collections Framework (JCF)
 - A Java Collections Framework é uma biblioteca padrão do Java que oferece um conjunto de classes e interfaces para trabalhar com coleções de dados;
 - Fornece implementações de estruturas de dados comuns como List, Set, Map, e Queue;
 - As coleções são altamente flexíveis e genéricas, permitindo que tipos de dados sejam parametrizados com Generics;

Introdução às collections

Interfaces Principais das Collections

- List
 - Armazena elementos ordenados de acordo com a ordem de inserção, aceita elementos duplicados;
 - Acesso rápido a elementos por índice;
 - Implementações: ArrayList, LinkedList;
 - Aplicação: Ideal quando você precisa manter a ordem dos elementos e acessar itens por índice;
- Set
 - Armazena uma coleção de elementos únicos (não permite duplicatas), não garante a ordem de inserção, exceto com implementações específicas;
 - Implementações: HashSet (não ordenado), LinkedHashSet (mantém a ordem de inserção), TreeSet (ordenado automaticamente);
 - Aplicação: Garantir que não existam duplicatas e a ordem dos elementos não é importante;
- Map
 - Armazena pares chave-valor, onde cada chave é única, não permite chaves duplicadas;
 - Implementações: HashMap, LinkedHashMap, TreeMap;
 - Aplicação: Ideal para armazenar dados com uma associação direta entre chave e valor, como dicionários ou catálogos;

Interfaces principais das collections

Trabalhando com List

- Introdução à Interface List
 - A interface List é usada para armazenar uma sequência ordenada de elementos;
 - Permite elementos duplicados e o acesso aos itens por índice;
 - Oferece métodos eficientes para manipulação de elementos como adicionar, remover e modificar;
- Tipos de List
- ArrayList
 - Implementa List usando um array dinâmico;
 - Acesso rápido por índice, porém operações de inserção/remoção são mais lentas, especialmente em grandes listas;
- LinkedList
 - Implementa List como uma lista duplamente encadeada;
 - Inserção/remoção rápida, especialmente em qualquer posição, mas o acesso por índice é lento;

Trabalhando com list

```
// 1 - List
List<String> listaDeNomes = new ArrayList<>();

// Adicionando elementos
listaDeNomes.add(e:"Fulano");
listaDeNomes.add(e:"Ciclano");
listaDeNomes.add(e:"Beltrano");

// Resgatando elementos
System.out.println("Primeiro nome: " + listaDeNomes.get(index:0));

// Alterando elementos
listaDeNomes.set(index:1, element:"Novo Ciclano");
System.out.println("Segundo nome alterado: " + listaDeNomes.get(index:1));

// Removendo elementos
listaDeNomes.remove(index:2);
System.out.println("Lista após remoção: " + listaDeNomes);

// Procura por valor de item
System.out.println("Lista contém 'Fulano'? " + listaDeNomes.contains(o:"Fulano")); // true

// LinkedList
List<Integer> numeros = new LinkedList<>();

numeros.add(e:10);
numeros.add(e:20);
numeros.add(e:30);
System.out.println("Lista de números: " + numeros);
System.out.println(numeros.get(index:1)); // Acessa o segundo elemento
numeros.remove(index:0); // Remove o primeiro elemento
// Ver os valores facilmente, util para debugging
System.out.println(numeros.toString());
numeros.set(index:1, element:40); // Altera o segundo elemento
System.out.println(numeros.get(index:1));

// As collections tem os mesmos métodos, mas nem todas podem aplicar todos os métodos
// Exemplo: LinkedList não tem o método 'contains' como ArrayList, mas pode ser usado com 'contains' porque é uma interface de List
```


ArrayList vs LinkedList

- **ArrayList**
 - Acesso Rápido: Acesso eficiente por índice.
 - Inserção/Remoção Lenta: Operações de inserção e remoção são lentas em grandes listas, especialmente no início/meio da lista.
 - Melhor para leitura frequente: Ideal quando há mais operações de leitura do que de modificação.
- **LinkedList**
 - Inserção/Remoção Rápida: Operações de inserção e remoção são rápidas, especialmente em qualquer posição da lista.
 - Acesso Lento: Acesso por índice é mais lento devido à estrutura encadeada.
 - Melhor para inserção/remoção frequente: Ideal para cenários com muitas modificações nos dados.

ArrayList vs LinkedList (diferenças)

Trabalhando com Set

- Introdução à Interface Set
 - Set é uma coleção que não permite duplicatas;
 - Os elementos em um Set não possuem uma ordem definida, exceto em implementações específicas;
 - Ideal para armazenar dados únicos onde a ordem não é tão importante;
- Tipos de Set
 - HashSet: Não garante a ordem dos elementos, mas é eficiente para operações como adicionar e verificar elementos;
 - LinkedHashSet: Mantém a ordem de inserção dos elementos;
 - TreeSet: Mantém os elementos ordenados de acordo com a ordem natural ou um Comparador personalizado;
- Quando Usar Set vs List
 - Use Set quando não quiser duplicatas e a ordem dos elementos não for prioritária;
 - Use List quando a ordem dos elementos for importante e duplicatas forem permitidas;

Trabalhando com set

```
// 2 - Set

// HashSet - Não mantém a ordem de inserção, não permite duplicatas
Set<String> conjunto = new HashSet<>();
// Adicionando elementos
conjunto.add(e:"Item 1");
conjunto.add(e:"Item 2");
conjunto.add(e:"Item 3");
conjunto.add(e:"Carlos");
conjunto.add(e:"Beltrano");
conjunto.add(e:"Ana");
// Tentando adicionar um elemento duplicado
conjunto.add(e:"Item 1"); // Não será adicionado, pois Set não permite duplicatas
// Exibindo o conjunto
System.out.println("Conjunto: " + conjunto);
// Verificando se um elemento está no conjunto
System.out.println("Conjunto contém 'Item 1'? " + conjunto.contains(o:"Item 1")); // true

// LinjkedHashSet - Mantém a ordem de inserção, não permite duplicatas
Set<Integer> numerais = new LinkedHashSet<>();
// Adicionando elementos
numerais.add(e:1);
numerais.add(e:2);
numerais.add(e:3);
System.out.println("Conjunto de números: " + numerais);

// Treeset - Mantém a ordem natural (alfabética e numérica) dos elementos, não
permite duplicatas
Set<String> nomes = new TreeSet<>();
// Adicionando elementos
nomes.add(e:"Carlos");
nomes.add(e:"Ana");
nomes.add(e:"Beltrano");
// Exibindo o conjunto
System.out.println("Conjunto de nomes: " + nomes);

// SET não permite o uso de get, logo não é possível acessar um elemento por
índice.
// Para acessar um elemento específico, é necessário iterar sobre o conjunto.
```

Trabalhando com Map

- Introdução à Interface Map
 - Map é uma coleção que armazena pares chave-valor, onde cada chave é única;
 - Uma chave pode ter apenas um valor associado, mas os valores podem se repetir;
 - Ideal para associações entre chave e valor, como dicionários ou catálogos;
- Tipos de Map
 - **HashMap**: Não mantém a ordem de inserção. Muito eficiente para busca e inserção;
 - **LinkedHashMap**: Mantém a ordem de inserção das chaves;
 - **TreeMap**: Mantém as chaves ordenadas de acordo com a ordem natural ou um Comparator;

Trabalhando com map

```
// 3 - Map

// HashMap - Armazena pares chave-valor, não mantém a ordem de inserção
Map<String, Integer> idadeMap = new HashMap<>();
// Adicionando elementos com put
idadeMap.put(key:"Carlos", value:30);
idadeMap.put(key:"Ana", value:25);
idadeMap.put(key:"Beltrano", value:40);
// Exibindo o mapa
System.out.println("Mapa de idades: " + idadeMap);
// Inserindo um elemento com chave duplicada
idadeMap.put(key:"Carlos", value:35); // Atualiza o valor da chave "Carlos"
System.out.println("Mapa de idades após atualização: " + idadeMap);
// Acessando um valor por chave
System.out.println("Idade de Ana: " + idadeMap.get(key:"Ana"));
// removendo um elemento
idadeMap.remove(key:"Beltrano");
System.out.println("Mapa de idades após remoção: " + idadeMap);
// Verificando se uma chave existe
System.out.println("Mapa contém chave 'Carlos'? " + idadeMap.containsKey(key:"Carlos")); // true
// Verificando se um valor existe
System.out.println("Mapa contém valor 25? " + idadeMap.containsValue(value:25)); // true
// Entryset
System.out.println("Entradas do mapa: " + idadeMap.entrySet());

// LinkedHashMap - Armazena pares chave-valor, mantém a ordem de inserção
Map<String, String> capitalMap = new LinkedHashMap<>();
capitalMap.put(key:"Brasil", value:"Brasília");
capitalMap.put(key:"Estados Unidos", value:"Washington, D.C.");
capitalMap.put(key:"França", value:"Paris");
System.out.println("Mapa de capitais: " + capitalMap);
System.out.println(capitalMap.containsKey(key:"Brasil")); // true

// TreeMap - Armazena pares chave-valor, mantém a ordem natural das chaves
Map<String, Double> produtoPrecoMap = new TreeMap<>();
produtoPrecoMap.put(key:"Maçã", value:2.50);
produtoPrecoMap.put(key:"Banana", value:1.20);
produtoPrecoMap.put(key:"Laranja", value:3.00);
System.out.println("Mapa de produtos e preços: " + produtoPrecoMap);
System.out.println(produtoPrecoMap.containsKey(key:"Banana")); // true
```

Iterando sobre Collections

- Maneiras de Iterar sobre Coleções
 - For-each Loop: Simples e legível, ideal para leitura de elementos;
 - Iterator: Permite navegação sobre coleções, com a capacidade de remover elementos de forma segura durante a iteração;
 - ListIterator: Estende o Iterator, permitindo navegar em ambas as direções (para frente e para trás) em coleções baseadas em lista (como ArrayList);
- Por que usar o Iterator?
 - Permite a remoção segura de elementos enquanto você percorre a coleção, o que não é possível diretamente com um for-each;
 - Evita exceções como ConcurrentModificationException ao modificar a coleção enquanto a percorre;



Iterando sobre collections

```
// 4 - Iterando sobre coleções

// For-each para List
System.out.println(x:"Iterando sobre lista de nomes com For-Each:");
for (String nome : nomes) {
    System.out.println(nome);
}

// Iterator
System.out.println(x:"Iterando sobre conjunto de nomes com Iterator:");
Iterator<String> nomesIterator = nomes.iterator();
// Remover um elemento durante a iteração
while (nomesIterator.hasNext()) {
    String nome = nomesIterator.next();
    if(nome.equals(anObject:"Carlos")) {
        nomesIterator.remove(); // Remove "Carlos" do conjunto
    } else {
        System.out.println(nome);
    }
}

// ListIterator - precisa de uma lista, permite iteração bidirecional
System.out.println(x:"Iterando sobre lista de nomes com ListIterator:");
ListIterator<String> listIteratorNomes = listaDeNomes.listIterator();
while (listIteratorNomes.hasNext()) {
    String nome = listIteratorNomes.next();
    System.out.println(nome);
}

// Iterando em ordem
System.out.println(x:"Iterando sobre lista de nomes em ordem reversa com ListIterator:");
while (listIteratorNomes.hasPrevious()) {
    String nome = listIteratorNomes.previous();
    System.out.println(nome);
}
```

Collections Imutáveis

- O que são coleções imutáveis?
 - Coleções imutáveis são aquelas cujo conteúdo não pode ser modificado (adicionar, remover ou atualizar elementos);
 - Após a criação, não é possível alterar os dados na coleção;
 - Evita modificações acidentais e melhora a segurança de dados em ambientes onde múltiplas threads estão acessando os dados;
- Quando usar coleções imutáveis?
 - Quando você quer garantir que os dados de uma coleção não serão alterados após sua criação;
 - Melhor prática em cenários de concorrência, onde múltiplas threads podem acessar os dados simultaneamente;
 - Em APIs que retornam dados para consumidores e não devem ser modificados;
- Criando Coleções Imutáveis
 - `Collections.unmodifiableList()`: Cria uma versão imutável de uma lista existente;
 - `List.of()`, `Set.of()`, `Map.of()`: Métodos da API Java 9+ que criam coleções imutáveis diretamente;

Collections Imutáveis

```
// 5 - Collections Imutáveis

// List Imutável
List<String> listaMutavel = new ArrayList<>();
listaMutavel.add(e:"Item 1");
listaMutavel.add(e:"Item 2");
// Criando uma lista imutável a partir da lista mutável
List<String> listaImutavel = Collections.unmodifiableList(listaMutavel);
System.out.println("Lista imutável: " + listaImutavel);
// Tentando modificar a lista imutável (lançará UnsupportedOperationException)
try {
    listaImutavel.add(e:"Item 3"); // Isso causará uma exceção
} catch (UnsupportedOperationException e) {
    System.out.println("Não é possível modificar a lista imutável: " + e.getMessage());
}

// List of
List<String> listaImutavel2 = List.of(e1:"Item A", e2:"Item B", e3:"Item C");
System.out.println("Lista imutável criada com List.of: " + listaImutavel2);
// Ela também não pode ser modificada

// Set of
Set<Integer> listaImutavelSet = Set.of(e1:1, e2:2, e3:3, e4:4, e5:5);
System.out.println("Set imutável criado com Set.of: " + listaImutavelSet);
// Adivinha? também não pode ser modificada!
```


Stream em Collections

- O que é Stream?
 - Stream é uma API introduzida no Java 8 para processamento de dados em coleções de forma declarativa;
 - Permite realizar operações funcionais como filtrar, mapear e reduzir coleções, sem modificar a coleção original;
- Operações Principais em Stream
 - Intermediárias: Executadas em uma coleção, retornam um novo Stream (ex.: filter(), map(), sorted());
 - Finais: Executadas ao final do pipeline de operações, retornam um resultado ou efeito colateral (ex.: collect(), forEach(), reduce());
- A seguir veremos vários destes métodos de Stream em ação!



Streams em collections

Filtragem de Collections

- O que é filtragem de Collections?
 - Filtragem é o processo de selecionar elementos de uma coleção com base em uma condição específica;
 - Permite extrair subconjuntos de dados sem modificar a coleção original;
- Quando usar filtragem?
 - Quando você deseja trabalhar com um subconjunto dos dados em vez de toda a coleção;
 - Para limitar ou refinar dados em coleções grandes antes de realizar operações adicionais;
- Métodos para Filtragem de Collections
 - Streams (Java 8+): Usando filter() em Streams para aplicar condições de filtragem;
 - Iteração manual: Usar loops para filtrar dados de uma coleção sem Streams;

Filtragem de collections

```
// 6 - Filter
// Filtrando uma lista para encontrar os números maiores que 3
List<Integer> numerosFiltrados = numeros.stream()
    .filter(numero -> numero > 3)
    .collect(Collectors.toList());
// numero -> numero > 3 é uma expressão lambda que define a condição de filtragem
// A quebra de linha é opcional, mas ajuda na legibilidade de filtros mais complexos
System.out.println("Números filtrados maiores que 3: " + numerosFiltrados);
System.out.println("Lista original de números: " + numeros);
```

Busca de Collections

- O que é busca em Collections?
 - Busca é o processo de encontrar um elemento dentro de uma coleção, com base em uma condição ou valor específico;
 - É utilizada para verificar se um determinado dado existe na coleção ou para localizar itens que atendem a critérios específicos;
- Quando usar busca?
 - Para verificar se um valor específico existe na coleção;
 - Para encontrar um ou mais elementos que atendam a determinadas condições;
- Métodos de busca em Collections
 - `contains()`: Verifica se a coleção contém um elemento específico;
 - **Streams (Java 8+)**: Usando `findFirst()`, `findAny()`, ou `filter()` para buscar elementos com base em condições;
 - **Iteração manual**: Usar loops para buscar manualmente elementos em coleções;

Busca em collections

```
// 7 - Busca
// Busca em collections usando for
int numeroParaBuscar = 5;
boolean encontrado = false;
for(Integer numero : numeros) {
    if(numero == numeroParaBuscar) {
        encontrado = true;
        break; // Encerra o loop se o número for encontrado
    }
}
System.out.println("Número " + numeroParaBuscar + " encontrado? " + encontrado);

// Usando contains
String nomeParaBuscar = "Carlos";
boolean nomeEncontrado = listaDeNomes.contains(nomeParaBuscar);
System.out.println("Nome " + nomeParaBuscar + " encontrado na lista? " +
nomeEncontrado);

// Usando findAny
Optional<Integer> qualquerNumero = numeros.stream()
    .findAny();
System.out.println("Qualquer número encontrado na lista: " + qualquerNumero);

// Usando findFirst
Optional<Integer> primeiroNumero = numeros.stream()
    .findFirst();
System.out.println("Primeiro número encontrado na lista: " + primeiroNumero);

// Encontrando o primeiro número par
Optional<Integer> primeiroPar = numeros.stream()
    .filter(numero -> numero % 2 == 0)
    .findFirst();
System.out.println("Primeiro número par encontrado: " + primeiroPar);
```


Utilizando o Map

- O que é map()?
 - map() é uma operação intermediária em Streams que aplica uma função a cada elemento de uma coleção, transformando-os em um novo valor ou tipo;
 - O resultado é um novo Stream com os elementos transformados, sem modificar a coleção original;
- Quando usar map()?
 - Use map() quando você precisar transformar os elementos de uma coleção para outra forma ou estrutura;
 - Exemplo: Converter uma lista de números em seus quadrados, ou transformar objetos complexos em um de seus atributos;
- Sintaxe
 - stream.map(Function): Aplica a função especificada a cada elemento do Stream;

Utilizando o map

```
// 8- Map com Stream
// Modificam a coleção original
List<Integer> quadrados = numeros.stream()
    .map(numero -> numero * numero) // Multiplica cada número por ele mesmo
    .collect(Collectors.toList());
System.out.println("Lista original de números: " + numeros);
System.out.println("Quadrados dos números: " + quadrados);

List<String> nomesMaiusculos = nomes.stream()
    .map(String::toUpperCase) // Converte cada nome para maiúsculas
    .collect(Collectors.toList());
System.out.println("Nomes originais: " + nomes);
System.out.println("Nomes em maiúsculas: " + nomesMaiusculos);
```

Outros métodos de modificação de lista

```
// 9 - Modificação de lista

numeros.add(e:50); // Adiciona um novo número à lista

numeros.remove(index:2); // Remove o número no índice 2 (que era 20)
numeros.remove(Integer.valueOf(i:30)); // Remove o número 30 da lista
numeros.removeIf(numero -> numero > 22); // Remove todos os números maiores que 22

numeros.replaceAll(numero -> numero * 3); // Multiplica todos os números por 3
```

Utilizando o Reduce

- O que é `reduce()`?
 - `reduce()` é uma operação final em Streams que reduz os elementos de um Stream a um único valor;
 - É usado para agregar os elementos de uma coleção de maneira cumulativa, como somar, multiplicar, ou concatenar;
- Quando usar `reduce()`?
 - Quando você precisa combinar ou agregar os elementos de uma coleção em um único resultado;
 - Exemplo: Somar todos os números em uma lista, calcular o produto de números, ou concatenar strings;
- Sintaxe
 - `stream.reduce(identity, BinaryOperator)`: Reduz os elementos aplicando a operação fornecida;
 - **identity**: Valor inicial da operação de redução;
 - **BinaryOperator**: Função que especifica como dois elementos devem ser combinados;

Utilizando o reduce

```
// 10 - Reduce
// Soma de todos os números da lista
int somaTotal = numeros.stream()
    .reduce(identity:0, Integer::sum); // Começa com 0 e soma todos os números
System.out.println("Soma total dos números: " + somaTotal);

// Concatenação de variáveis de texto
String frase = nomes.stream()
    .reduce(identity:"", (acumulador, nome) -> acumulador + " " + nome); //
    Concatena todos os nomes com um espaço
System.out.println("Frase formada pelos nomes: " + frase.trim()); // trim() para
    remover espaços extras no início e no final

// Versão mais moderna
/*String frase = nomes.stream()
    .collect(Collectors.joining(" "));
*/
```

Ordenação Personalizada com Comparator

- O que é Comparator?
 - A interface Comparator é usada para definir uma ordenação personalizada de objetos;
 - Permite criar comparações complexas entre objetos sem modificar a classe original;
- Vantagens do Comparator
 - Flexibilidade para criar múltiplas ordenações para o mesmo tipo de objeto;
 - Facilita a ordenação por múltiplos atributos, como ordenar por nome e, em seguida, por idade;
- Métodos principais
 - **compare(T o1, T o2)**: Compara dois objetos e retorna um valor negativo, zero, ou positivo dependendo da ordem;
 - **thenComparing()**: Permite criar uma ordenação aninhada, aplicando múltiplos critérios de comparação;



Ordenação personalizada com Comparator

```
// 11 - Ordenação com Comparator
List<Pessoa> pessoas = new ArrayList<>();
pessoas.add(new Pessoa(nome:"Carlos", idade:30));
pessoas.add(new Pessoa(nome:"Ana", idade:25));
pessoas.add(new Pessoa(nome:"Ana", idade:18));
pessoas.add(new Pessoa(nome:"Beltrano", idade:40));
pessoas.add(new Pessoa(nome:"Fulano", idade:20));

// Ordenando pelo nome
pessoas.sort(Comparator.comparing(Pessoa::getNome));

for (Pessoa pessoa : pessoas) {
    System.out.println("Pessoa ordenada por nome: " + pessoa);
}

// Ordenando pelo nome e idade
pessoas.sort(Comparator.comparing(Pessoa::getNome)
    .thenComparing(Pessoa::getIdade));

for (Pessoa pessoa : pessoas) {
    System.out.println("Pessoa ordenada por nome e idade: " + pessoa);
}
```

Podemos usar infinitos '.thenComparing' para chegar no nosso resultado ideal, a média de usos é 1 ou 2!

Uso avançado de Streams

- O que é flatMap()?
 - flatMap() é usado para transformar coleções de coleções em uma única coleção, "achutando" o resultado;
 - Permite processar estruturas aninhadas, como listas dentro de listas;
- Pipeline de Streams Avançado
 - Combinar múltiplas operações intermediárias como:
 - filter(): Filtra os elementos com base em uma condição;
 - map(): Transforma cada elemento;
 - sorted(): Ordena os elementos;
- Operações são encadeadas em um pipeline de Stream, que só é executado quando uma operação terminal é chamada;

Uso avançado de streams

```
// 12 - Uso avançado de Streams
// Listas dentro de listas
List<List<String>> listaDeListas = Arrays.asList(
    Arrays.asList(...a:"Maça", "Banana", "Laranja"),
    Arrays.asList(...a:"Banana", "Mamão", "Morango"),
    Arrays.asList(...a:"Laranja", "Abacaxi", "Uva")
);

// Usando flatMap para achatar a lista de listas em uma única lista
List<String> frutas = listaDeListas.stream()
    .flatMap(List::stream) // Achata a lista de listas em uma única lista
    .collect(Collectors.toList());

System.out.println("Lista de frutas achatada: " + frutas);

// Pipeline
List<Integer> resultado = numeros.stream()
    .filter(numero -> numero > 10) // Filtra números maiores que 10
    .map(numero -> numero * 2) // Multiplica cada número por 2
    .sorted() // Ordena os números
    .collect(Collectors.toList()); // Coleta o resultado em uma lista

System.out.println("Resultado do pipeline: " + resultado);
```

Pipeline é o ato de encadear as essas operações intermediárias!

Manipulação de Coleções com Collectors

- O que é Collectors?
 - Collectors é uma classe utilitária que oferece várias operações de redução em coleções;
 - Permite agregar, agrupar, contar, e transformar coleções de forma eficiente;
- Operações Principais com Collectors
 - **collect()**: Operação final que converte o resultado de um Stream em uma coleção ou outro tipo de dado;
 - **Agrupamento (groupingBy)**: Agrupa elementos com base em uma chave;
 - **Particionamento (partitioningBy)**: Divide a coleção em duas partes com base em uma condição;
 - **Agregação (counting(), summingInt())**: Conta ou soma os elementos da coleção;



Manipulação de coleções com collectors

```
// 13 - Collectors
List<Produto> produtos = Arrays.asList(
    new Produto(nome:"Maçã", categoria:"Fruta"),
    new Produto(nome:"Rádio", categoria:"Eletrônico"),
    new Produto(nome:"Batata", categoria:"Legume"),
    new Produto(nome:"Telefone", categoria:"Eletrônico")
);

// Agrupando produtos por categoria
Map<String, List<Produto>> produtosPorCategoria = produtos.stream()
    .collect(Collectors.groupingBy(p -> p.categoria));

System.out.println("Produtos não agrupados: " + produtos);
System.out.println("Produtos agrupados por categoria: " +
    produtosPorCategoria);

// Particionando os produtos
Map<Boolean, List<Produto>> eletronicosEnaoEletronicos = produtos.stream()
    .collect(Collectors.partitioningBy(p -> p.categoria.equals
        (anObject:"Eletrônico")));
System.out.println("Produtos eletrônicos e não eletrônicos: " +
    eletronicosEnaoEletronicos);

// Contando quantos produtos existem
Long totalProdutos = produtos.stream()
    .collect(Collectors.counting());
System.out.println("Total de produtos: " + totalProdutos);
```


O que são Expressões Regulares?

- **Expressões Regulares (Regex)** são padrões utilizados para procurar e manipular texto;
- Elas permitem verificar se um padrão de caracteres aparece em uma *string*, como números, letras e símbolos;
- São amplamente usadas para validação, extração e substituição de texto em diversas linguagens de programação;
- A regex usa metacaracteres e sequências especiais para definir padrões complexos, como `.` (qualquer caractere), `*` (zero ou mais ocorrências) e `[]` (conjunto de caracteres);
- Em Java, expressões regulares são manipuladas com as classes `Pattern` e `Matcher` do pacote `java.util.regex`;
- Regex é muito utilizada para validações como verificar e-mails, números de telefone, senhas, e formatos de datas;
- Elas podem variar de simples a extremamente complexas, dependendo do padrão a ser definido.

O que são expressões regulares (regex)?

Sintaxe básica das Regex

- Metacaracteres são símbolos especiais que representam padrões complexos nas expressões regulares:
 - `.`: Representa qualquer caractere, exceto nova linha;
 - `*`: Indica zero ou mais ocorrências do caractere anterior;
 - `+`: Indica uma ou mais ocorrências do caractere anterior;
 - `?`: Indica zero ou uma ocorrência do caractere anterior (opcional);
 - `[]`: Define um conjunto de caracteres. Ex: `[abc]` corresponde a "a", "b", ou "c";
 - `^`: Representa o início da string;
 - `$`: Representa o fim da string;
- Diferença entre caracteres literais e metacaracteres:
 - **Caractere literal**: Representa o próprio caractere, como "a", "1", ou "@";
 - **Metacaractere**: Tem um significado especial na regex, como `.` ou `*`. Para usá-los como literais, é preciso escapá-los com uma barra invertida (`\`). Ex: `\.` corresponde a um ponto literal;

Sintaxe básica das regex

Quantificadores

- Quantificadores definem quantas vezes um padrão pode ocorrer:
- ***** (zero ou mais): O caractere anterior pode aparecer zero ou mais vezes. Exemplo: `a*` corresponde a "", "a", "aa", "aaa", etc.
- **+** (uma ou mais): O caractere anterior deve aparecer pelo menos uma vez. Exemplo: `a+` corresponde a "a", "aa", "aaa", mas não "".
- **?** (zero ou uma): O caractere anterior pode aparecer zero ou uma vez. Exemplo: `a?` corresponde a "", "a".
- **{n}**: O caractere anterior deve aparecer exatamente n vezes. Exemplo: `a{3}` corresponde a "aaa".
- **{n,}**: O caractere anterior deve aparecer pelo menos n vezes. Exemplo: `a{2,}` corresponde a "aa", "aaa", etc.
- **{n,m}**: O caractere anterior deve aparecer entre n e m vezes. Exemplo: `a{2,4}` corresponde a "aa", "aaa", ou "aaaa".

Quantificadores

```
// 1 - Quantificadores em expressões regulares (Regex)
String regex = "a+"; // O quantificador '+' indica que o caractere 'a' deve
aparecer uma ou mais vezes
String texto = "b aaab aa ba";

Pattern pattern = Pattern.compile(regex); // Compila a expressão regular
Matcher matcher = pattern.matcher(texto); // Cria um objeto Matcher para
buscar no texto

// Encontrar todas as ocorrências
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'");
}
```


Âncoras e Fronteiras

- Âncoras são utilizadas para definir posições dentro de uma string:
 - `^`: Indica o início de uma string;
Exemplo: `^a` encontra strings que começam com "a";
 - `$`: Indica o fim de uma string;
Exemplo: `a$` encontra strings que terminam com "a";
- Fronteiras de Palavras e Não-Fronteiras:
 - `\b`: Representa uma fronteira de palavra (entre um caractere de palavra e um não caractere de palavra);
Exemplo: `\bword\b` encontra "word" isolada, mas não "password";
 - `\B`: Representa uma não fronteira de palavra (quando não há uma fronteira de palavra);
Exemplo: `\Bword` encontra "password", mas não "word" isolada;

Âncoras e Fronteiras – ATENÇÃO!!!!

```
// 2 - Âncoras e fronteiras
// O caractere '^' indica o início da string e '$' indica o final da string
regex = "^c"; // A string deve começar com 'c'
texto = "casa";

pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);

System.out.println(x: "\n0corrências de ^c no texto:");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'");
}

regex = "c$"; // A string deve terminar com 'c'
texto = "casa";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x: "\n0corrências de c$ no texto:");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'");
}

// Fronteiras de palavras com b
regex = "\\bpalavra\\b"; // A string deve conter 'palavra' como uma palavra isolada
texto = "Esta é uma palavra isolada, mas não é uma palavraisolada.";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x: "\n0corrências de \\bpalavra\\b no texto:");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'");
}

// Fronteiras de palavras com B
regex = "\\Bpalavra\\B"; // A string não deve conter 'palavra' como uma palavra isolada
texto = "Esta é uma palavra isolada, mas não é uma palavraisolada.";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x: "\n0corrências de \\Bpalavra\\B no texto:");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'");
}
```

Grupos e Captura

- **Grupos e Captura:**
 - Usam parênteses () para agrupar e capturar partes específicas de uma expressão regular;
 - Cada grupo capturado pode ser reutilizado ou referenciado;
- **Backreferences:**
 - Permite reutilizar um grupo capturado dentro da própria expressão com \1, \2, etc;
 - Útil para encontrar padrões repetidos ou substituir partes de uma string;
- **Principais Usos de Grupos:**
 - **Agrupamento:** Controla a ordem de aplicação dos operadores, como * ou +;
 - **Captura:** Extrai e reutiliza subseqüências correspondentes na expressão;

Grupos e captura

```
// 3- Grupos e capturas
regex = "(\\d{2})-(\\d{2})-(\\d{4})"; // Captura datas no formato dd-mm-aaaa
texto = "A data de hoje é 09-07-2025 e a data de ontem é 08-07-2025.";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x:"\nDatas encontradas no texto:");
while (matcher.find()) {
    System.out.println("Dia: '" + matcher.group(group:1) + "'");
    System.out.println("Mês: '" + matcher.group(group:2) + "'");
    System.out.println("Ano: '" + matcher.group(group:3) + "'");
    System.out.println("Todos os grupos: '" + matcher.group(group:0) +
        "'"); // O grupo 0 é a correspondência completa
    // Ao contrário dos arrays, os grupos são numerados a partir de 1
}

// Backreferences
String textoSubstituido = texto.replaceAll(regex,
replacement:"$3-$2-$1"); // Inverte a data para o formato aaaa-mm-dd
System.out.println(x:"\nTexto com datas invertidas:");
System.out.println(textoSubstituido);
```

Avançando em Pattern e Matcher

- Principais Métodos da Classe Pattern:
 - `compile(String regex)`: Compila uma expressão regular em um padrão;
 - `matches(String regex, CharSequence input)`: Verifica se a string corresponde totalmente ao padrão;
 - `split(String input)`: Divide uma string com base em um padrão;
- Principais Métodos da Classe Matcher:
 - `find()`: Busca subsequências que correspondem ao padrão;
 - `group()`: Retorna o valor da última correspondência encontrada;
 - `replaceAll(String replacement)`: Substitui todas as correspondências em uma string;
 - `matches()`: Verifica se a string corresponde totalmente ao padrão;
 - `start()` e `end()`: Retorna o índice de início e fim da correspondência encontrada;

Avançando em pattern e matcher

```
// 4 - Avançando em pattern e matcher

// Correspondência parcial
regex = "\\d{3}"; // Busca por três dígitos consecutivos
texto = "123 456 789 10 11";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x:"\nOcorrências de três dígitos consecutivos no texto usando lookingAt :");
if (matcher.lookingAt()) { // Verifica se a correspondência começa no início do texto
    System.out.println("'" + matcher.group() + "'");
} else {
    System.out.println(x:"Nenhuma correspondência encontrada.");
}

// Contando grupos com groupCount
regex = "(\\d{3})-(\\d{3})-(\\d{3})";
texto = "123-456-721";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x:"\nNúmero de grupos na expressão regular: ");
if (matcher.matches()) {
    System.out.println("Número de grupos: " + matcher.groupCount()); // Exibe o número de grupos capturados
    System.out.println("Grupo 1: '" + matcher.group(group:1) + "'");
    System.out.println("Grupo 2: '" + matcher.group(group:2) + "'");
    System.out.println("Grupo 3: '" + matcher.group(group:3) + "'");
}
```

```

// Start e end para obter correspondências na string
regex = "\\d{3}"; // Busca por três dígitos consecutivos
texto = "O código é 123 e o número é 456.";
pattern = Pattern.compile(regex);
matcher = pattern.matcher(texto);
System.out.println(x:"\nPosições de correspondência de três dígitos
consecutivos no texto:");
while (matcher.find()) {
    System.out.println("Correspondência: '" + matcher.group() + "'");
    System.out.println("Início: " + matcher.start()); // Posição inicial da
correspondência
    System.out.println("Fim: " + matcher.end()); // Posição final da
correspondência
}

// Quote para tratar caracteres especiais
String literalRegex = Pattern.quote(s:"1 + 1 = 2"); // Trata a string
como um literal, escapando caracteres especiais
texto = "1 + 1 = 2 é uma expressão matemática.";
pattern = Pattern.compile(literalRegex);
matcher = pattern.matcher(texto);
if (matcher.find()) {
    System.out.println("\nOcorrência de expressão literal no texto: '" +
matcher.group() + "'");
} else {
    System.out.println(x:"Nenhuma ocorrência encontrada.");
}

```

Expressões Regulares Avançadas

- **Lookaheads:**
 - Verifica se há um padrão à frente sem consumi-lo na correspondência;
- **Sintaxe:**
 - Lookahead positivo: `(?=...)`
 - Lookahead negativo: `(?!...)`
 - `\d+(?=\$)`: Encontra números seguidos de um símbolo de dólar (\$), mas não inclui o dólar na correspondência.
- **Lookbehinds:**
 - Verifica se há um padrão antes sem consumi-lo na correspondência;
- **Sintaxe:**
 - Lookbehind positivo: `(?<=...)`
 - Lookbehind negativo: `(?<!\...)`
- **Exemplo:**
 - `(?<=#)\w+`: Encontra palavras que seguem imediatamente um símbolo de #, mas não inclui o # na correspondência;

Expressões regulares avançadas

```
// 5 - Expressões regulares avançadas
// lookahead positivo
String regexLookAhead = "\\d+(?=\\$)"; // Busca por dígitos seguidos de um
cifração, mas não inclui o cifração na correspondência
texto = "O preço é 100$ e o desconto é 20$.";
pattern = Pattern.compile(regexLookAhead);
matcher = pattern.matcher(texto);
System.out.println(x:"\n0corrências de dígitos seguidos de cifração (lookahead
positivo):");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'"); // Exibe os dígitos
    encontrados
}

// Lookbehind positivo
String regexLookBehind = "(?<=\\$)\\d+"; // Busca por dígitos precedidos por
um cifração, mas não inclui o cifração na correspondência
texto = "O preço é $100 e o desconto é $20.";
pattern = Pattern.compile(regexLookBehind);
matcher = pattern.matcher(texto);
System.out.println(x:"\n0corrências de dígitos precedidos por cifração
(lookbehind positivo):");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'"); // Exibe os dígitos
    encontrados
}
```

```

// Lookahead negativo
// Encontrar palavras que não terminam com ponto final
// [a-z] != 'é'
// p{L} == inclui letras acentuadas, unicode no geral
texto = "Essa é uma frase. e esta é outra";
String regexLookAheadNegativo = "\\b[\\p{L}]+\\b(?!\\.)"; // Busca por
palavras que não terminam com ponto final
pattern = Pattern.compile(regexLookAheadNegativo);
matcher = pattern.matcher(texto);
System.out.println(x:"\nOcorrências de palavras que não terminam com ponto
final (lookahead negativo:");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'"); // Exibe as palavras
    encontradas
}

// Lookbehind negativo
// Encontrar palavras que não são precedidas por um #
String regexLookBehindNegativo = "(?<#)\\b\\w+\\b"; // Busca por palavras
que não são precedidas por um '#'
texto = "#palavra1 palavra2 #palavra3";
pattern = Pattern.compile(regexLookBehindNegativo);
matcher = pattern.matcher(texto);
System.out.println(x:"\nOcorrências de palavras que não são precedidas por
'#':");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'"); // Exibe as palavras
    encontradas
}

```


Regex para Manipulação de Datas e Horários

- Validação de Datas com Regex:
 - Formato DD/MM/YYYY:
 - `^([0-2][0-9]|3[01])/(0[1-9]|1[0-2])/([0-9]{4})$`
 - Valida dias de 01 a 31, meses de 01 a 12, e anos de 4 dígitos;
- Validação de Horários com Regex:
 - Formato HH:MM:SS
 - `^([01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$`
 - Valida horas de 00 a 23, minutos de 00 a 59, e segundos de 00 a 59;



Regex para manipulação de datas e horários

```
public static boolean validarRegex(String regex, String texto) {
    Pattern pattern = Pattern.compile(regex);
    Matcher matcher = pattern.matcher(texto);
    return matcher.matches(); // Retorna true se a string corresponder à
    expressão regular
}
```

```
// 6 - Validação de data e horario
String regexData = "^([0-2][0-9]|3[01])-(0[1-9]|1[0-2])-([0-9]{4})$"; //
Valida datas no formato dd-mm-aaaa
String[] datas = {
    "01-01-2020", "31-12-2020", "29-02-2020", // Válidas
    "31-04-2020", "30-02-2020", "32-01-2020" // Inválidas
};
System.out.println(x:"\nValidação de datas:");
for (String data : datas) {
    if (validarRegex(regexData, data)) {
        System.out.println(data + " é uma data válida.");
    } else {
        System.out.println(data + " não é uma data válida.");
    }
}

// Validação de horário
String regexHorario = "^([01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$"; // Valida
horários no formato HH:mm
String[] horarios = {
    "12:30:45", "23:59:59", "00:00:00", // Válidos
    "24:00:00", "12:60:00", "12:30:61" // Inválidos
};
System.out.println(x:"\nValidação de horários:");
for (String horario : horarios) {
    if (validarRegex(regexHorario, horario)) {
        System.out.println(horario + " é um horário válido.");
    } else {
        System.out.println(horario + " não é um horário válido.");
    }
}
```


Regex com Flags (Opções Especiais)

- O que são Flags:
 - Flags são opções especiais que alteram o comportamento padrão das expressões regulares em Java;
 - São usadas para modificar a forma como o regex é processado, permitindo correspondências mais flexíveis;
- Principais Flags em Java:
 - **Pattern.CASE_INSENSITIVE**: Ignora diferenças entre maiúsculas e minúsculas;
 - **Pattern.MULTILINE**: Trata o início (^) e fim (\$) como correspondências por linha em vez de toda a string;
 - **Pattern.DOTALL**: Permite que o caractere . corresponda a todas as linhas, inclusive quebras de linha;
 - **Pattern.COMMENTS**: Permite adicionar comentários e ignorar espaços em branco no regex, tornando-o mais legível;
- Sintaxe para Usar Flags:
 - **Pattern.compile(String regex, int flags)**: Compila uma expressão regular com a flag fornecida;

Regex com flags (Opções Especiais)

```
// 7 - Flags

// Case insensitive - ignora diferenças entre maiúsculas e minúsculas
regex = "java";
texto = "Java é uma linguagem de programação. JAVA é popular.";
pattern = Pattern.compile(regex, Pattern.CASE_INSENSITIVE); // Cria o padrão
com a flag CASE_INSENSITIVE
matcher = pattern.matcher(texto);
System.out.println(x:"\nOcorrências de 'java' (case insensitive):");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'"); // Exibe as ocorrências
    encontradas
}

// Multiline - trata cada linha como uma string separada
regex = "^\\d+"; // Busca por números no início de cada linha
texto = "123\n456\n789\n10\n11"; // Texto com várias linhas
pattern = Pattern.compile(regex, Pattern.MULTILINE); // Cria o padrão com a
flag MULTILINE
matcher = pattern.matcher(texto);
System.out.println(x:"\nOcorrências de números no início de cada linha
(multiline):");
while (matcher.find()) {
    System.out.println("'" + matcher.group() + "'"); // Exibe os números
    encontrados
}
```

O que são Annotations?

- Annotations são metadados que fornecem informações adicionais ao compilador e à máquina virtual;
- Elas não afetam diretamente a execução do código, mas podem ser usadas para modificar comportamentos durante a compilação, tempo de execução ou até mesmo em frameworks;
- **Funções das Annotations:**
 - Sinalizam instruções especiais para o compilador;
 - Auxiliam no tratamento de erros e na documentação do código;
 - Automatizam tarefas repetitivas por meio de ferramentas de processamento;
- **Tipos de Annotations:**
 - Predefinidas: Como `@Override`, `@Deprecated`, e `@SuppressWarnings`;
 - Customizadas: Desenvolvedores podem criar suas próprias anotações para atender a necessidades específicas;

O que são annotations (anotações) ATENÇÃO!!!!

Anotações Predefinidas em Java

- **@Override:**
 - Indica que um método está sobrescrevendo um método da superclasse;
 - Fornece verificação em tempo de compilação para garantir que a sobrescrita seja válida;
- **@Deprecated:**
 - Marca métodos, classes ou variáveis que estão obsoletos e podem ser removidos em versões futuras;
 - Gera um aviso no compilador ao usar elementos marcados com essa anotação;
- **@SuppressWarnings:**
 - Instrui o compilador a suprimir certos tipos de avisos que ele geraria;
 - Pode ser configurado para ignorar diferentes tipos de avisos, como `unchecked`, `deprecation`, etc;
- **Por que usar?**
 - `@Override`: Garante que a sobrescrita seja correta;
 - `@Deprecated`: Sinaliza elementos obsoletos, orientando o desenvolvedor a evitar seu uso;
 - `@SuppressWarnings`: Permite limpar o código de avisos desnecessários, mantendo o foco em alertas impor

Annotations (anotações) predefinidas em java

```

public class Animal {
    public void emititirSom() {
        System.out.println(x:"O animal faz um som.");
    }

    @Deprecated
    public void mover() {
        System.out.println(x:"O animal está se movendo.");
    }
}

```

```

public class Cachorro extends Animal {

    @Override
    public void emititirSom() {
        System.out.println(x:"O cachorro late.");
    }

    @SuppressWarnings("deprecation") // Suprime o aviso de uso de método deprecated
    // Os warnings não param a compilação, mas é uma boa prática evitá-los
    public void testeMover(){
        mover(); // Chama o método deprecated
    }
}

```

```

// 1 - Annotations (anotações) pré-definidas
Cachorro cachorro = new Cachorro();
cachorro.emititirSom(); // Chama o método sobrescrito
cachorro.testeMover(); // Chama o método deprecated, mas suprimido o aviso
cachorro.mover(); // Chama o método sobrescrito

```

Criando Annotations

- O que são Anotações Customizadas?
 - São anotações definidas pelo desenvolvedor para fornecer informações ou comportamentos específicos no código;
 - Podem ser usadas em classes, métodos, parâmetros, variáveis e campos;
- Elementos de uma Anotação Customizada:
 - `@Target`: Define onde a anotação pode ser aplicada (classe, método, etc.);
 - `@Retention`: Define quando a anotação está disponível (em tempo de compilação, execução ou apenas no código fonte);
 - `@Documented`: Indica que a anotação deve ser incluída na documentação;
 - `@Inherited`: Permite que a anotação seja herdada por subclasses;
- Sintaxe de uma Anotação Customizada:
 - Anotações são definidas com `@interface`;
 - Elas podem conter parâmetros com valores padrão ou obrigatórios;

Criando annotations

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME) // Define que a anotação estará disponível em
tempo de execução
@Target(ElementType.METHOD) // Define que a anotação pode ser aplicada a métodos
@interface Executar {
    int vezes() default 1; // Define um atributo com valor padrão
}
```

```
public class Exemplo {

    @Executar(vezes = 3) // Anotação personalizada aplicada ao método
    public void mostrarMensagem(){
        System.out.println(x:"Executando o método mostrarMensagem");
    }
}
```

```

// 2 - Annotations (anotações) personalizadas
Exemplo exemplo = new Exemplo();

// Anotações normalmente precisam de try-catch para serem acessadas
for(var metodo : exemplo.getClass().getDeclaredMethods()) { // Itera sobre os
métodos da classe Exemplo
    System.out.println(metodo);
    if (metodo.isAnnotationPresent(annotationClass:Executar.class)) { //
Verifica se o método possui a anotação @Executar
        Executar anotacao = metodo.getAnnotation(annotationClass:Executar.
class); // Obtém a anotação @Executar do método
        for (int i = 0; i < anotacao.vezes(); i++) { // Executa o método o
número de vezes especificado na anotação
            // Imprime o nome do método e a quantidade de vezes que será
executado
            System.out.println("Executando método: " + metodo.getName() + " -
" + (i + 1) + " vez(es)");
            try { // Invoca o método anotado
                metodo.invoke(exemplo); // Invoca o método anotado
            } catch (Exception e) {
                e.printStackTrace();
                // Tudo isso para executar o método mostrarMensagem 3 vezes :0
            }
        }
    }
}
}

```

Validação de Campos com Annotation

- Anotação Customizada para Validação de Campos:
 - **Objetivo:** Criar uma anotação customizada para validar se um campo de uma classe atende a uma determinada regra (ex.: não pode ser nulo ou vazio);
- Elementos Importantes:
 - **@Target(ElementType.FIELD):** Define que a anotação pode ser aplicada a campos de uma classe;
 - **@Retention(RetentionPolicy.RUNTIME):** A anotação está disponível em tempo de execução; permitindo sua verificação por meio de reflection;
 - **@interface:** Define a anotação customizada com parâmetros específicos;



Validação de campos com annotations

```
@Retention(RetentionPolicy.RUNTIME) // Anotação disponível em tempo de execução
@Target(ElementType.FIELD) // Anotação pode ser aplicada a campos
@interface NotEmpty {
    String message() default "O campo não pode ser vazio ou nulo";
}
```

```
@NotEmpty(message = "Nome não pode ser vazio")
private String nome;

@NotEmpty(message = "Email não pode ser vazio")
private String email;

public Usuario(String nome, String email) {
    this.nome = nome;
    this.email = email;
}
```

```
// 3 - Annotations (anotações) de validação de campos
Usuario usuario = new Usuario(nome:"Carlos", email:"");
validarCampos(usuario); // Chama o método para validar os campos do usuário
}

// Método para validar os campos do usuário
public static void validarCampos(Object objeto) throws IllegalArgumentException {
    // Temos de pegar a classe, pegar os campos e verificar se eles possuem a
    // anotação @NotEmpty
    Class<?> classe = objeto.getClass(); // Obtém a classe do objeto
    for(Field campo : classe.getDeclaredFields()) {
        if (campo.isAnnotationPresent(annotationClass:NotEmpty.class)) { //
            // Verifica se o campo possui a anotação @NotEmpty
            NotEmpty anotacao = campo.getAnnotation(annotationClass:NotEmpty.
                class); // Obtém a anotação @NotEmpty do campo
            campo.setAccessible(flag:true); // Torna o campo acessível, mesmo que
            // seja privado
            try {
                String valor = (String) campo.get(objeto); // Obtém o valor do
                // campo do objeto
                if (valor == null || valor.trim().isEmpty()) { // Verifica se o
                // valor é nulo ou vazio
                    throw new IllegalArgumentException(anotacao.message()); //
                    // Lança exceção com a mensagem da anotação
                }
            } catch (IllegalAccessException e) {
                e.printStackTrace(); // Trata exceção caso não consiga acessar o
                // campo
            }
        }
    }
}
```


Processadores de Annotations

- O que são Processadores de Anotações?
 - São ferramentas que permitem analisar e processar anotações customizadas em tempo de compilação ou execução;
 - Os processadores são usados para automatizar tarefas com base nas informações fornecidas pelas anotações, como gerar código, realizar validações e otimizar o comportamento do programa;
- Tipos de Processamento:
 - Em Tempo de Execução (Runtime): Usando Reflection para verificar e processar anotações;
 - Em Tempo de Compilação (Compile-time): Usando Annotation Processors (processadores de anotação) para analisar e manipular anotações durante a compilação;



Processadores de annotations

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Log {

}
```

```
import java.io.FileWriter;
import java.lang.reflect.Method;

public class LogProcessor {

    public static void processarLog(Object objeto) throws Exception {

        Class<?> classe = objeto.getClass();

        String currentDir = System.getProperty(key:"user.dir") + "\\secao26\\";

        for(Method metodo : classe.getDeclaredMethods()){
            if(metodo.isAnnotationPresent(annotationClass:Log.class)) {
                metodo.setAccessible(flag:true);
                Object retorno = metodo.invoke(objeto);

                String nomeArquivo = metodo.getName() + ".txt";
                String caminhoArquivo = currentDir + nomeArquivo;

                try (FileWriter writer = new FileWriter(caminhoArquivo, append:true))
                {
                    writer.write("Método: " + metodo.getName() + "\n");
                    writer.write("Retorno: " + retorno + "\n");
                    writer.write(str:"=====\\n");
                } catch (Exception e) {
                    System.err.println("Erro ao escrever no arquivo: " + e.getMessage());
                }
            }
        }
    }
}
```

```

public class Servico {
    @Log
    public void executarTarefa(){
        System.out.println(x:"Tarefa executada com sucesso!");
    }

    @Log
    public void processarDados(){
        System.out.println(x:"Dados processados com sucesso!");
    }
}

```

```

// 4 - Annotations (anotações) de log com processador
Servico servico = new Servico();
try {
    LogProcessor.processarLog(servico); // Processa o log dos métodos
    anotados com @Log
    servico.executarTarefa(); // Executa o método anotado com @Log
    servico.processarDados(); // Executa outro método anotado com @Log
} catch (Exception e) {
    e.printStackTrace(); // Trata exceção caso ocorra algum erro no
    processamento do log
}

```

secao26	
Animal.java	2, U
Anotacoes.java	7, U
Cachorro.java	2, U
Executar.java	2, U
executarTarefa.txt	U
Exemplo.java	2, U
Log.java	2, U
LogProcessor.java	2, U
NotEmpty.java	2, U
processarDados.txt	U
Servico.java	2, U
Usuario.java	6, U

Executando o método mostrarTarefa,

```

Tarefa executada com sucesso!
Dados processados com sucesso!
Tarefa executada com sucesso!
Dados processados com sucesso!

```

O que é JavaFX?

- JavaFX é uma biblioteca de software para criar interfaces gráficas de usuário (GUIs) em Java;
- Substitui o antigo Swing como principal biblioteca gráfica no Java;
- Permite o desenvolvimento de aplicações desktop e RIAs (Rich Internet Applications) com Java;
- Oferece suporte a diversos tipos de componentes gráficos: botões, tabelas, gráficos, etc;
- Utiliza uma abordagem baseada em cena e nó (Scene Graph), onde os elementos gráficos são organizados em uma estrutura de árvore;
- Possui integração com CSS para estilização e com FXML, um formato XML para descrever a interface;
- Suporta animações, multimídia e a criação de gráficos 2D e 3D;
- Funciona em diferentes plataformas: Windows, Mac e Linux;
- Ideal para o desenvolvimento de aplicações, como ferramentas de visualização, jogos simples e softwares empresariais;

O que é JavaFx?

Como instalar e executar o JavaFX

- Documentação oficial de instalação: [clique aqui](#).
- Primeiramente precisamos baixar o JavaFX no site oficial, e extrair os arquivos para uma pasta;
- Depois vamos abrir o VS Code e criar uma pasta para a seção;
- Acesse a opção Help -> Show All Commands -> Create Java Project, e selecione No build tools;
- Selecione a pasta pai (pasta da seção), e nomeie o projeto, por exemplo: HelloWorldFX;
- Agora em Java Projects, podemos adicionar os arquivos de JavaFX em Referenced Libraries;
- Delete o arquivo criado automaticamente com o nome de App.java;
- Crie uma pasta com o nome do projeto em src, exemplo: helloworldfx;
- Adicione os arquivos Main, fxml e Controller, que são fornecidos pela documentação;
- Vá até a aba Run and Debug do VS Code, e selecione a opção Create json file;
- Configure o valor de vmArgs no launch.json, conforme a documentação indica;
- Adapte o arquivo Main e .fxml para o nome que foi criado, pois está com o nome da documentação;
- Altere a mensagem do setText e execute o arquivo! =)

Como instalar e executar o JavaFX

Link - [AQUI!](#)

Versão: JavaFX and Visual Studio Code / Non modular from IDE

Way too boring tbh

A Classe Application e o Método Start

- **Estrutura Básica de um Aplicativo JavaFX**
 - Todo aplicativo JavaFX herda da classe Application;
 - O ciclo de vida do aplicativo começa ao executar o método launch();
- **Como Usar a Classe Application**
 - A classe Application fornece a base para criar a interface gráfica;
 - O método abstrato start() precisa ser implementado para inicializar a interface;
- **O Ciclo de Vida do Método start()**
 - O método start(Stage primaryStage) é chamado após o launch();
 - É responsável por definir o conteúdo principal da janela (Stage);
- **O ciclo de vida do aplicativo JavaFX:**
 - **Iniciação:** O método launch() é chamado;
 - **Inicialização:** O método start() é chamado para configurar a interface;
 - **Execução:** A interface é exibida e interage com o usuário;
 - **Encerramento:** Quando o usuário fecha a janela ou o sistema encerra o aplicativo;

A classe application e o método start

Stage e Scene em JavaFX

- **Stage:** Representa a janela principal da aplicação JavaFX
 - O Stage é a janela que pode ser exibida na tela, contendo o título, o tamanho e o conteúdo;
 - Pode ser fechado, maximizado, minimizado, e várias janelas (stages) podem ser criadas;
- **Scene:** Representa o conteúdo dentro de um Stage
 - A Scene contém os elementos visuais (botões, layouts, textos) que serão exibidos ao usuário;
 - Define o tamanho da área de exibição e pode conter um layout com vários componentes;
- **Relacionamento entre Stage e Scene**
 - O Stage é a "janela", e a Scene é o "conteúdo" dentro dessa janela;
 - Uma Stage pode conter apenas uma Scene de cada vez, mas essa Scene pode ser trocada dinamicamente durante a execução da aplicação;
- Vamos ver na prática!
- Obs: todo novo arquivo deve ter configurado o vmArgs em launch.json, e o nome do arquivo aparece em launch quando executamos ele via Run Java;

Stage e scene em javafx


```

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class StageSceneExample extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception{

        // Criar botão
        Button btn = new Button("Clique aqui");

        // Criar layout e adicionar o botão
        StackPane root = new StackPane();
        root.getChildren().add(btn);

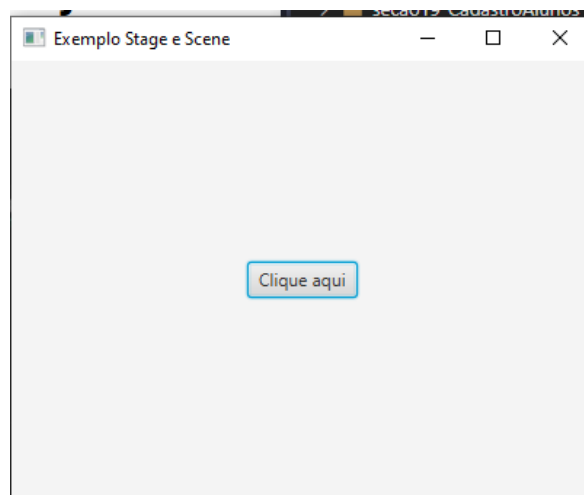
        // Criando uma cena com o layout
        Scene scene = new Scene(root, 400, 300);

        // Configurar o palco (Stage)
        primaryStage.setTitle("Exemplo Stage e Scene");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        launch(args);
    }
}

```

É importante adicionar as configurações de VMargs no launcher do VsCode (meu deus)



Layouts em JavaFX

- **O que são Layouts em JavaFX?**
 - Layouts são gerenciadores responsáveis por organizar a posição e o tamanho dos componentes (botões, labels, etc.) dentro de uma interface;
 - Controlam como os componentes se ajustam ao redimensionamento da janela;
 - Garantem consistência na apresentação da interface, independentemente do tamanho da tela ou dispositivo;
- **Por que usar Layouts?**
 - Evitam a necessidade de posicionar cada componente manualmente;
 - Adaptam os componentes de forma eficiente para diferentes resoluções;
 - Facilitam o design de interfaces complexas de maneira estruturada e escalável;
- **Tipos de layout:**
 - VBox, HBox, BorderPane, GridPane;

Layouts em javafx

VBox e HBox em JavaFX

- **VBox:** Organiza os componentes em uma coluna, de cima para baixo
 - Útil para empilhar elementos como botões, labels e caixas de texto em uma disposição vertical;
- **HBox:** Organiza os componentes em uma linha, da esquerda para a direita
 - Ideal para criar layouts onde os componentes estão lado a lado, como uma barra de ferramentas ou opções;
- **Espaçamento:** Controla o espaço entre os componentes
 - Em VBox, o espaçamento é vertical (entre os elementos empilhados);
 - Em HBox, o espaçamento é horizontal (entre os elementos lado a lado);
- **Alinhamento:** Define como os componentes serão alinhados dentro do contêiner
 - Pode ser definido para alinhar à esquerda, direita, centro ou ajustado ao conteúdo;

Vbox e hbox em javafx


```

public class VBoxHBoxExample extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception{

        // Criar alguns botões vbox
        Button btn1 = new Button("Botão 1");
        Button btn2 = new Button("Botão 2");
        Button btn3 = new Button("Botão 3");

        VBox vbox = new VBox(15); // Espaçamento de 15 pixels
        vbox.getChildren().addAll(btn1, btn2, btn3); // Adiciona os botões ao vbox

        // Criar alguns botões hbox
        Button btn4 = new Button("Botão A");
        Button btn5 = new Button("Botão B");
        Button btn6 = new Button("Botão C");

        HBox hbox = new HBox(25); // Espaçamento de 15 pixels
        hbox.getChildren().addAll(btn4, btn5, btn6);

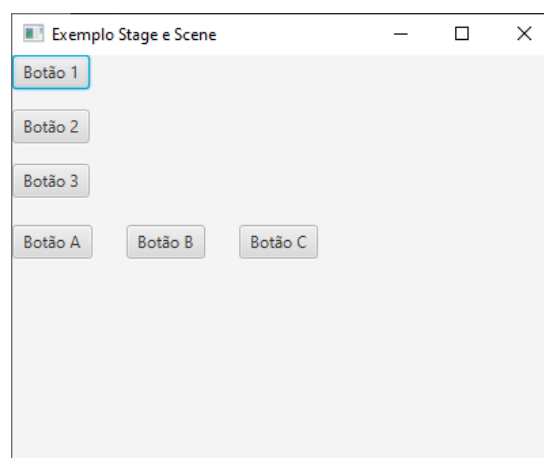
        VBox root = new VBox(20); // Espaçamento de 10 pixels
        root.getChildren().addAll(vbox, hbox); // Adiciona o vbox e o hbox ao root

        // Criando uma cena com o layout
        Scene scene = new Scene(root, 400, 300);

        // Configurar o palco (Stage)
        primaryStage.setTitle("Exemplo Stage e Scene");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        launch(args);
    }
}

```

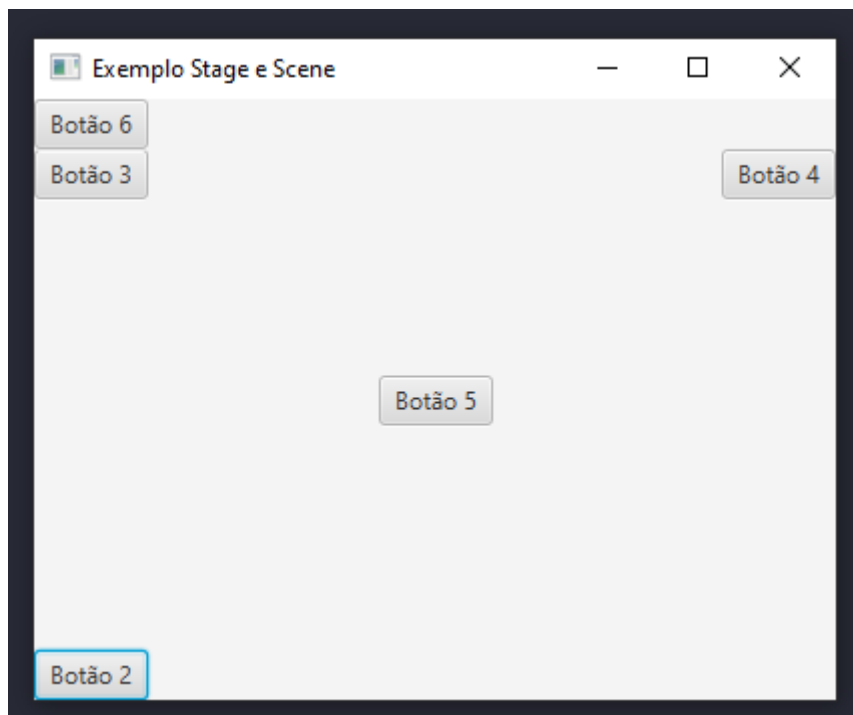


BorderPane em JavaFX

- O BorderPane organiza os componentes em cinco regiões:
 - **Top:** Parte superior, geralmente usada para cabeçalhos ou menus;
 - **Bottom:** Parte inferior, comum para rodapés ou botões de ação;
 - **Left:** Lado esquerdo, pode conter menus ou painéis de navegação;
 - **Right:** Lado direito, usado para informações adicionais ou detalhes;
 - **Center:** Área central, onde fica o conteúdo principal da aplicação;
- Cada região pode conter um único componente ou layout. Esse componente pode ser um botão, texto, ou até mesmo outro layout;
- Os métodos `setTop()`, `setBottom()`, `setLeft()`, `setRight()` e `setCenter()` são usados para definir os componentes em cada região do BorderPane;

Borderpane em javafx

```
public class BorderPaneExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception{  
  
        Button button1 = new Button("Botão 1");  
        Button button2 = new Button("Botão 2");  
        Button button3 = new Button("Botão 3");  
        Button button4 = new Button("Botão 4");  
        Button button5 = new Button("Botão 5");  
        Button button6 = new Button("Botão 6");  
  
        // Criando o BorderPane  
        BorderPane borderPane = new BorderPane();  
  
        borderPane.setTop(button1); // Adiciona o botão na parte superior  
        borderPane.setBottom(button2); // Adiciona o botão na parte inferior  
        borderPane.setLeft(button3); // Adiciona o botão na parte esquerda  
        borderPane.setRight(button4); // Adiciona o botão na parte direita  
        borderPane.setCenter(button5); // Adiciona o botão no centro  
        borderPane.setTop(button6); // Adiciona outro botão na parte superior,  
        sobrepondo o primeiro  
  
        // Criando uma cena com o layout  
        Scene scene = new Scene(borderPane, 400, 300);  
  
        // Configurar o palco (Stage)  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```



GridPane em JavaFX

- **GridPane** é um layout que organiza os componentes em uma grade de linhas e colunas;
- Ideal para criar formulários, tabelas e layouts com alinhamento preciso;
- Cada célula da grade **pode conter um único componente** (botões, campos de texto, etc.);
- Cada componente é colocado em uma célula usando o método `add(node, columnIndex, rowIndex)`, onde:
 - `columnIndex`: Posição da coluna onde o componente será inserido;
 - `rowIndex`: Posição da linha onde o componente será inserido;
- Também é possível fazer o componente ocupar várias colunas ou linhas com `setColumnSpan()` e `setRowSpan()`;

Gridpane em javafx

```
public class GridPaneExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception{  
  
        Button button1 = new Button("Botão 1");  
        Button button2 = new Button("Botão 2");  
        Button button3 = new Button("Botão 3");  
        Button button4 = new Button("Botão 4");  
        Button button5 = new Button("Botão 5");  
        Button button6 = new Button("Botão 6");  
  
        // Criar o gridPane  
        GridPane gridPane = new GridPane();  
        gridPane.add(button1, 0, 0); // Adiciona o botão na posição (0, 0)  
        gridPane.add(button2, 0, 1); // Adiciona o botão na posição (0, 1)  
        gridPane.add(button3, 1, 0); // Adiciona o botão na posição (1, 0)  
        gridPane.add(button4, 1, 1); // Adiciona o botão na posição (1, 1)  
        gridPane.add(button5, 2, 2); // Adiciona o botão na posição (2, 2)  
        gridPane.add(button6, 2, 1); // Adiciona o botão na posição (2, 1)  
  
        gridPane.setHgap(10); // Espaçamento horizontal entre as colunas  
        gridPane.setVgap(10); // Espaçamento vertical entre as linhas  
  
        // Criando uma cena com o layout  
        Scene scene = new Scene(gridPane, 400, 300);  
  
        // Configurar o palco (Stage)  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```



StackPane e AnchorPane em JavaFX

- StackPane empilha os componentes uns sobre os outros;
- Útil quando você deseja sobrepor elementos, como colocar texto sobre uma imagem;
- O último componente adicionado é exibido no topo;
- Posicionando Elementos com AnchorPane
 - AnchorPane permite ancorar (fixar) componentes nas bordas da janela;
 - Cada componente pode ser ancorado em uma ou mais bordas (top, bottom, left, right);
 - É útil para layouts onde você deseja que os elementos permaneçam fixos ou redimensionados conforme a janela muda de tamanho;



Stackpane e anchorpane em javafx

```
public class StackPaneAnchor extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        Button button1 = new Button("Botão 1");  
        Button button2 = new Button("Botão 2");  
        Button button3 = new Button("Botão 3");  
        Button button4 = new Button("Botão 4");  
        Button button5 = new Button("Botão 5");  
        Button button6 = new Button("Botão 6");  
  
        // Criar o stackPane  
        StackPane stackPane = new StackPane();  
        stackPane.getChildren().addAll(button1, button2);  
  
        Scene stackScene = new Scene(stackPane, 400, 300);  
  
        // Criar o anchorPane  
        AnchorPane anchorPane = new AnchorPane();  
  
        AnchorPane.setTopAnchor(button3, 10.0);  
        AnchorPane.setRightAnchor(button3, 10.0);  
  
        anchorPane.getChildren().add(button3);  
  
        Scene anchorScene = new Scene(anchorPane, 400, 300);  
  
        // Configurar o palco (Stage)  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(stackScene);  
        primaryStage.show();  
  
        // mudar a cena para o anchorPane após 2 segundos  
        new Thread(() -> {  
            try {  
                Thread.sleep(2000);  
                Platform.runLater(() -> {  
                    primaryStage.setScene(anchorScene);  
                });  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }).start();  
    }  
}
```


Botões e Labels em JavaFX

- O que são Controles?
 - Controles são componentes interativos usados em interfaces gráficas, como botões, caixas de texto, checkboxes, entre outros;
 - Permitem que o usuário interaja com a aplicação de maneira intuitiva;
- Exemplos comuns de controles incluem:
 - Button (Botão)
 - Label (Rótulo de texto)
 - TextField (Campo de texto)
- Criando e Configurando Botões e Labels
 - **Button:** Representa um botão clicável
 - Pode ser configurado com texto, ícones, ou ambos;
 - Ação de clique é configurada com um EventHandler;
 - **Label:** Exibe um rótulo de texto não interativo
 - Usado para mostrar informações estáticas ou descrever outros controles;

Botões e Labels em JavaFX

```
public class ButtonLabelExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        // Criando uma Label  
        Label label = new Label("Olá, Mundo!");  
  
        Button button = new Button("Clique Aqui");  
  
        // Evento no botão  
        button.setOnAction(event -> {  
            label.setText("Botão foi clicado!");  
        });  
  
        // Vamos usar um layout VBox para organizar os componentes verticalmente  
        VBox vbox = new VBox();  
        vbox.getChildren().addAll(label, button);  
  
        // Criando uma cena com o layout  
        Scene scene = new Scene(vbox, 400, 300);  
  
        // Configurar o palco (Stage)  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```

TextField e TextArea em JavaFX

- Criando Campos de Texto e Áreas de Texto
 - **TextField:** Campo de texto de linha única usado para entradas simples, como nomes ou números. Ideal para entradas curtas e simples;
 - **TextArea:** Campo de texto de múltiplas linhas, usado para entradas longas, como descrições ou comentários. Permite ao usuário inserir texto em várias linhas;
- Limite de Caracteres:
 - Pode-se limitar o número de caracteres que o usuário pode digitar em um TextField ou TextArea utilizando eventos de texto;
- Interatividade:
 - Ambos os componentes podem reagir a eventos de teclado, como pressionar teclas, inserção de texto, e podem ser usados para captura de dados;

TextField e TextArea em javafx

```
public class TextFieldTextAreaExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        // Criando um textfield e um textarea  
        TextField textField = new TextField();  
        textField.setPromptText("Digite algo aqui...");  
  
        TextField textField2 = new TextField();  
        textField2.setPromptText("Digite algo aqui...");  
  
        // Limitando os caracteres do textfield  
        textField2.textProperty().addListener((observable, oldValue, newValue) -> {  
            if (newValue.length() > 3) {  
                textField2.setText(oldValue);  
            }  
        });  
  
        // Textarea  
        TextArea textArea = new TextArea();  
        textArea.setPromptText("Digite algo aqui...");  
  
        // Limitar as linhas do textarea  
        textArea.setPrefRowCount(5); // Definindo o número de linhas visíveis  
  
        Label label = new Label("Texto digitado: ");  
  
        textArea.textProperty().addListener((observable, oldValue, newValue) -> {  
            label.setText("Texto digitado: " + newValue); // Atualiza o label com o  
            texto do textarea  
        });  
  
        VBox vbox = new VBox(10, textField, textField2, textArea, label);  
  
        // Criando uma cena com o layout  
        Scene scene = new Scene(vbox, 400, 300);  
  
        // Configurar o palco (Stage)  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```

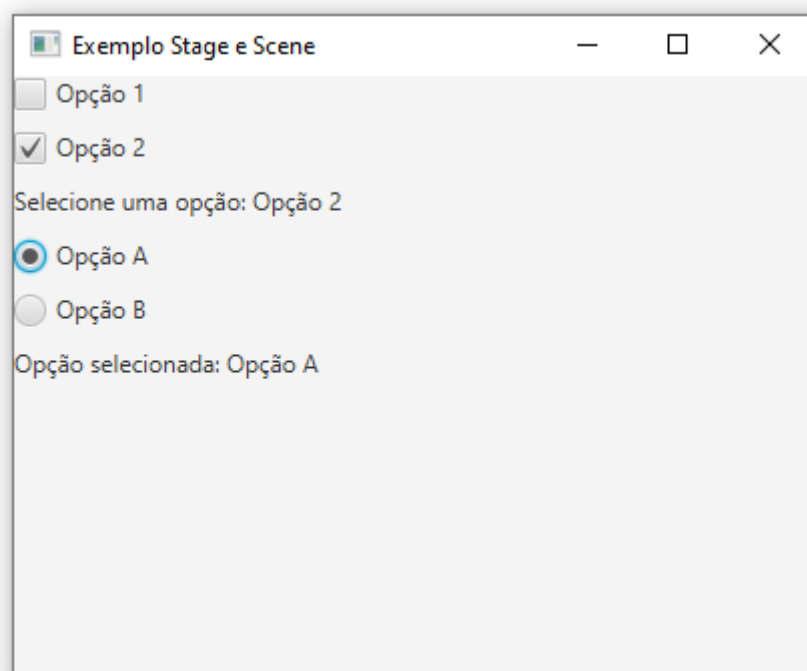
CheckBox e RadioButton em JavaFX

- **CheckBox:**
 - Representa uma caixa de seleção independente;
 - O usuário pode selecionar ou desmarcar várias opções simultaneamente;
 - Ideal para quando várias opções podem ser escolhidas;
- **RadioButton:**
 - Usado quando apenas uma opção pode ser selecionada dentro de um grupo de opções;
 - Para funcionar corretamente, os RadioButtons devem ser agrupados usando um ToggleGroup;
 - Útil quando o usuário deve escolher uma única opção de várias;
- **ToggleGroup:**
 - Um ToggleGroup é usado para agrupar RadioButtons, garantindo que apenas uma opção dentro do grupo possa ser selecionada por vez;
 - Todos os RadioButtons dentro de um ToggleGroup se comportam como uma escolha única (ou se pode haver um selecionado ao mesmo tempo);

Checkbox e radiobutton em javafx

```
public void updateCheckboxLabel(CheckBox checkbox1, CheckBox checkbox2, Label
Label) {
    String selected = "Selecione uma opção: ";
    if(checkbox1.isSelected()) selected += checkbox1.getText() + " ";
    if(checkbox2.isSelected()) selected += checkbox2.getText() + " ";
    Label.setText(selected);
    // Atualiza o texto do label com as opções selecionadas
}
```

Como fica:



```

public class RadioCheckboxRadioButtonExample extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception{

        // Criando o checkbox
        CheckBox checkbox1 = new CheckBox("Opção 1");
        CheckBox checkbox2 = new CheckBox("Opção 2");

        Label checkboxLabel = new Label("Selecione uma opção:");
        // Adicionando ação ao checkbox
        // Quando o checkbox for selecionado, atualiza o texto do label
        checkbox1.setOnAction(e -> updateCheckboxLabel(checkbox1, checkbox2,
        checkboxLabel));
        checkbox2.setOnAction(e -> updateCheckboxLabel(checkbox1, checkbox2,
        checkboxLabel));

        // Radio buttons
        RadioButton radioButton1 = new RadioButton("Opção A");
        RadioButton radioButton2 = new RadioButton("Opção B");
        // Agrupando os radio buttons
        // Apenas um radio button pode ser selecionado por vez
        ToggleGroup toggleGroup = new ToggleGroup();
        radioButton1.setToggleGroup(toggleGroup);
        radioButton2.setToggleGroup(toggleGroup);

        Label radioButtonLabel = new Label("Selecione uma opção de radio button:");

        toggleGroup.selectedToggleProperty().addListener((observable, oldValue,
        newValue) -> {
            if (newValue != null) {
                RadioButton selectedRadioButton = (RadioButton) toggleGroup.
                getSelectedToggle();
                radioButtonLabel.setText("Opção selecionada: " + selectedRadioButton.
                getText());
            }
        });

        VBox vbox = new VBox(10, checkbox1, checkbox2, checkboxLabel,
        radioButton1, radioButton2, radioButtonLabel);

        // Criando uma cena com o layout
        Scene scene = new Scene(vbox, 400, 300);

        // Configurar o palco (Stage)
        primaryStage.setTitle("Exemplo Stage e Scene");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}

```

ComboBox e ListView em JavaFX

- **ComboBox:** Um controle de seleção em formato de menu suspenso que permite ao usuário escolher uma opção de uma lista
 - Permite que o usuário selecione uma única opção;
 - É possível adicionar opções (itens) dinamicamente ou de forma estática;
- **ListView:** Um controle que exibe uma lista de itens em uma coluna
 - O usuário pode selecionar um ou mais itens da lista;
 - Suporta seleção simples e múltipla;
 - Permite adicionar, remover e manipular os itens da lista durante a execução do programa;

Combobox e listview em javafx

```
// Criando o combobox
ComboBox<String> comboBox = new ComboBox<>();

// Adicionando itens ao combobox
comboBox.getItems().addAll("Item 1", "Item 2", "Item 3");

comboBox.setPromptText("Selecione um item");

Label comboBoxLabel = new Label("Selecione uma opção do combobox:");

comboBox.setOnAction(event -> {
    String selected = comboBox.getSelectionModel().getSelectedItem();
    comboBoxLabel.setText("Você selecionou: " + selected);
    // Atualiza o texto do label com a opção selecionada
});

// Criando a listView
ObservableList<String> itens = FXCollections.observableArrayList("Item A", "Item B", "Item C", "Item D"); // Lista de itens para a ListView

ListView<String> listView = new ListView<>(itens); // Cria a ListView com os itens

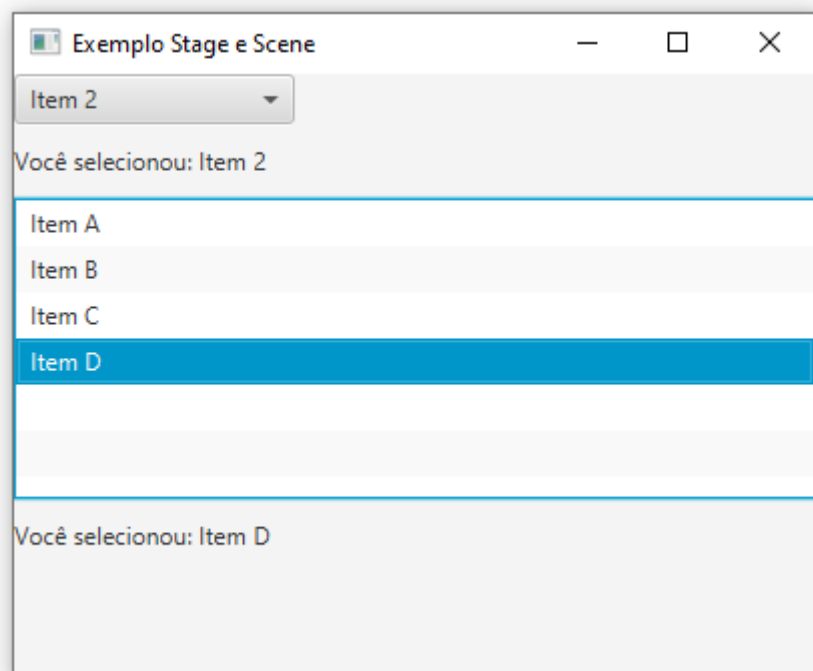
listView.setPrefHeight(150); // Define a altura preferencial da ListView

Label listViewLabel = new Label("Selecione uma opção do List View:");

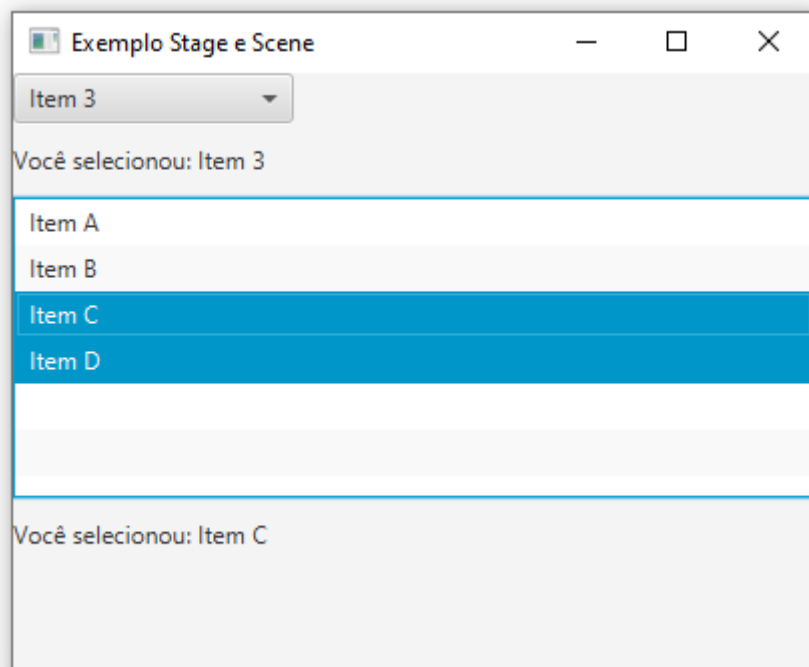
listView.getSelectionModel().selectedItemProperty().addListener((observable, oldValue, newValue) -> {
    String selected = listView.getSelectionModel().getSelectedItem();
    listViewLabel.setText("Você selecionou: " + newValue);
    // Atualiza o texto do label com a opção selecionada
});

listView.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE); // Define o modo de seleção para múltipla

VBox vbox = new VBox(10, comboBox, comboBoxLabel, listView, listViewLabel);
```



Com múltipla seleção:



Eventos em JavaFx

- **O que são eventos em JavaFX?**
 - Eventos são ações realizadas pelo usuário ou pelo sistema que a aplicação pode "ouvir" e responder;
- **Exemplos de eventos:**
 - **Clique de botão:** Quando um botão é pressionado pelo usuário;
 - **Movimento do mouse:** O movimento ou clique do mouse em uma área da interface;
 - **Teclado:** Pressionamento de uma tecla ou combinação de teclas;
 - **Mudança de foco:** Quando um campo de texto ganha ou perde foco;
 - Cada evento gera um objeto de evento que contém informações sobre a ação realizada;
- **Modelo de eventos em JavaFX segue o padrão Delegação de Eventos, onde o evento é capturado e encaminhado para o manipulador correto:**
 - **Origem do evento:** O componente que gera o evento, como um botão;
 - **Objeto de evento:** A informação sobre o evento (ex.: MouseEvent,(ActionEvent));
 - **Listener/Handler (Manipulador de Evento):** Um bloco de código que responde ao evento;
- **Event Handling (Manipulação de Eventos):**
 - O listener ou handler é associado ao componente (ex.: botão) que deseja capturar o evento;
 - Quando o evento ocorre, o listener é chamado para executar o código associado;

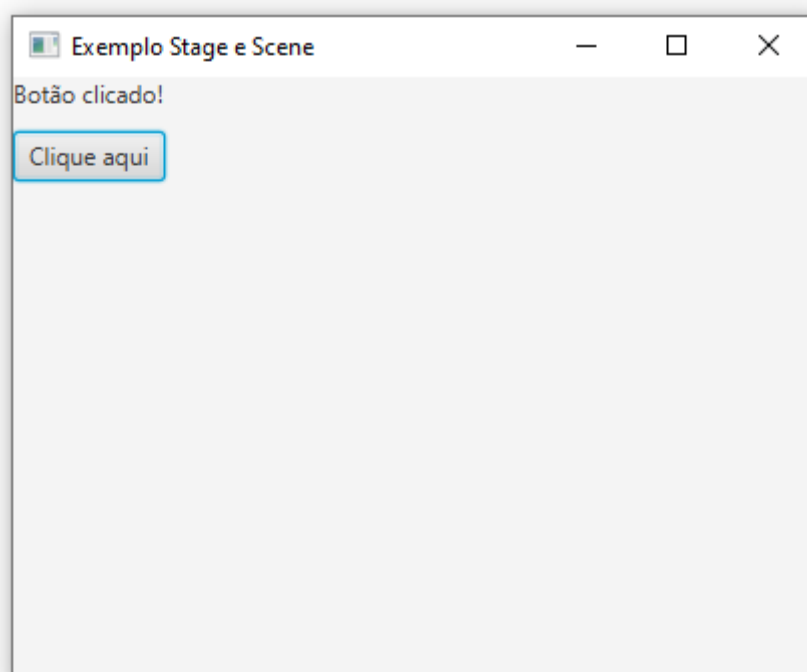
Eventos em javafx

Eventos de Clique em JavaFX

- Eventos de clique são capturados quando o usuário interage com um botão, pressionando-o e soltando-o;
- O evento mais comum é o **ActionEvent**, que é gerado ao clicar em um botão;
- Para lidar com eventos de clique, usamos **event handlers** ou **listeners**, que são blocos de código associados ao componente (botão);
- **Passos para Capturar um Evento de Clique:**
 - Criar o componente interativo (ex.: Button);
 - Adicionar um EventHandler usando o método `setOnAction()` do botão;
 - Implementar o código que deve ser executado quando o evento de clique ocorre;

Eventos de clique em javafx

```
public class ButtonClickExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        Label label = new Label("Clique no botão para ver o resultado");  
        Button button = new Button("Clique aqui");  
  
        button.setOnAction(event -> {  
            label.setText("Botão clicado!");  
            // Aqui você pode adicionar mais lógica para o que acontece quando o botão é  
            // clicado  
        });  
  
        VBox vbox = new VBox(10, label, button);  
    }  
}
```



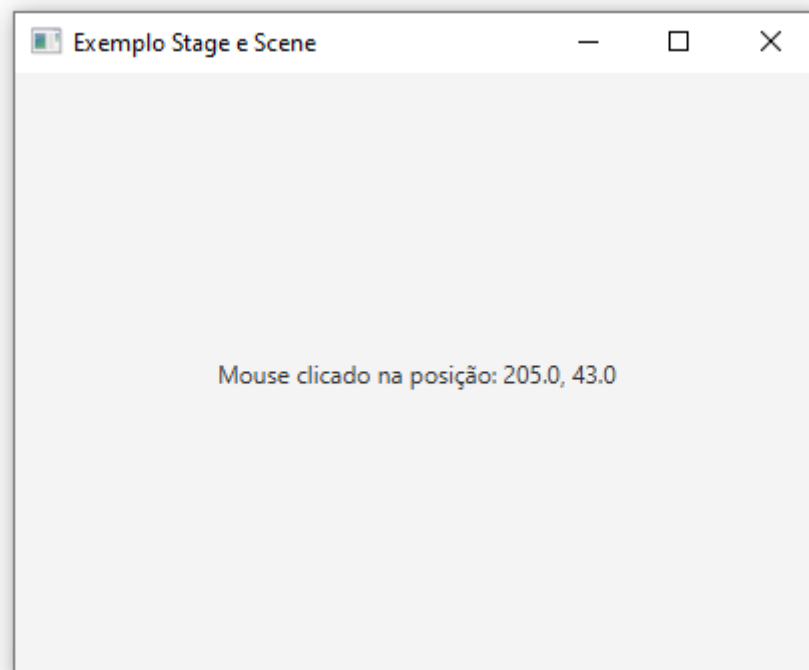
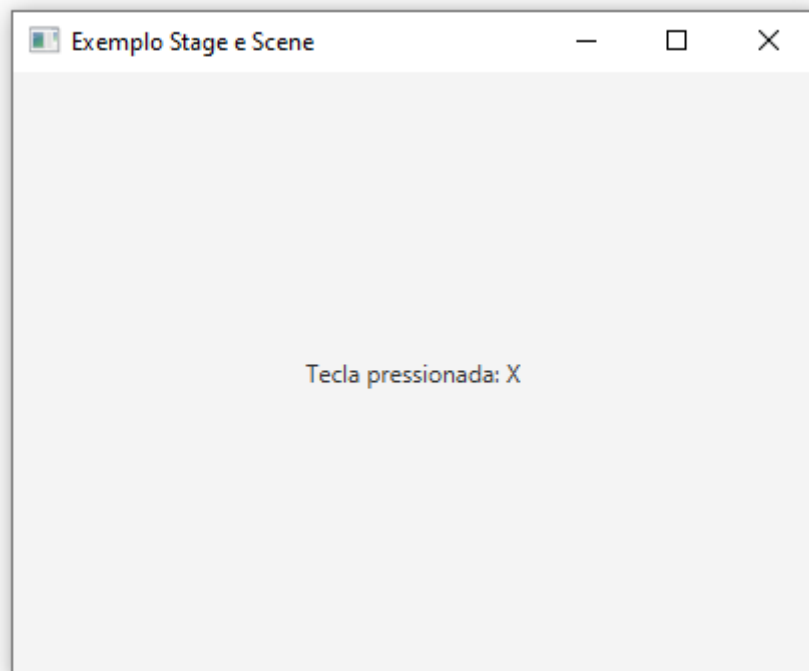
Eventos de Teclado e Mouse em JavaFX

- Eventos de Teclado:
 - Capturam ações do usuário relacionadas ao teclado, como pressionar, soltar ou digitar uma tecla;
- Principais eventos:
 - `KeyPressed`: Quando uma tecla é pressionada;
 - `KeyReleased`: Quando uma tecla é liberada;
 - `KeyTyped`: Quando uma tecla é digitada (geralmente usado para texto);
- Eventos de Mouse:
 - Capturam interações do usuário com o mouse, como movimento e cliques;
- Principais eventos:
 - `MouseClicked`: Quando o mouse é clicado;
 - `MouseMoved`: Quando o mouse é movido;
 - `MouseEntered/MouseExited`: Quando o ponteiro do mouse entra ou sai de um componente;

Eventos de teclado e mouse em javafx

```
public class KeyboardMouseEvent extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception{  
  
        Label label = new Label("Interaja com o seu teclado");  
  
        StackPane root = new StackPane();  
        root.getChildren().add(label);  
  
        // Capturar eventos de teclado  
        root.setOnKeyPressed(event -> {  
            String key = event.getCode().toString();  
            label.setText("Tecla pressionada: " + key);  
        });  
  
        // Evento de mouse  
        root.setOnMouseClicked(event -> {  
            label.setText("Mouse clicado na posição: " + event.getX() + ", " + event.getY()  
            ());  
        });  
  
        // Criando uma cena com o layout  
        Scene scene = new Scene(root, 400, 300);  
        // Configurar o foco para receber eventos de teclado  
        scene.setOnKeyPressed(root.setOnKeyPressed());  
    }  
}
```

Não esqueça do enunciado!



Tratamento de Estilos com CSS em JavaFX

- CSS (Cascading Style Sheets) é amplamente utilizado para estilizar componentes de interface no JavaFX;
- Com o uso de CSS, é possível modificar:
 - Cores, fontes, margens, tamanhos de componentes;
 - Estilos dinâmicos para eventos, como o foco ou o hover;
- Benefícios do CSS em JavaFX:
 - Facilita a personalização da interface de forma organizada e consistente;
 - Permite separar a lógica do design, melhorando a manutenção do código;
- No JavaFX, os componentes podem ser estilizados diretamente pelo método `setStyle()`, ou, preferencialmente, através de arquivos CSS externos;
- Estilos comuns incluem:
 - **Background-color**: Cor de fundo do componente;
 - **Text-fill**: Cor do texto;
 - **Padding**: Espaçamento interno do componente;
 - **Border**: Estilo, cor e largura da borda;

Tratamento de estilos com css em javafx

```
Label label = new Label("Label estilizada com CSS");

label.getStyleClass().add(e:"label-custom"); // Adiciona uma classe de estilo
personalizada

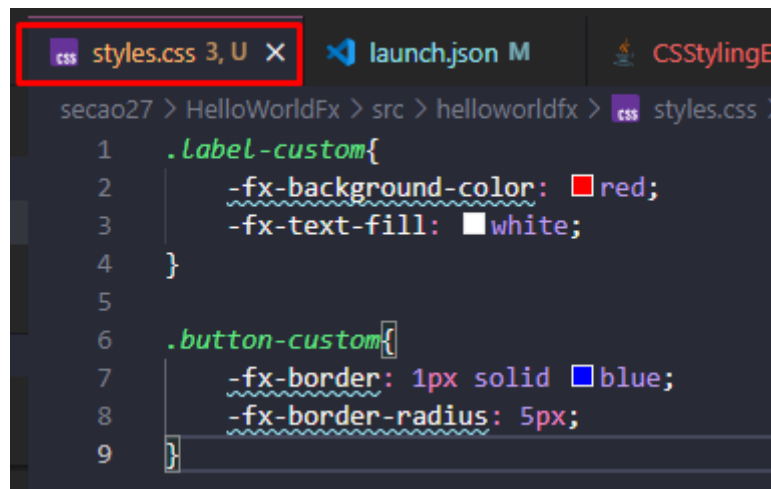
Button button = new Button("Botão estilizado com CSS");

button.getStyleClass().add(e:"button-custom"); // Adiciona uma classe de estilo
personalizada

VBox vbox = new VBox(10);
vbox.getChildren().addAll(label, button);

// Criando uma cena com o layout
Scene scene = new Scene(vbox, 400, 300);

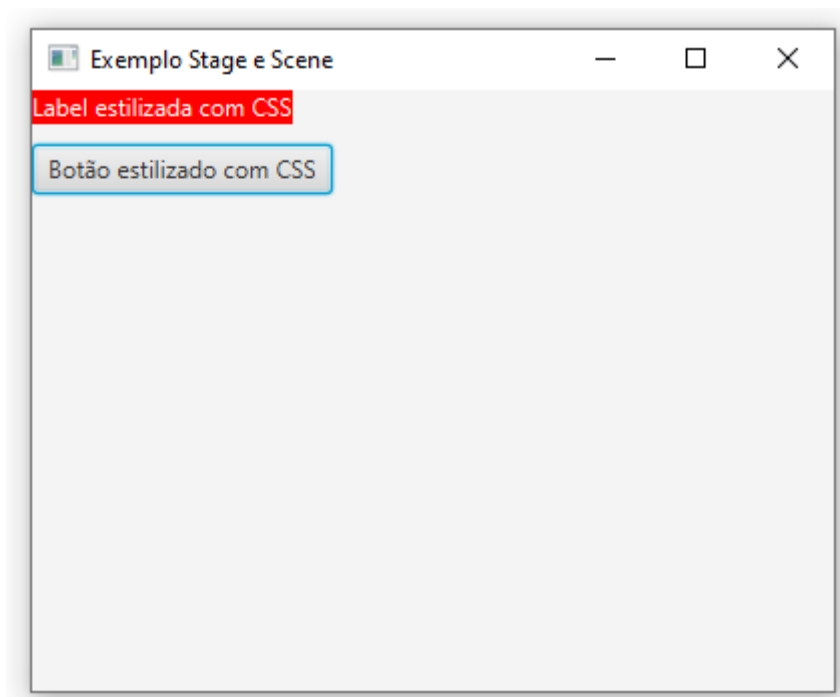
// Adicionando o arquivo CSS à cena
scene.getStylesheets().add(getClass().getResource(name:"styles.css").
toExternalForm());
```



The image shows a code editor window with a dark theme. The top bar has three tabs: 'styles.css 3, U' (highlighted with a red box), 'launch.json M', and 'CSSStylingE'. The main editor area shows the following CSS code:

```
1  .label-custom{  
2      -fx-background-color: red;  
3      -fx-text-fill: white;  
4  }  
5  
6  .button-custom{  
7      -fx-border: 1px solid blue;  
8      -fx-border-radius: 5px;  
9  }
```

Se usa o prefixo e sufixo -fx- antes das estilizações!



Criando Janelas Secundárias em JavaFX

- Em JavaFX, a janela principal é representada por um Stage;
- É possível criar janelas secundárias, chamadas de Secondary Stages, para exibir diferentes conteúdos ou diálogos;
- Cada janela secundária é uma nova instância da classe Stage;
- Para criar uma nova janela:
 - Crie uma nova instância de Stage;
 - Defina o conteúdo usando uma Scene, semelhante ao Stage principal;
 - Configure a janela (título, dimensões, etc.);
 - Use o método show() para exibir a nova janela;



Criando janelas secundárias em javafx

```
public class MultipleWindowsExample extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception{

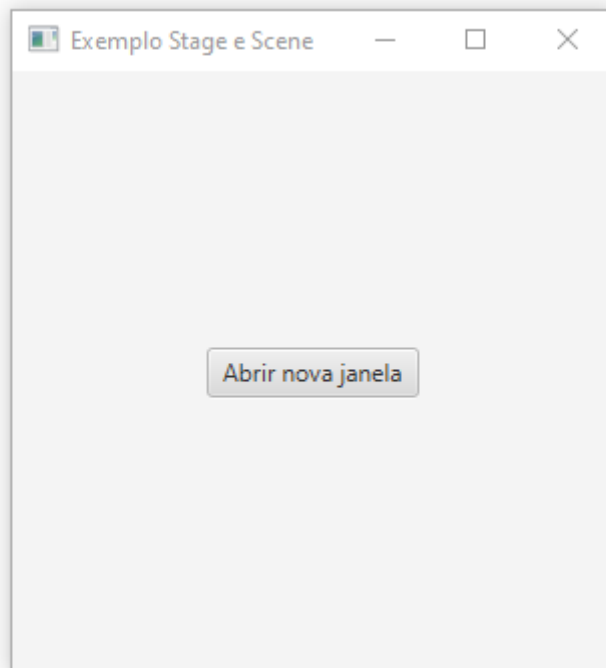
        Button button = new Button("Abrir nova janela");
        button.setOnAction(event -> openSecondaryWindow());

        StackPane primaryLayout = new StackPane();
        primaryLayout.getChildren().addAll(button);

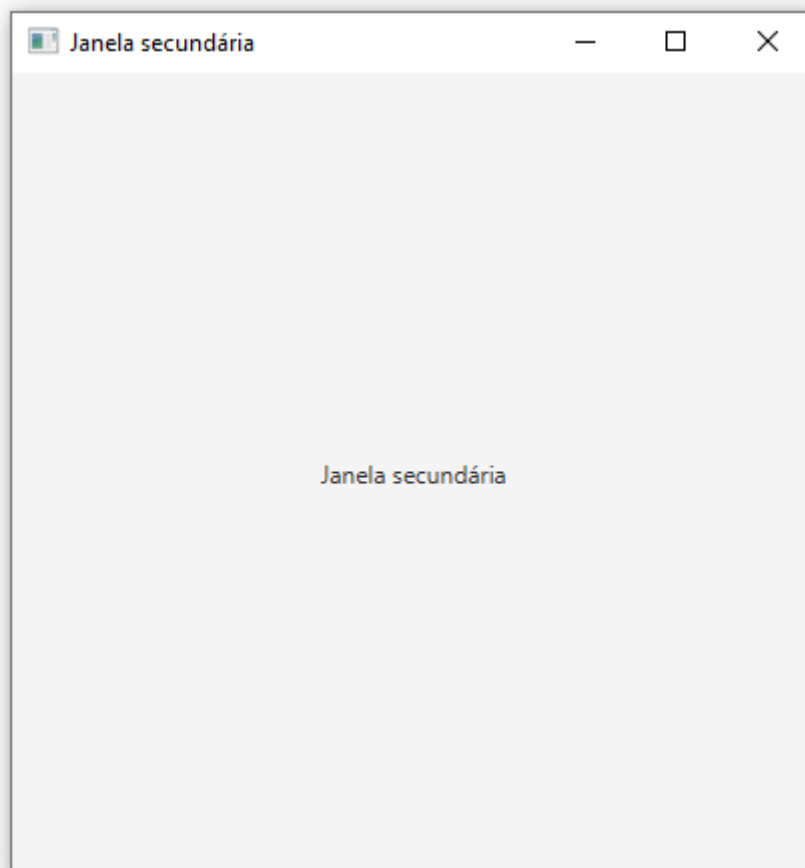
        Scene primaryScene = new Scene(primaryLayout, 300, 300);

        // Configurar o palco (Stage)
        primaryStage.setTitle("Exemplo Stage e Scene");
        primaryStage.setScene(primaryScene);
        primaryStage.show();
    }

    // Função para nova janela
    public void openSecondaryWindow(){
        Stage secondaryStage = new Stage();
        Label label = new Label("Janela secundária");
        StackPane secondaryLayout = new StackPane();
        secondaryLayout.getChildren().addAll(label);
        Scene secondaryScene = new Scene(secondaryLayout, 400, 400);
        secondaryStage.setTitle("Janela secundária");
        secondaryStage.setScene(secondaryScene);
        secondaryStage.show();
    }
}
```



Ele abre outras janelas, e não atualiza a principal!



Diálogos de Alerta em JavaFX

- **Alert** é uma classe em JavaFX que permite exibir caixas de diálogo com mensagens para o usuário.
- Os diálogos de alerta são usados para:
 - Informar o usuário sobre uma ação;
 - Alertar sobre erros, advertências ou pedir confirmação;
- Os tipos de alerta são configurados usando a enumeração `Alert.AlertType`:
 - **INFORMATION**: Exibe informações gerais para o usuário;
 - **WARNING**: Exibe um aviso sobre uma situação que requer atenção;
 - **ERROR**: Indica um erro que ocorreu na aplicação;
 - **CONFIRMATION**: Solicita uma confirmação do usuário antes de prosseguir com uma ação;

Diálogos de alerta em javafx

```
Button buttonInfo = new Button("Alerta de informação");
Button buttonWarning = new Button("Alerta de perigo");
Button buttonError = new Button("Alerta de erro");
Button buttonConfirmation = new Button("Alerta de confirmação");

buttonInfo.setOnAction(event -> showAlert(AlertType.INFORMATION,
title:"Informação", message:"Alerta de informação"));

buttonWarning.setOnAction(event -> showAlert(AlertType.WARNING,
title:"Perigo", message:"Alerta de perigo"));

buttonError.setOnAction(event -> showAlert(AlertType.ERROR,
title:"Erro", message:"Alerta de erro"));

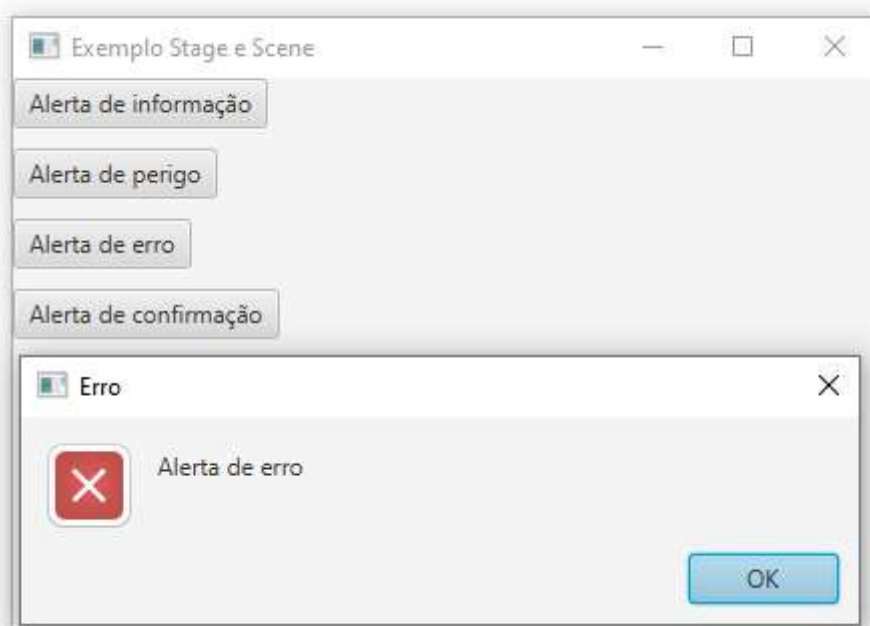
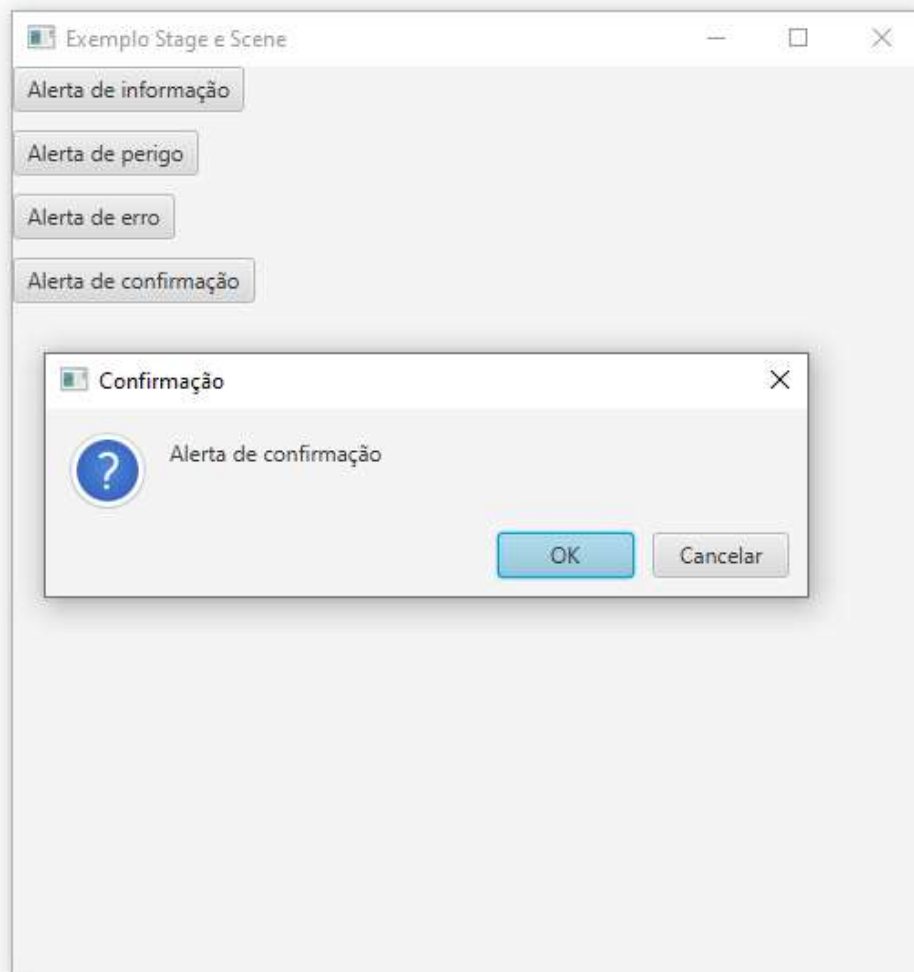
buttonConfirmation.setOnAction(event -> showAlert(AlertType.
CONFIRMATION, title:"Confirmação", message:"Alerta de
confirmação"));

VBox vBox = new VBox(10);
vBox.getChildren().addAll(buttonInfo, buttonWarning, buttonError,
buttonConfirmation);

Scene scene = new Scene(vBox, 500, 500);
```

```
private void showAlert(AlertType alertType, String title, String
message){

    Alert alert = new Alert(alertType);
    alert.setTitle(title);
    alert.setHeaderText(null);
    alert.setContentText(message);
    alert.showAndWait();
}
```



Exibindo Imagens no JavaFX

- `ImageView` é uma classe usada para exibir imagens em JavaFX;
- Para carregar uma imagem:
 - Crie uma instância da classe `Image` fornecendo o caminho da imagem;
 - Passe a instância de `Image` para o `ImageView` para exibi-la;
- É possível ajustar o tamanho da imagem usando os métodos:
 - `setFitWidth()`: Define a largura desejada da imagem;
 - `setFitHeight()`: Define a altura desejada da imagem;
- Para manter a proporção da imagem ao redimensionar:
 - Use `setPreserveRatio(true)`;
 - A imagem pode ser posicionada dentro do layout como qualquer outro componente JavaFX, como dentro de um `VBox`, `HBox`, etc;

Exibindo imagens no javafx

```
public class ImageViewExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        Image image = new Image  
        // Note o uso de file: antes do caminho do arquivo!  
        ("file:C:\\Users\\Carlos\\Desktop\\Programação 2024 e Redes\\Java  
        HdC\\secao27\\HelloWorldFx\\src\\helloworldfx\\loro.jpg");  
  
        ImageView imageView = new ImageView(image);  
  
        imageView.setFitHeight(300);  
        imageView.setFitWidth(200);  
        imageView.setPreserveRatio(true);  
  
        VBox vBox = new VBox(10);  
        vBox.getChildren().addAll(imageView);  
  
        Scene scene = new Scene(vBox, 500, 500);  
  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```



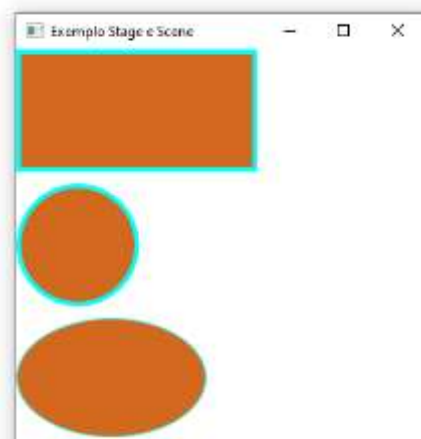
Desenhando Formas Geométricas em JavaFX

- JavaFX oferece classes prontas para criar formas geométricas, como:
 - `Rectangle`: Para criar retângulos;
 - `Circle`: Para criar círculos;
 - `Ellipse`, `Line`, `Polygon`, etc;
- As formas podem ser facilmente manipuladas com métodos para ajustar tamanho, posição, bordas e preenchimento;
- É possível personalizar as formas com cores e bordas, usando:
 - `setFill()`: Define a cor de preenchimento da forma;
 - `setStroke()`: Define a cor da borda;
 - `setStrokeWidth()`: Define a largura da borda;



Desenhando formas geométricas em javafx

```
public class ShapeDrawingExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        // Retangulo  
        Rectangle rectangle = new Rectangle(200, 100);  
        rectangle.setFill(Color.CHOCOLATE);  
        rectangle.setStroke(Color.AQUA);  
        rectangle.setStrokeWidth(4);  
  
        // circulo  
        Circle circle = new Circle(50);  
        circle.setFill(Color.CHOCOLATE);  
        circle.setStroke(Color.AQUA);  
        circle.setStrokeWidth(4);  
  
        // elipse  
        Ellipse ellipse = new Ellipse(80, 50);  
        ellipse.setFill(Color.CHOCOLATE);  
        ellipse.setStroke(Color.AQUA);  
  
        VBox vBox = new VBox(10);  
        vBox.getChildren().addAll(rectangle, circle, ellipse);  
  
        Scene scene = new Scene(vBox, 500, 500);  
    }  
}
```



Canvas e Gráficos 2D em JavaFX

- Canvas é um componente em JavaFX usado para desenhar gráficos 2D diretamente;
- Ele fornece uma área na qual podemos desenhar formas, imagens e texto usando uma API de gráficos;
- Para desenhar no Canvas, utilizamos o `GraphicsContext`, que fornece métodos para desenhar linhas, formas, e aplicar cores;
- O objeto `GraphicsContext` permite:
 - Desenhar linhas, círculos, retângulos, texto, etc;
 - Configurar o estilo de preenchimento, cor, e espessura de traço;
- Métodos comuns para desenhar:
 - `strokeLine()`: Desenha uma linha;
 - `fillRect()`: Preenche um retângulo;
 - `strokeOval()`: Desenha um oval/círculo sem preenchimento;
 - `fillText()`: Escreve texto;

Canvas e gráficos 2d em javafx

```
public class CanvasExample extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception {

        Canvas canvas = new Canvas(400, 400);
        GraphicsContext gc = canvas.getGraphicsContext2D();

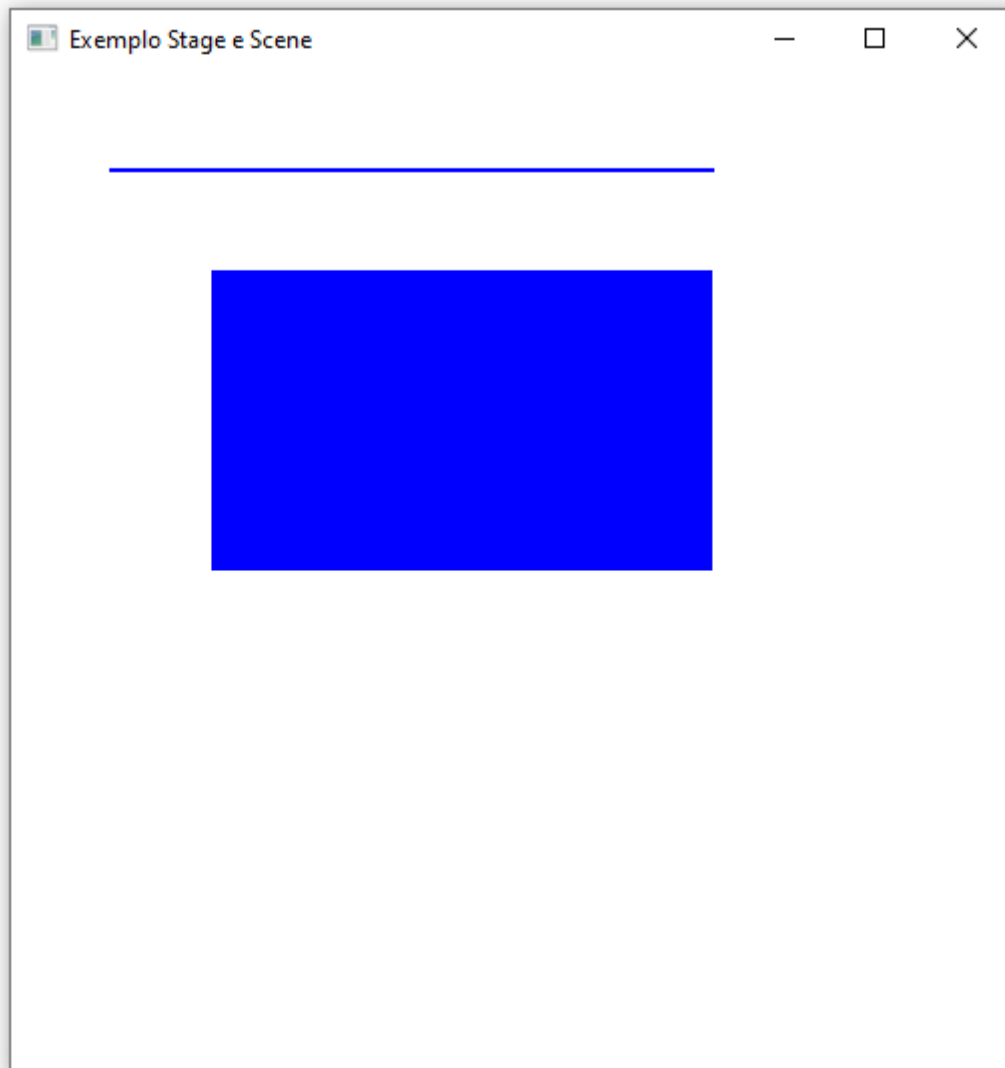
        // Desenhar o canvas
        drawShapes(gc);

        VBox vBox = new VBox(10);
        vBox.getChildren().addAll(canvas);

        Scene scene = new Scene(vBox, 500, 500);

        primaryStage.setTitle("Exemplo Stage e Scene");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // Métodos do canvas
    private void drawShapes(GraphicsContext gc){
        gc.setStroke(Color.BLUE);
        gc.setLineWidth(2);
        gc.strokeLine(50, 50, 350, 50);
        gc.setFill(Color.BLUE);
        gc.fillRect(100, 100, 250, 150);
    }
}
```



Animações no JavaFX

- JavaFX oferece suporte robusto para animações, permitindo criar interações dinâmicas na interface do usuário;
- As animações podem ser usadas para:
 - Mover objetos na tela;
 - Alterar propriedades como opacidade, rotação, escala, etc;
 - Tornar a interface mais interativa e visualmente atrativa;
- **Timeline** é uma das classes principais para criar animações simples em JavaFX;
- Funciona ao modificar uma ou mais propriedades de um nó (componente) ao longo do tempo;
- Um **KeyFrame** representa um ponto específico no tempo, onde propriedades do nó podem ser modificadas;



Animações no Javafx

```
public class SmoothAnimationExample extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        // Animando um círculo  
        Circle circle = new Circle(50, Color.BLUE); // Cria um círculo azul  
        com raio 50  
        circle.setTranslateX(-200); // Define a posição inicial do círculo  
  
        Timeline timeline = new Timeline(); // Cria uma linha do tempo para  
        animação  
  
        KeyFrame keyFrame = new KeyFrame(Duration.seconds(2),  
            new KeyValue(circle.translateXProperty(), 200)); // Anima o  
        círculo para a posição X 200 em 2 segundos  
  
        timeline.getKeyFrames().add(keyFrame); // Adiciona o KeyFrame à  
        linha do tempo  
        timeline.setCycleCount(Timeline.INDEFINITE); // Define a contagem  
        de ciclos da animação como indefinida  
        timeline.setAutoReverse(true); // Permite que a animação reverta  
        após completar  
        timeline.play(); // Inicia a animação  
  
        StackPane stackPane = new StackPane();  
        stackPane.getChildren().add(circle); // Adiciona o círculo ao  
        StackPane  
  
        Scene scene = new Scene(stackPane, 500, 500);  
  
        primaryStage.setTitle("Exemplo Stage e Scene");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```

Rpg?

Organizando o Código JavaFX: Padrão MVC

- MVC (Model-View-Controller) é um padrão de arquitetura de software que separa a lógica de negócios, os dados e a interface do usuário:
 - **Model:** Representa os dados e a lógica de negócios. Gerencia o estado da aplicação;
 - **View:** Responsável pela interface do usuário. Exibe os dados e atualiza as informações conforme o estado do modelo;
 - **Controller:** Coordena a interação entre o Model e a View. Recebe ações do usuário e solicita mudanças no modelo;
- Uma boa ordem de criação pode ser: Model => Controller => fxml => Classe principal;



Organizando o código javafx: Padrão MVC

Model, View, Controller

```
public class CounterModel {  
  
    private int count;  
  
    public int getCount() {  
        return count;  
    }  
  
    public void increment() {  
        count++;  
    }  
  
    public void decrement() {  
        if (count > 0) {  
            count--;  
        }  
    }  
  
    public void reset() {  
        count = 0;  
    }  
}
```

```
import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class CounterController {

    private CounterModel model;

    @FXML
    private Label countLabel;

    public CounterController() {
        model = new CounterModel();
    }

    @FXML
    private void handleIncrement() {
        model.increment();
        updateCountLabel();
    }

    @FXML
    private void handleDecrement() {
        model.decrement();
        updateCountLabel();
    }

    @FXML
    private void handleReset() {
        model.reset();
        updateCountLabel();
    }

    private void updateCountLabel() {
        countLabel.setText(Integer.toString(model.getCount()));
    }
}
```

```

secao27 > HelloWorldFx > src > helloworldfx > </> Counter.fxml
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.scene.control.Label?>
4  <?import javafx.scene.control.Button?>
5  <?import javafx.scene.layout.VBox?>
6
7
8  <VBox xmlns="http://javafx.com/javafx/8" xmlns:fx="http://javafx.com/fxml/1" fx:controller="helloworldfx.CounterController"
9  spacing="10" alignment="CENTER">
10     <Label fx:id="countLabel" text="0" style="-fx-font-size: 24px;"/>
11     <Button text="Incrementar" onAction="#handleIncrement"/>
12     <Button text="Decrementar" onAction="#handleDecrement"/>
13     <Button text="Resetar" onAction="#handleReset"/>
14
15 </VBox>

```

```

package helloworldfx;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class MainApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception{
        Parent root = FXMLLoader.load(getClass().getResource("counter.fxml"));
        primaryStage.setTitle("Contador Padrão MVC");
        primaryStage.setScene(new Scene(root, 500, 500));
        primaryStage.show();
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        launch(args);
    }

```



Projeto Calculadora em JavaFX

```
// Logica da Calculadora
private void handleButtonPress(String value) {
    switch (value) {
        case "C" -> {
            currentInput = ""; // Limpa a entrada atual
            operator = ""; // Limpa o operador
            previousValue = 0; // Reseta o valor anterior
            display.setText(""); // Limpa o display
        }
        case "=" -> {
            if(!currentInput.isEmpty() && !operator.isEmpty()) {
                double currentValue = Double.parseDouble(currentInput); // Converte a entrada atual para double
                double result = calculate(previousValue, currentValue, operator); // Realiza o cálculo
                display.setText(String.valueOf(result)); // Exibe o resultado
                currentInput = String.valueOf(result); // Atualiza a entrada atual com o resultado
                operator = ""; // Limpa o operador
            }
        }
        case "+", "-", "*", "/" -> {
            if(!currentInput.isEmpty()) {
                operator = value; // Define o operador
                previousValue = Double.parseDouble(currentInput); // Converte a entrada atual para double
                currentInput = ""; // Limpa a entrada atual para o próximo número
            }
        }
        default -> {
            currentInput += value; // Adiciona o valor ao input atual
            display.setText(currentInput); // Atualiza o display
        }
    }
}
```

```
// Realizar o cálculo quando o usuário pressionar "="
private double calculate(double a, double b, String op) {
    switch (op) {
        case "+" -> {
            return a + b;
        }
        case "-" -> {
            return a - b;
        }
        case "*" -> {
            return a * b;
        }
        case "/" -> {
            if (b != 0) {
                return a / b;
            } else {
                display.setText("Erro: Divisão por zero");
                return 0; // Retorna 0 em caso de erro
            }
        }
        default -> {
            return 0; // Retorna 0 se o operador não for reconhecido
        }
    }
}
```

```

@Override
public void start(Stage primaryStage) throws Exception{

    primaryStage.setTitle("Calculadora JavaFX");

    // Layout principal
    VBox root = new VBox();
    root.setPadding(new Insets(10)); // Espaçamento interno do layout
    root.setSpacing(10); // Espaçamento entre os componentes

    // Display
    display.setId("display");
    display.setMinSize(250, 50); // Tamanho mínimo do display
    display.setMaxSize(250, 50); // Tamanho máximo do display
    display.setMaxWidth(Double.MAX_VALUE); // Permite que o display ocupe toda a largura disponível
    VBox.setVgrow(display, Priority.NEVER); // Não permite que o display cresça verticalmente
    root.getChildren().add(display);

    // Pannel de botões
    GridPane grid = new GridPane();
    grid.setHgap(10); // Espaçamento horizontal entre os botões
    grid.setVgap(10); // Espaçamento vertical entre os botões
    grid.setPadding(new Insets(10)); // Espaçamento interno do painel

```

```

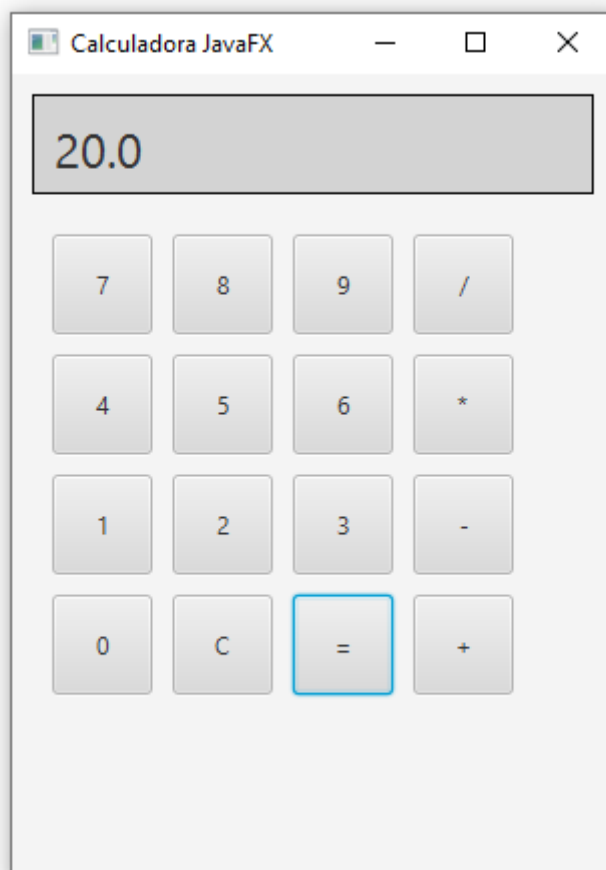
String[] buttons = {
    "7", "8", "9", "/",
    "4", "5", "6", "*",
    "1", "2", "3", "-",
    "0", "C", "=", "+"
};

int row = 0;
int col = 0;
for(String text : buttons) {
    Button button = new Button(text);
    button.setMinSize(50, 50); // Tamanho mínimo dos botões
    button.addEventHandler(MouseEvent.MOUSE_CLICKED, e -> handleButtonPress(text));
    grid.add(button, col, row); // Adiciona o botão na grade
    col++;
    if(col > 3) {
        col = 0;
        row++;
    }
}
root.getChildren().add(grid);

// Cena
Scene scene = new Scene(root, 300, 400);
scene.getStylesheets().add(getClass().getResource(name:"styles.css").toExternalForm()); // Adiciona o CSS
primaryStage.setScene(scene);
primaryStage.show();

```

```
#display {  
    -fx-font-size: 24px;  
    -fx-background-color: lightgray;  
    -fx-padding: 10px;  
    -fx-border-color: black;  
    -fx-border-width: 1px;  
}
```



Projeto Cronometro em JavaFx

```
// Inicia o cronômetro
private void startTimer(){
    if(timeline == null || timeline.getStatus() != Timeline.Status.RUNNING){
        timeline = new Timeline(new KeyFrame(Duration.seconds(1), e -> updateTimer()));
        timeline.setCycleCount(Timeline.INDEFINITE);
        timeline.play();
    }
}

// Pausa o cronômetro
private void pauseTimer(){
    if(timeline != null && timeline.getStatus() == Timeline.Status.RUNNING){
        timeline.pause();
    }
}
```

```
// Reinicia o cronômetro
private void resetTimer(){
    if(timeline != null){
        timeline.stop();
    }
    secondsElapsed = 0;
    updateTimerDisplay();
}

// Atualiza o display do cronômetro
private void updateTimer(){
    secondsElapsed++;
    updateTimerDisplay();
}

private void updateTimerDisplay(){
    int hours = secondsElapsed / 3600;
    int minutes = (secondsElapsed % 3600) / 60;
    int seconds = secondsElapsed % 60;
    timelabel.setText(String.format(format:"%02d:%02d:%02d", hours, minutes, seconds));
}
```

```
public class Cronometro extends Application {

    // Componentes do cronômetro
    private Label timelabel = new Label("00:00:00");
    private int secondsElapsed = 0;
    private Timeline timeline;
```

```

@Override
public void start(Stage primaryStage) throws Exception{

    primaryStage.setTitle("Cronometro JavaFX");

    // Layout principal
    VBox root = new VBox();
    root.setPadding(new Insets(10)); // Espaçamento interno do layout
    root.setSpacing(10); // Espaçamento entre os componentes

    // Configuração do display do cronômetro
    timeLabel.setId("timeLabel");
    root.getChildren().add(timeLabel);

    // Layout dos botões
    HBox hbox = new HBox();
    hbox.setSpacing(10); // Espaçamento entre os botões

```

```

Button startButton = new Button("Iniciar");
startButton.setMinSize(100, 50);
startButton.setId("startButton");
startButton.setOnAction(e -> {
    startTimer(); // Inicia o cronômetro ao clicar no botão
});

Button pauseButton = new Button("Pausar");
pauseButton.setMinSize(100, 50);
pauseButton.setId("pauseButton");
pauseButton.setOnAction(e -> {
    pauseTimer(); // Pausa o cronômetro ao clicar no botão
});

Button resetButton = new Button("Reiniciar");
resetButton.setMinSize(100, 50);
resetButton.setId("resetButton");
resetButton.setOnAction(e -> {
    resetTimer(); // Reinicia o cronômetro ao clicar no botão
});

hbox.getChildren().addAll(startButton, pauseButton, resetButton);
root.getChildren().add(hbox);

// Cena
Scene scene = new Scene(root, 400, 200);
scene.getStylesheets().add(getClass().getResource("styles.css").toExternalForm()); // Adiciona o CSS
primaryStage.setScene(scene);
primaryStage.show();

```

```

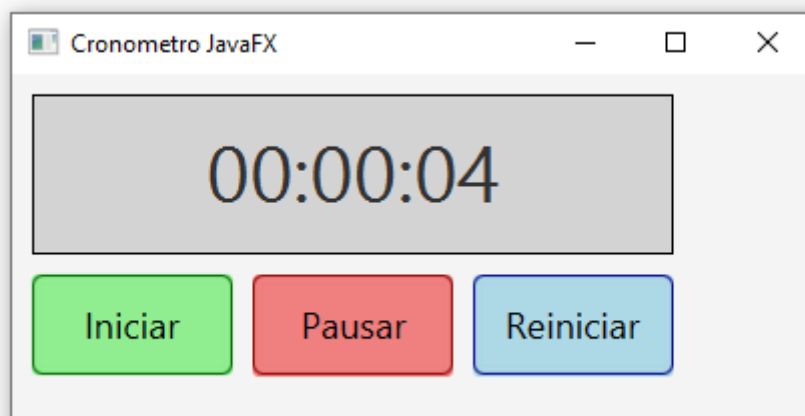
#timeLabel {
    -fx-font-size: 40px;
    -fx-background-color: lightgray;
    -fx-padding: 10px;
    -fx-border-color: black;
    -fx-border-width: 1px;
    -fx-pref-width: 320px;
    -fx-alignment: center;
}

#startButton, #pauseButton, #resetButton {
    -fx-font-size: 18px;
    -fx-background-color: lightgreen;
    -fx-border-radius: 5px;
    -fx-border-color: darkgreen;
    -fx-text-fill: black;
}

#pauseButton {
    -fx-background-color: lightcoral;
    -fx-border-color: darkred;
}

#resetButton {
    -fx-background-color: lightblue;
    -fx-border-color: darkblue;
}

```



Projeto Quiz em JavaFX

```
private void handleNextQuestion(){
    RadioButton selectedRadioButton = (RadioButton) toggleGroup.getSelectedToggle();
    if (selectedRadioButton != null) {
        int selectedIndex = getSelectedIndex();
        if (selectedIndex == correctAnswers[currentQuestion]) {
            score++; // Incrementa a pontuação se a resposta estiver correta
        }
        currentQuestion++; // Avança para a próxima pergunta
        if (currentQuestion < questions.length) {
            updateQuestion(); // Atualiza a pergunta e as opções
        } else {
            showScore(); // Exibe a pontuação final se não houver mais perguntas
        }
    }
}
```

```
private void updateQuestion(){
    questionLabel.setText(questions[currentQuestion]); // Atualiza o texto da pergunta
    root.getChildren().removeIf(node -> node instanceof RadioButton); // Remove os botões de opção antigos
    toggleGroup = new ToggleGroup();
    for (String option : options[currentQuestion]) {
        RadioButton radioButton = new RadioButton(option);
        radioButton.setToggleGroup(toggleGroup);
        radioButton.getStyleClass().add("radio-button"); // Adiciona estilo CSS
        root.getChildren().add(index:1, radioButton); // Adiciona os novos botões de opção após o Label da pergunta, label tem índice 0
    }
}
```

```
private void showScore(){
    root.getChildren().clear(); // Limpa o layout
    Label scoreLabel = new Label("Pontuação Final: " + score + "/" + questions.length);
    root.getChildren().add(scoreLabel); // Adiciona a pontuação final
}

private int getSelectedIndex() {
    RadioButton selectedRadioButton = (RadioButton) toggleGroup.getSelectedToggle();
    return root.getChildren().indexOf(selectedRadioButton) - 1; // -1 para ignorar o Label da pergunta
}
```

```

public class Quiz extends Application {

    private int correnteQuestion = 0; // Índice da pergunta atual
    private int score = 0; // Pontuação do jogador

    // Perguntas e Respostas
    private String[] questions = {
        "Qual é a capital da França?",
        "Quem escreveu 'Dom Quixote'?",
        "Qual é o maior planeta do sistema solar?"
    };

    private String[][] options = {
        {"Paris", "Londres", "Roma", "Berlim"},
        {"Miguel de Cervantes", "William Shakespeare", "Mark Twain", "Jorge Amado"},
        {"Terra", "Marte", "Júpiter", "Saturno"}
    };

    private int[] correctAnswers = {0, 3, 1}; // Índices das respostas corretas

    private Label questionLabel;
    private ToggleGroup toggleGroup;
    private VBox root;

```

```

@Override
public void start(Stage primaryStage) throws Exception {

    primaryStage.setTitle("Quiz JavaFX");

    // Layout principal
    root = new VBox();
    root.setPadding(new Insets(10)); // Espaçamento interno do layout
    root.setSpacing(10); // Espaçamento entre os componentes

    // Mostrar a pergunta atual
    questionLabel = new Label(questions[currentQuestion]);
    questionLabel.getStyleClass().add(e:"label"); // Adiciona estilo CSS
    root.getChildren().add(questionLabel);

    // Grupo de botões de opção
    toggleGroup = new ToggleGroup();
    for (String option : options[currentQuestion]) {
        RadioButton radioButton = new RadioButton(option);
        radioButton.setToggleGroup(toggleGroup);
        radioButton.getStyleClass().add(e:"radio-button"); // Adiciona estilo CSS
        root.getChildren().add(radioButton);
    }

    // Botão de próxima pergunta
    Button nextButton = new Button("Próxima Pergunta");
    nextButton.getStyleClass().add(e:"button"); // Adiciona estilo CSS
    root.getChildren().add(nextButton);
    nextButton.setOnAction(event -> handleNextQuestion()); // Ação do botão

```

```
// Cena
Scene scene = new Scene(root, 400, 450);
scene.getStylesheets().add(getClass().getResource(name:"styles.css").toExternalForm()); // Adiciona o CSS
primaryStage.setScene(scene);
primaryStage.show();
```

```
.Label {
    -fx-font-size: 22px;
    -fx-padding: 15px;
    -fx-text-fill: #2e4053;
    -fx-font-weight: bold;
}

.radio-button {
    -fx-font-size: 18px;
    -fx-padding: 10px;
    -fx-background-color: #e8f8f5;
    -fx-border-color: #1abc9c;
    -fx-border-radius: 5px;
    -fx-pref-width: 300px;
}

.button {
    -fx-font-size: 18px;
    -fx-padding: 10px 30px;
    -fx-background-color: #3498db;
    -fx-text-fill: #ffffff;
    -fx-border-radius: 5px;
}
```

Quiz JavaFX

Qual é a capital da França?

☒ Paris

☐ Londres

☐ Roma

☐ Berlim

Próxima Pergunta

Quiz JavaFX

Pontuação Final: 3/3